Palco — Documentación del proyecto

Materia	Desarrollo de Aplicaciones II · UADE
Comisión	Viernes, 08:45 a 11:45
Integrantes	Santiago Loto, Sebastian Deya, Julian Zuchowicki, Valentina Frisoli, Joaquin Molina
Repositorio	https://github.com/santttii/deapps2
Versión	Borrador unificado del 16/10/2026

Estado del documento

Qué refleja	El código del commit `725b0ba` (16/10/2026) y la evidencia de `docs/entrega3/evidencia/`, tomada el 14/10 del servidor armado desde el tag `entrega-parcial-2`
De dónde sale	La Entrega Parcial N.º 1, el documento de integración de la Entrega Parcial N.º 2, los talleres de las clases 6, 8, 9, 10 y 11, la OpenAPI, la colección de Postman y el código. El anexo D lista cada documento
Para qué	Es el documento maestro. De acá salen el documento técnico de la Entrega Obligatoria N.º 2 (13/11, de 8 a 12 páginas) y la documentación final consolidada (10 a 15 páginas)
Estados	"Implementado": está en el código y se probó. "Simulado": funciona con un sustituto declarado. "Diseñado, pendiente": está diseñado y no implementado

Índice

1. Presentación
2. Requisitos y estado
3. Arquitectura general
4. Componentes
5. Decisiones de diseño
6. Patrones aplicados
7. Transacciones
8. Estrategia de integración (SOA)
9. API REST
10. Mensajería
11. Servicios web SOAP: Facturación con un ARCA simulado
12. Sistemas externos
13. Diagramas de secuencia
14. Seguridad
15. Frontend
16. Pruebas y evidencia
17. Cómo levantar y probar el sistema
18. Gestión del proyecto

19. Evolución del diseño
20. Qué es real y qué es simulado hoy
21. Limitaciones conocidas
22. Trabajo futuro

- Aprendizajes del equipo
- Declaración de uso de herramientas de IA generativa
- Anexos

1. Presentación

1.1 Qué es Palco y qué problema resuelve

Palco es una plataforma de venta de entradas para eventos (recitales, partidos, fiestas y otros espectáculos masivos) con **reventa oficial dentro de la misma plataforma**. Ataca dos problemas del mercado de entradas:

- **La reventa especulativa.** Cuando un evento se agota, las entradas reaparecen en canales informales a precios muy superiores al original. En Palco, quien no puede ir revende su entrada solo dentro de la plataforma y con un precio máximo fijo: el original más un 10 %. El tope lo fija la plataforma y el organizador no lo puede cambiar.
- **El fraude y la entrada duplicada.** Las entradas son **nominativas**: cada una lleva el nombre y el DNI de su titular, y en la puerta se valida que la entrada sea auténtica, que no se haya usado y que corresponda a la persona que la presenta.

Las reglas numéricas del dominio están en un solo lugar del código, `backend/common/src/main/java/com/palco/common/ReglasDeNegocio.java`, y valen para todos los servicios:

Regla	Valor	Constante
Tope de reventa sobre el precio original	+10 %	`TOPE_REVENTA_PORCENTAJE`
Precio mínimo de reventa	50 % del original	`MINIMO_REVENTA_PORCENTAJE`
Tasa de servicio sobre el subtotal	10 %	`TASA_SERVICIO_PORCENTAJE`
Máximo de entradas por compra	6	`MAX_ENTRADAS_POR_ORDEN`
Cierre de la reventa antes del evento	3 horas	`HORAS_LIMITE_REVENTA`
Duración de la reserva de cupo durante la compra	10 minutos	`MINUTOS_RESERVA`

Los montos se manejan en pesos enteros (`long`) en todo el backend, para evitar errores de redondeo.

1.2 Actores y roles

El sistema distingue cuatro roles. Los tres primeros son de personas y se guardan en `backend/servicios/usuarios/src/main/java/com/palco/usuarios/dominio/Rol.java` (`USER`,

`ORGANIZER`, `STAFF`); viajan en el token de sesión (JWT, claim `groups`) y son los que usan las anotaciones `@RolesAllowed` de cada recurso REST. El cuarto es técnico.

Actor	Rol en el sistema	Qué hace	Estado
Comprador	`user`	Se registra, verifica su email, compra entradas nominativas a su nombre o a nombre de otros titulares, consulta sus entradas. Con la identidad verificada puede además revender una entrada propia: no es un tipo de cuenta aparte sino una capacidad del comprador (el "vendedor" de la Entrega Parcial N.º 1).	Compra: implementado. Reventa: diseñado, pendiente
Organizador	`organizer`	Da de alta eventos con sus sectores, precios y cupos; el evento queda a su nombre. En el diseño, además, ve ventas, ocupación, monitor de reventa, accesos y liquidación.	Alta de eventos: implementado. Panel con datos reales: pendiente (el panel del frontend usa datos fijos)
Personal de puerta	`staff`	Escanea la entrada en el ingreso, la marca como usada y contrasta nombre y DNI del titular con el documento.	Diseñado, pendiente (la pantalla de puerta del frontend funciona con datos locales)
Otro servicio de Palco	`servicio`	Rol técnico para las llamadas entre servicios: por ejemplo, Ventas actualiza el cupo en Eventos o pide un cobro a Pagos. El token lo emite Usuarios en `POST /api/usuarios/servicios/token` y lo obtiene `common/seguridad/TokenDeServicio.java`.	Implementado

Los sistemas externos del dominio son la pasarela de pago (Mercado Pago, partner moderno por REST) y el sistema de facturación electrónica de AFIP/ARCA (legado, por SOAP). Hoy la pasarela está **simulada** (`backend/servicios/pagos/src/main/java/com/palco/pagos/pasarela/PagosFake.java`) y la integración con AFIP/ARCA está **diseñada, pendiente**.

1.3 Flujos principales

Compra nominativa con reserva y pago — Implementado (con pasarela simulada)

1. El comprador elige un evento del catálogo (`GET /api/eventos`, público) y las cantidades por sector.
2. Carga los titulares: nombre y DNI de cada entrada (hasta 6 por compra).
3. Ventas abre la compra (`POST /api/v1/ventas`): valida, **reserva el cupo** en Eventos con un UPDATE atómico (dos compras simultáneas del último lugar: pasa una sola; evidencia en `docs/entrega3/evidencia/compras-simultaneas.txt`) y calcula subtotal más tasa de servicio. La compra en curso vive en un componente **stateful** (`ServicioDeVentas`, `@Stateful` con `@StatefulTimeout` de 10 minutos).
4. El comprador paga (`POST /api/v1/ventas/{ventaId}/pagos`). Ventas cobra a través de Pagos, que hoy responde con la pasarela simulada. Si el cobro sale aprobado, una única transacción declarativa guarda la venta como confirmada y publica el mensaje `PAGO_CONFIRMADO`.
5. Las entradas se emiten de forma **asincrónica**: el mensaje llega por la cola `cola.emision-entradas` al consumidor de emisión (`EmisionDeEntradasMDB`), que pasa las entradas de `PENDIENTE_EMISION` a `VALIDA`. El mismo mensaje sale por el tópico `topico.compra-`

`confirmada` hacia el aviso al comprador (que hoy **se escribe en el log**, no se envía por email: simulado) y hacia la auditoría (suscripción durable, implementado).

6. Si el comprador cancela o pasan los 10 minutos sin pagar, el cupo se libera (`DELETE /api/v1/ventas/{ventaId}` o el barrido de `VencimientoDeReservas`, un `@Singleton` con `@Schedule` cada minuto).

Reventa oficial con tope +10 % — Diseñado, pendiente

El titular con identidad verificada publica una entrada propia con un precio entre el 50 % y el 110 % del original, hasta 3 horas antes del evento. Otro comprador la compra por la plataforma: se le cobra, se emite una entrada nueva a su nombre, la original queda invalidada y el vendedor recibe un aviso. La validación del tope se hace en el servidor, no solo en la interfaz. El servicio de Reventa solo tiene `GET /api/reventa/health`; los estados `PUBLICADA` y `VENDIDA` de la entrada ya existen en `ventas/dominio/EstadoEntrada.java`, y la pantalla de reventa del frontend (`frontend/src/pages/ResalePage.tsx`) funciona con datos locales.

Validación en puerta — Diseñado, pendiente

El personal de puerta escanea el QR (o ingresa el código a mano). El sistema responde válida, usada o inválida, marca la entrada como usada de forma atómica (aunque dos puertas la escaneen a la vez) y devuelve nombre y DNI del titular para contrastarlos con el documento. El servicio de Validación solo tiene `GET /api/validacion/health`; la pantalla `frontend/src/pages/GatePage.tsx` funciona con datos locales.

Alta de eventos — Implementado

El organizador crea el evento con su categoría, fecha, lugar y sectores con precio y cupo (`POST /api/eventos`, rol `organizer`). El servidor valida los datos, genera un slug único y no deja crear eventos que ya empezaron. La edición (`PUT`) y la cancelación de eventos están pendientes.

1.4 Alcance del trabajo práctico

Entra en el alcance:

- Los 8 componentes de negocio definidos en la Entrega Parcial N.º 1, cada uno como un WAR independiente con su propio esquema de base: Usuarios, Eventos, Ventas, Pagos, Validación, Reventa, Notificaciones y Facturación. Se comunican solo por REST o por JMS.
- Los cuatro flujos de la sección 1.3, de punta a punta, con el frontend React conectado al backend.
- La integración síncrona por REST (API propia y cliente de la pasarela de pago) y por SOAP (facturación contra el sistema legado), y la asincrónica por cola y tópico sobre el ActiveMQ Artemis embebido en WildFly.
- Seguridad declarativa por rol y transacciones declarativas en el flujo de compra.

Queda fuera del alcance (o se resuelve con una versión simulada):

- **Cobro real.** El 13/10 se decidió no integrar Mercado Pago real: Pagos sigue con `PagosFake` y el requisito de REST lo cubre la API propia. El diseño de la integración real queda en el diagrama `docs/entrega-obligatoria-2/diagramas/diagrama_secuencia_mercadopago.png`, y se retoma si la cátedra pide una pasarela real.
- **Envío real de emails.** El código de verificación y el aviso de compra quedan en el log del servidor; no hay proveedor de correo configurado.
- **Verificación de identidad con foto del DNI.** Al verificar el email, la cuenta queda con la identidad verificada; no hay validación documental.
- **Facturación contra el ambiente real de AFIP/ARCA.** El diseño prevé un servicio SOAP propio que modela el sistema legado.
- **Despliegue en la nube.** La consigna no lo exige; el sistema corre en un WildFly local con H2 en modo PostgreSQL. La migración a Supabase (PostgreSQL) está prevista, sin probar.
- Recuperación de contraseña, modo offline de la puerta, liquidación al vendedor y al organizador, y reembolsos.

2. Requisitos y estado

2.1 Cómo leer la tabla

La tabla reúne los requisitos de cada entrega de la consigna (sección 5) y los transversales obligatorios (sección 6), con su estado al **16/10**. Las fechas son las de la comisión del viernes; la consigna está escrita para la comisión del lunes y sus fechas se citan solo como referencia.

Entrega (nombre de la consigna)	Fecha del viernes	La consigna dice
Entrega Parcial N.º 1	04/09 (ya pasó)	31/08
Entrega Obligatoria N.º 1 — 1.º Parcial	18/09 (ya pasó)	14/09
Entrega Parcial N.º 2	23/10, **sin confirmar con la cátedra**	19/10
Entrega Obligatoria N.º 2 — 2.º Parcial	13/11	09/11
Entrega Final	Sistema completo para el 27/11; defensa el 04/12 o el 11/12 (a confirmar)	30/11 en el detalle, 04/12 u 11/12 en el encabezado

Estados: **Cumplido** (está en el código o en la evidencia y se puede mostrar), **Parcial** (una parte está hecha y otra no, o una pieza está simulada) y **Pendiente** (no hay código). Las rutas de código son relativas a `backend/servicios/<servicio>/src/main/java/com/palco/<servicio>/` salvo que se indique otra cosa.

2.2 Requisitos por entrega

#	Requisito	Se exige en	Estado al 16/10	Dónde se ve
E1.1	Componentes candidatos (mínimo 6) con su interfaz: nombre, operaciones y responsabilidad	Entrega Parcial N.º 1 (04/09)	Cumplido	`docs/entrega1/Entrega_Parcial_1_Arquitectura.docx`, sección 2 (8 componentes con interfaz `I<Servicio>`)
E1.2	Diagrama de arquitectura general	Entrega Parcial N.º 1 (04/09)	Cumplido	`docs/entrega1/Diagrama_Capas_EntradasEventos.drawio.png`
E1.3	Stack tecnológico elegido y justificado	Entrega Parcial N.º 1 (04/09)	Cumplido	Documento de la Entrega Parcial N.º 1, sección 4
E1.4	Un componente completo desplegado en un servidor real, con capas separadas, y su justificación stateful o stateless	Entrega Parcial N.º 1 (04/09)	Cumplido	`ServicioDeEventos` (`@Stateless`) en WildFly; justificación en la sección 5 del documento
E1.5	Documento de 2 a 4 páginas	Entrega Parcial N.º 1 (04/09)	Cumplido	`docs/entrega1/Entrega_Parcial_1_Arquitectura.docx`
O1.1	Al menos 3 componentes implementados y desplegados, cada uno en capas	Entrega Obligatoria N.º 1 (18/09)	Cumplido hoy (5 componentes). Si se cumplía al 18/09 no se puede ver en el repo: el historial de git tiene 3 commits hasta el 17/09 y el resto desde el 02/10	Usuarios, Eventos, Ventas, Pagos y Notificaciones en `backend/servicios/`
O1.2	Un componente stateful y uno stateless, con ciclo de vida gestionado por el contenedor	Entrega Obligatoria N.º 1 (18/09)	Cumplido	`ventas/negocio/ServicioDeVentas.java` (`@Stateful`, `@StatefulTimeout`, `@PostConstruct`, `@PreDestroy`, `@Remove`); `eventos/negocio/ServicioDeEventos.java` y `usuarios/negocio/ServicioDeUsuarios.java` (`@Stateless`)
O1.3	Al menos 3 patrones de diseño aplicados y justificados por escrito	Entrega Obligatoria N.º 1 (18/09)	Parcial: aplicados en el código, sin documento que los justifique en el repo	Factory: `pagos/pasarela/FabricaDePasarela.java` y `ventas/negocio/externos/FabricaDeCobros.java`. Adapter: `ventas/negocio/externos/ClienteDePagos.java`. Repository (DAO): `ventas/datos/VentaRepository.java`, `pagos/datos/PagoRepository.java`. Interfaz con implementaciones intercambiables: `pagos/pasarela/PasarelaDePagos.java`
O1.4	Seguridad declarativa mínima en una operación sensible	Entrega Obligatoria N.º 1 (18/09)	Cumplido	`@RolesAllowed("organizer")` en `POST /api/eventos` (`eventos/api/EventosResource.java`)

#	Requisito	Se exige en	Estado al 16/10	Dónde se ve
O1.5	Documento técnico de 5 a 8 páginas	Entrega Obligatoria N.º 1 (18/09)	No está en el repo	—
P2.1	Diagrama de arquitectura de integración (SOA)	Entrega Parcial N.º 2 (23/10, sin confirmar)	Cumplido	`docs/entrega3/Diagrama_Integracion_SOA.svg`
P2.2	Clasificación justificada de cada integración externa en síncrona o asincrónica	Entrega Parcial N.º 2 (23/10, sin confirmar)	Cumplido	`docs/entrega3/Entrega_Parcial_2_Integracion.md`
P2.3	Un proceso de negocio de punta a punta con mensajería real y broker configurado	Entrega Parcial N.º 2 (23/10, sin confirmar)	Cumplido	Compra → `PAGO_CONFIRMADO` → emisión de entradas. Broker: Artemis embebido, destinos en `backend/servidor/src/main/wildfly/configurar.cli`. Evidencia en `docs/entrega3/evidencia/` (tag `entrega-parcial-2`)
O2.1	Dos procesos con mensajería de punta a punta: uno por cola y uno por tópico	Entrega Obligatoria N.º 2 (13/11)	Cumplido, con el aviso al comprador simulado (se escribe en el log)	Cola: `ventas/mensajeria/EmisionDeEntradasMDB.java`. Tópico: `notificaciones/mensajeria/AvisoDeCompraMDB.java` (no durable) y `AuditoriaDeComprasMDB.java` (durable). Evidencia: `docs/entrega3/evidencia/auditoria.txt`, `suscriptor-durable.txt`, `mensajes-pendientes.txt`
O2.2	Un servicio SOAP con su WSDL, justificado como sistema legado	Entrega Obligatoria N.º 2 (13/11)	Pendiente	No hay `@WebService` en `backend/` ni la capa `webservices` en `backend/servidor/pom.xml`
O2.3	Un servicio REST (partner moderno o API de consumo externo)	Entrega Obligatoria N.º 2 (13/11)	Cumplido como API propia; el partner moderno (Mercado Pago) está simulado	`docs/api/openapi.yaml`, colección de Postman en `docs/api/`, `pagos/pasarela/PagosFake.java`
O2.4	Transacciones declarativas en un flujo crítico de varios pasos	Entrega Obligatoria N.º 2 (13/11)	Cumplido; falta una prueba repetible en el repo de la falla a mitad del flujo	`ServicioDeVentas` confirma la venta con `@TransactionAttribute(REQUIRED)` y publica con `common/mensajeria/PublicadorPagoConfirmadoJms.java` (`MANDATORY`, por `java:/JmsXA`), así que la venta y el mensaje se confirman o se deshacen juntos
O2.5	Documento técnico de 8 a 12 páginas con un diagrama de secuencia por integración	Entrega Obligatoria N.º 2 (13/11)	Parcial	Solo existe el diagrama de secuencia de Mercado Pago (diseño): `docs/entrega-obligatoria-2/diagramas/`. Este documento es la base del técnico
F.1	Sistema completo integrado, con el checklist de la sección 6 cumplido	Entrega Final (27/11)	Parcial	5 de 8 componentes implementados; faltan Validación, Reventa y Facturación

#	Requisito	Se exige en	Estado al 16/10	Dónde se ve
F.2	Documentación final de 10 a 15 páginas	Entrega Final (27/11)	Pendiente	Se recorta de este documento
F.3	Repositorio git con historial incremental y participación visible de todos	Entrega Final (27/11)	Parcial	Ver T.11
D.1	Declaración del uso de IA en el documento de cada entrega obligatoria (consigna, sección 9)	Entregas obligatorias (13/11 y final)	Pendiente	La redacta el grupo

2.3 Requisitos transversales (sección 6 de la consigna)

Se exigen todos para la Entrega Final; varios ya se piden antes, en la entrega que figura en la tabla anterior.

#	Requisito	Estado al 16/10	Dónde se ve
T.1	Mínimo 6 componentes de negocio, cada uno con interfaz explícita y documentada	Parcial: 8 componentes diseñados con su interfaz; las interfaces existen en código en 5	`IUsuarios`, `IEventos`, `IVentas`, `IPagos`, `INotificaciones` (y `IAuditoria`) en `negocio/` de cada servicio. Validación, Reventa y Facturación solo tienen la clase de aplicación JAX-RS y el `health`. Interfaces documentadas en `docs/entrega1/`
T.2	Un componente stateful y uno stateless, justificados por escrito	Cumplido	`ServicioDeVentas` (`@Stateful`) y `ServicioDeEventos` (`@Stateless`); la justificación está en el comentario de cada clase y en el documento de la Entrega Parcial N.º 1
T.3	Arquitectura en capas (presentación, negocio, datos) en cada componente implementado	Parcial	Ventas y Pagos tienen `api/`, `negocio/`, `datos/` (repositorios) y `dominio/`. Usuarios y Eventos no tienen `datos/`: el Session Bean usa el `EntityManager` directamente, y la capa de datos es JPA con las entidades de `dominio/`
T.4	Al menos 3 patrones de diseño distintos, aplicados y justificados	Parcial	Ver O1.3: Factory, Adapter y Repository están en el código; la justificación escrita está solo en los comentarios de las clases
T.5	Integración SOAP con WSDL, justificada como sistema legado	Pendiente	Ver O2.2
T.6	Integración REST con partner moderno o API de consumo externo	Cumplido como API de consumo externo; Mercado Pago simulado	Ver O2.3
T.7	Dos procesos asincrónicos: uno por cola y uno por tópico	Cumplido	Ver O2.1; contrato en `docs/api/mensajeria-pago-confirmado.md`

#	Requisito	Estado al 16/10	Dónde se ve
T.8	Seguridad declarativa (autenticación y autorización por rol) en al menos 2 operaciones sensibles	Cumplido	`@RolesAllowed` / `@PermitAll` en cada operación: `organizer` en el alta de eventos, `user` en las 6 operaciones de Ventas, `servicio` en el cupo, en Pagos y en Notificaciones. JWT RS256 firmado por Usuarios y validado con MicroProfile JWT. 401 sin token y 403 con rol incorrecto. Desde el 16/10, implementado y probado en vivo: el 401 por token inválido sale en Problem Details (`/problemas/token-invalido`) con `WWW-Authenticate: Bearer error="invalid_token"` y CORS (`common/rest/ErrorDeAutenticacionServlet.java` y `common/src/main/resources/META-INF/web-fragment.xml`); 5 logins fallidos en 15 minutos para un mismo email dan 429 con `Retry-After` (`usuarios/negocio/IntentosDeLogin.java`, migración V4); el 500 devuelve un `idError` que se busca en el log; PBKDF2 a 600.000 iteraciones. Diseñado, pendiente: vigencia del token y permisos por destino en el broker, las dos a decidir en grupo
T.9	Transacciones declarativas en un flujo crítico de varios pasos	Cumplido	Ver O2.4
T.10	Stack consistente y justificado (Jakarta EE, Spring Boot, .NET, etc.)	Cumplido	Jakarta EE 11 sobre WildFly 41, Java 25, Maven; frontend React + Vite + TypeScript. Justificación en la Entrega Parcial N.º 1, sección 4
T.11	Repositorio git con historial incremental	Parcial	30 commits entre el 16/09 y el 14/10, concentrados en cuatro días (02/10, 06/10, 13/10 y 14/10). Autores: santttii, vfrisoli, Sebastian Deya, Julián Zuchowicki y una identidad "Improvement". No hay commits a nombre de Joaquin Molina. Tag `entrega-parcial-2`

2.4 Desafíos opcionales (sección 7 de la consigna)

Desafío	Estado al 16/10	Dónde se ve
Resiliencia ante la caída de un sistema externo	Parcial	Si un consumidor de mensajes se detiene, la compra sigue y los mensajes esperan en el broker (`docs/entrega3/evidencia/mensajes-pendientes.txt`); hasta 5 entregas con 2 segundos entre reintentos y después la cola de mensajes fallidos (`java:/jms/queue/DLQ`), configurados en `configurar.cli`. Sin Circuit Breaker
Heterogeneidad tecnológica (un componente en otra tecnología)	Pendiente	Todos los servicios son Jakarta EE
Prueba de escalabilidad horizontal	Pendiente	—
Al menos 2 ADR	Parcial: un borrador, sin decidir	Borrador sobre la vigencia del token, a decidir en grupo. El resto de las decisiones no tiene formato ADR

3. Arquitectura general

3.1 Stack

Pieza	Tecnología y versión	Para qué se usa
Lenguaje	Java 25 (`maven.compiler.release` 25 en `backend/pom.xml`)	Todo el backend
Plataforma	Jakarta EE 11 (`jakarta.jakartaee-api` 11.0.0)	JAX-RS, CDI, EJB (Session Beans y MDB), JPA, Bean Validation, JMS, JSON-B
Servidor	WildFly 41.0.1.Final, armado con Galleon	Contenedor de los 8 WAR
Construcción	Maven multimódulo: `common`, `servicios` (un módulo por servicio) y `servidor`	Compila, prueba y arma el servidor con los WAR adentro
Base de datos	H2 en memoria en modo PostgreSQL en desarrollo; PostgreSQL (Supabase) después, cambiando solo variables de entorno	Un esquema por servicio
Migraciones	Flyway 13.9.0, scripts en `src/main/resources/db/migracion` de cada servicio	Crean y actualizan el esquema al desplegar
Seguridad	MicroProfile JWT 2.2 (capa `microprofile-jwt` de WildFly), tokens firmados con RSA por Usuarios. Cada servicio verifica la firma con la clave pública que descarga de `/api/usuarios/jwks` y exige el emisor `palco` (`common/src/main/resources/META-INF/microprofile-config.properties`; la dirección de Usuarios sale de `PALCO_URL_INTERNA`)	`@RolesAllowed` / `@PermitAll` en cada operación
Mensajería	ActiveMQ Artemis embebido en el mismo WildFly (capa `embedded-activemq`)	Cola de emisión de entradas y tópico de compras confirmadas
Frontend	React + Vite + TypeScript (`frontend/`)	SPA que consume la API REST

Por qué este stack. La Entrega Parcial N.º 1 eligió Jakarta EE porque es el stack de los laboratorios de la materia (WildFly), da soporte nativo a lo que pide el TP como requisito transversal (transacciones y seguridad declarativas, ciclo de vida manejado por el contenedor) y es portable entre servidores compatibles. Esa elección se mantuvo. H2 en modo PostgreSQL permite trabajar sin instalar una base, con la condición de que todo el SQL de las migraciones corra en las dos.

Estado del frontend. Sesión, catálogo, alta de eventos y compra usan el backend real. Reventa, puerta (validación) y estadísticas del organizador siguen con datos locales (`localStorage` y datos fijos): **Simulado** en el cliente, pendiente de backend.

3.2 Ocho WAR independientes

Cada componente de negocio es un WAR propio (`backend/servicios/<servicio>`), con su ruta (`context-root` en `WEB-INF/jboss-web.xml`) y, si guarda datos, su propio esquema (`hibernate.default_schema` en `META-INF/persistence.xml`). Todos comparten un único datasource de WildFly (`java:jboss/datasources/PalcoDS`), pero cada uno lee y escribe solo su esquema.

Servicio	Ruta base	Esquema	Estado
Usuarios	`/api/usuarios`	`usuarios`	Implementado
Eventos	`/api/eventos`	`eventos`	Implementado

Servicio	Ruta base	Esquema	Estado
Ventas	`/api/v1` (`/ventas`, `/entradas`)	`ventas`	Implementado
Pagos	`/api/v1/pagos`	`pagos`	Implementado, con pasarela **simulada** (`PagosFake`)
Notificaciones	`/api/notificaciones`	`notificaciones`	Implementado; el aviso al comprador es **simulado** (se escribe en el log)
Validación	`/api/validacion`	no tiene	Diseñado, pendiente: solo `health`
Reventa	`/api/reventa`	no tiene	Diseñado, pendiente: solo `health`
Facturación	`/api/facturacion`	no tiene	Diseñado, pendiente: solo `health`

Por qué un WAR por componente. Hace visible en el despliegue el límite entre componentes: un servicio no puede usar las clases de otro porque no están en su WAR, y cada uno se puede dar de baja o redesplegar solo (la evidencia de la Entrega Parcial N.º 2 lo usa para mostrar qué pasa cuando Pagos no está). El costo es más configuración que en un único WAR, y que entre servicios no hay transacción compartida: cada llamada REST queda confirmada en el otro servicio apenas responde, y quien llama se hace cargo de compensar (por ejemplo, Ventas devuelve el cupo a Eventos si la compra no se completa; ver `ClienteDeEventos`).

Rutas. Usuarios, Eventos y Notificaciones nacieron en `/api/<servicio>`. Ventas y Pagos se publicaron versionados (`/api/v1`). La decisión vigente es que los servicios nuevos nazcan versionados: Validación en `/api/v1/validacion` y Reventa en `/api/v1/reventas`; Facturación sigue en `/api/facturacion`. Hoy Validación y Reventa todavía responden en su ruta vieja (`/api/validacion`, `/api/reventa`), porque solo existe su esqueleto: el cambio de ruta llega con su implementación.

Regla de integración. Los servicios se hablan solo por REST o por JMS, nunca por clases o tablas del otro (`BaseDeDatos` en `common` lo deja escrito). Las llamadas entre servicios llevan un token con rol `servicio`, que consigue `TokenDeServicio` presentándole a Usuarios un secreto compartido (credenciales de cliente; token de 5 minutos). En desarrollo, el servidor arranca con un `PALCO_SECRETO_SERVICIOS` propio guardado en `backend/.env`, que no se versiona; los guiones de demo, escenarios y evidencia lo leen de la variable o de ese archivo, y la colección de Postman, de su variable `secretoServicios`. Que fuera de desarrollo el servidor no arranque sin un secreto propio está propuesto y no implementado. La única excepción a "nada compartido" es `backend/common`, que no es un servicio sino una biblioteca que va adentro de cada WAR.

3.3 Capas dentro de cada servicio

Todos los servicios siguen la misma estructura de paquetes, tomada de Eventos y Usuarios:

Paquete	Qué contiene	Regla
`api/`	Recurso JAX-RS y DTOs	Capa delgada: valida la entrada (Bean Validation), declara el rol y delega. No tiene lógica de negocio
`negocio/`	Interfaz `I<Servicio>` (`@Local`) y su Session Bean; migraciones (`Migraciones<Servicio>`, `@Singleton` `@Startup`); clientes de otros servicios en `negocio/externos/`	Acá viven las reglas y las transacciones

Paquete	Qué contiene	Regla
`datos/`	Repositorios (`<Entidad>Repository` + implementación JPA)	Solo en Ventas y Pagos; Eventos, Usuarios y Notificaciones usan el `EntityManager` desde el Session Bean
`dominio/`	Entidades JPA y enums de estado	Sin dependencias de JAX-RS
`mensajeria/`	Message-Driven Beans	Puntos de entrada delgados, como el recurso REST: leen el mensaje y delegan

Pagos tiene además `pasarela/` (la interfaz `PasarelaDePagos`, su fábrica y el simulador `PagosFake`), que es su capa de integración con el exterior.

3.4 `backend/common`

Biblioteca compartida (`palco-common`) que va dentro de cada WAR. No es un servicio y no tiene estado propio.

Clase	Qué hace
`ReglasDeNegocio`	Fuente única de las reglas: tope de reventa (+10 %), mínimo de reventa (50 %), cargo de servicio (10 %), máximo de 6 entradas por orden, cierre de la reventa 3 horas antes, reserva de 10 minutos, validación de DNI. Montos en pesos enteros (`long`). El frontend tiene una copia en `frontend/src/types/index.ts` que tiene que coincidir
`ErroresDeNegocio`	Excepciones de negocio (`@ApplicationException(rollback = true)`) que el negocio lanza sin depender de JAX-RS. Incluye `DemasiadosIntentos`, que lleva los segundos de espera (nunca menos de 1)
`rest/ManejadoresDeErrores`, `rest/Problema`	Traducen cada excepción a un código HTTP con cuerpo Problem Details (RFC 9457, `application/problem+json`). Nunca se devuelve un stack trace. `DemasiadosIntentos` sale como 429 con la cabecera `Retry-After`. El 500 (`Inesperado`) genera un UUID, lo escribe en el log junto con la traza y lo devuelve en el miembro de extensión `idError`; los demás errores no lo llevan
`rest/ErrorDeAutenticacionServlet` y `META-INF/web-fragment.xml`	El 401 por un token que no se puede verificar (firma alterada, mal formado) lo emite el mecanismo MP-JWT del servidor antes de llegar a JAX-RS. El `web-fragment.xml` del jar registra este servlet como página de error del 401 en los 8 servicios, y responde `application/problem+json` (tipo `token-invalido`) con `WWW-Authenticate: Bearer error="invalid_token"` y las cabeceras CORS. Pedida directo, la ruta responde 404
`rest/JsonProvider`	JSON-B en lugar del Jackson de RESTEasy: fechas ISO, `null` incluidos, y JSON mal formado como 400 con el mismo formato de error
`rest/HealthResource`	`GET <ruta del servicio>/health`: 200 si el servicio llega a la base, 503 si no
`rest/CorsFilter`	Orígenes permitidos desde `PALCO_CORS_ORIGENES`. Las cabeceras se arman en `cabecerasPara`, que usa también `ErrorDeAutenticacionServlet`: hay una sola lista de orígenes y cabeceras
`seguridad/Sesion`, `seguridad/TokenDeServicio`	Datos del usuario sacados del JWT; obtención y renovación del token de rol `servicio`
`mensajeria/PagoConfirmadoMensaje`, `DestinosJms`, `PublicadorPagoConfirmado(Jms)`, `MensajeInvalido`	El contrato del mensaje `PAGO_CONFIRMADO` (contrato en `docs/api/mensajeria-pago-confirmado.md`), los nombres JNDI de la cola y el tópico, y el publicador, que exige transacción (`MANDATORY`) para que el mensaje forme parte de la transacción de la venta

Clase	Qué hace
`datos/BaseDeDatos`, `Textos`	Acceso a la base compartida y utilidades de texto

Por qué existe. Las reglas de negocio y el contrato del mensaje tienen que ser los mismos para quien publica y quien consume; duplicarlos en cada servicio permitiría que diverjan. Lo que va en `common` es contrato y plomería transversal, nunca lógica de un servicio en particular.

3.5 Armado y despliegue del servidor

El módulo `backend/servidor` arma un WildFly a medida con el `wildfly-maven-plugin` y Galleon, en `servidor/target/server`. Usa los feature packs `wildfly-galleon-pack` (41.0.1.Final) y `wildfly-datasources-galleon-pack` (11.5.0.Final) y solo estas capas:

Capa	Para qué
`jaxrs-server`	REST, CDI, JPA, Bean Validation y transacciones
`ejb`	Session Beans completos: `@Stateful`, `@Schedule` y Message-Driven Beans
`embedded-activemq`	Broker JMS dentro del mismo servidor
`microprofile-jwt`	Validación de los tokens que firma Usuarios
`h2-driver`, `postgresql-driver`	Drivers de las dos bases

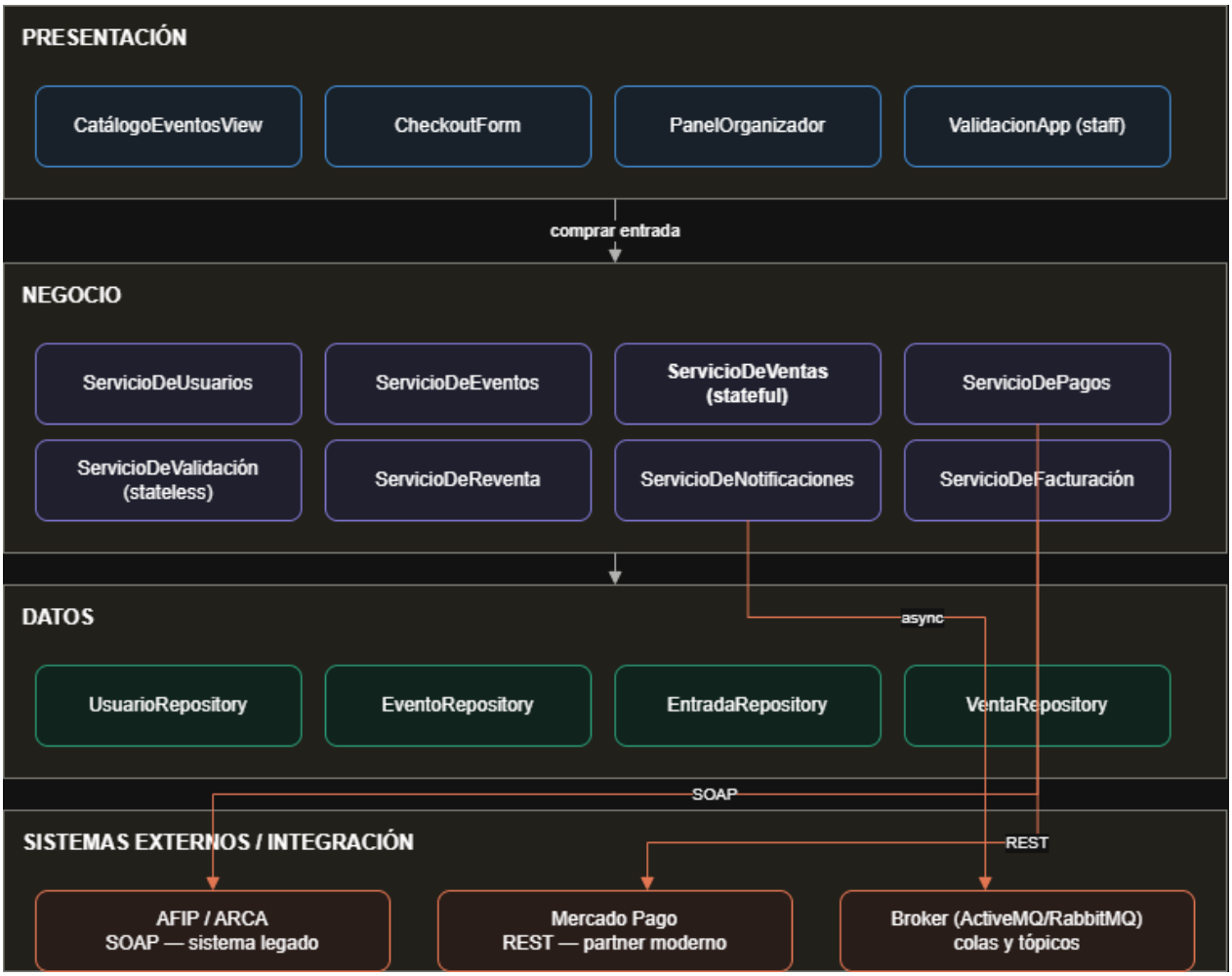
Después de provisionar, aplica `backend/servidor/src/main/wildfly/configurar.cli`, que deja versionada toda la configuración:

- **Datasource `PalcoDS`**: H2 en memoria (`MODE=PostgreSQL`) por defecto; con `PALCO_DB_DRIVER=postgresql`, `PALCO_DB_URL`, `PALCO_DB_USUARIO` y `PALCO_DB_PASSWORD` apunta a Supabase sin tocar los servicios. Pool de hasta 10 conexiones, compartido por los 8 servicios (por eso Pagos y el barrido de reservas evitan transacciones largas).
- **Mensajería**: la cola `cola.emision-entradas` (`java:/jms/queue/emisionEntradas`, punto a punto) y el tópico `topico.compra-confirmada` (`java:/jms/topic/compraConfirmada`, publicación/suscripción). Reintentos para todos los destinos: hasta 5 entregas con 2 segundos entre una y otra, y después la cola de mensajes fallidos (DLQ) que trae WildFly.
- **Seguridad**: `resteasy-role-based-security=true`, para que `@RolesAllowed` y `@PermitAll` se apliquen en los recursos REST. Los roles salen del claim `groups` del JWT.

El servidor tiene como dependencias los 8 WAR y los despliega. El script corre una sola vez, cuando se arma el servidor: un cambio en `configurar.cli` se aplica con `jboss-cli` sobre el servidor en marcha o rearmando con `mvn clean install` (lo que borra los mensajes guardados del broker y, con H2 en memoria, los datos). En desarrollo, `backend/dev.sh` levanta el servidor con los 8 servicios.

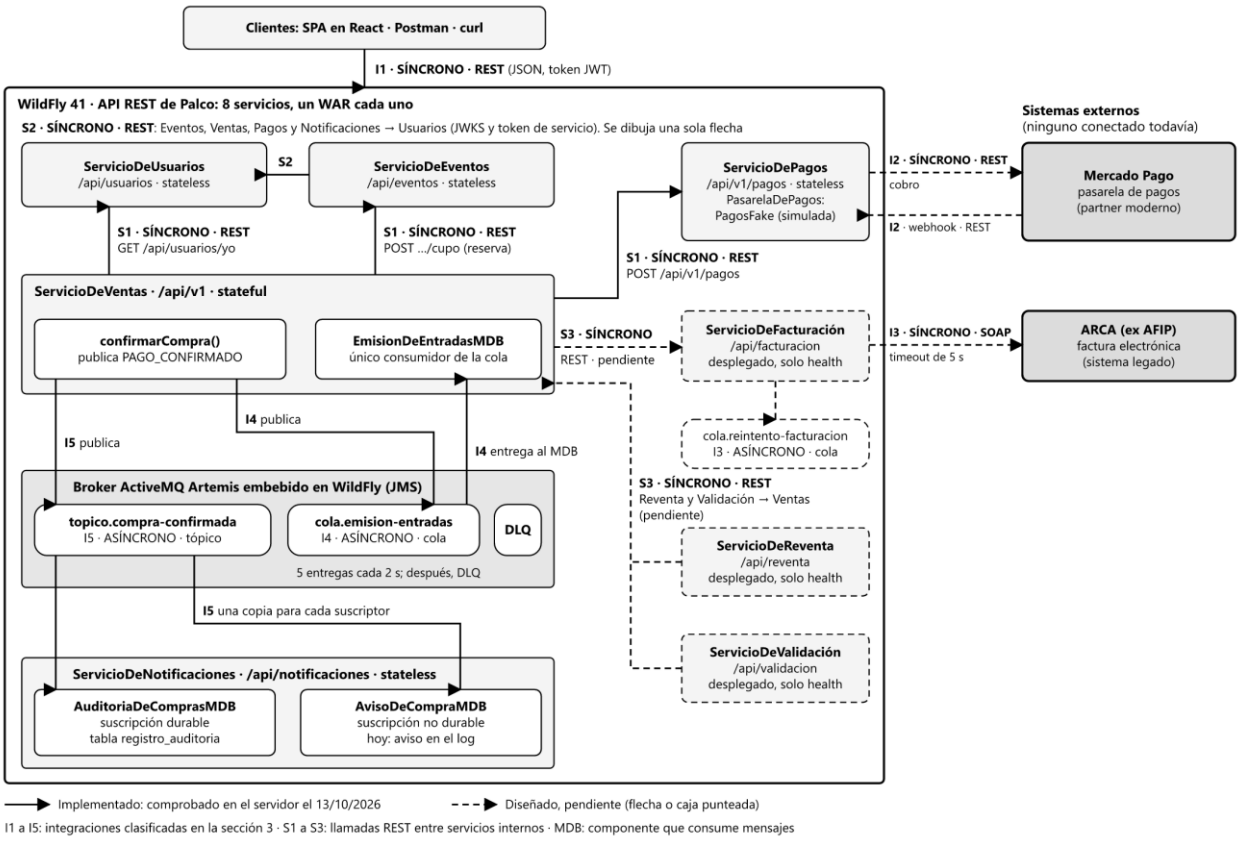
3.6 Diagramas

Diagrama de capas de la Entrega Parcial N.º 1 (entregada el viernes 04/09; la consigna dice 31/08):



Representa la arquitectura lógica en tres capas (presentación, negocio, datos) más una capa de integración con tres sistemas externos: ARCA (ex AFIP) por SOAP para factura electrónica, Mercado Pago por REST y un broker de mensajería para los procesos asincrónicos. La flecha marca el recorrido del caso de uso "comprar entrada". Es un diagrama de diseño: en esa fecha solo estaba implementado ServicioDeEventos. Lo que cambió desde entonces: la capa de negocio quedó repartida en 8 WAR con un esquema cada uno; ARCA y Mercado Pago siguen sin conexión real (Facturación está pendiente y Pagos usa una pasarela simulada); el broker sí está funcionando.

Diagrama de integración SOA de la Entrega Parcial N.º 2 (`docs/entrega3/`, para la entrega del 23/10, fecha sin confirmar con la cátedra):



Lleva el título "arquitectura de integración (SOA) al 13/10/2026": es una foto del sistema en esa fecha. Muestra los 8 servicios desplegados en WildFly, cada uno con su ruta y si es stateless o stateful; el broker Artemis con la cola `cola.emision-entradas` (I4, asincrónica, con su único consumidor `EmisionDeEntradasMDB`), el tópico `topico.compra-confirmada` (I5, con los dos suscriptores de Notificaciones) y la DLQ; las llamadas REST síncronas entre servicios (S1: Ventas hacia Usuarios, Eventos y Pagos; S2: los servicios hacia Usuarios por la clave pública y el token de servicio) y los dos sistemas externos, Mercado Pago (I2) y ARCA (I3, SOAP con timeout de 5 s y `cola.reintento-facturacion`). Las flechas llenas están implementadas y comprobadas en el servidor el 13/10; las punteadas (Facturación, Reventa y Validación, Mercado Pago, ARCA) están diseñadas y pendientes. El diseño de Facturación (sección 4.8) propone que la factura se dispare desde el tópico y no por REST dentro de la compra; si se aprueba, ese tramo del diagrama cambia.

4. Componentes

Los ocho componentes salen de la Entrega Parcial N.º 1. Cinco están implementados y tres tienen solo el esqueleto (`<Servicio>Application`, `pom.xml` y el `health` de `common`). En las tablas de endpoints, "rol" es la anotación de seguridad de la operación en el recurso JAX-RS. Todos los servicios, incluidos los tres pendientes, responden `GET <ruta>/health` sin token.

4.1 Usuarios

- **Responsabilidad:** identidad, autenticación y roles. Registra usuarios, verifica el email con un código, inicia sesión y firma los tokens JWT que validan los demás servicios. También emite el token de rol `servicio` para las llamadas entre servicios.

- **Interfaz:** `IUsuarios` (`@Local`): `registrar`, `verificarEmail`, `reenviarCodigo`, `iniciarSesion`, `obtener`, `emitirTokenDeServicio`. El comentario de la interfaz lo presenta como "el servicio reutilizable del taller de integración: Ventas, Reventa, Validación y el panel del organizador dependen de él".
- **Stateless.** `ServicioDeUsuarios`: "cada operación recibe todo lo que necesita (los datos del pedido o el id del token) y lo que tiene que durar queda en la base, así que el contenedor reutiliza un pool de instancias".
- **Seguridad propia:** contraseñas con hash PBKDF2-HMAC-SHA256 (`seguridad/Contrasenas`; los hashes nuevos usan 600.000 iteraciones, `Contrasenas.ITERACIONES`, y los anteriores, de 310.000, siguen verificando porque el formato guarda las iteraciones de cada hash), claves RSA para firmar los JWT (`seguridad/ClavesJwt`) y publicación de la clave pública en `/jwks`. El código de verificación admite hasta 5 intentos (`MAX_INTENTOS_CODIGO`).
- **Límite de intentos de login (SEC-W04):** con 5 fallidos en 15 minutos para un mismo email (`MAX_INTENTOS_LOGIN` y `VENTANA_INTENTOS_LOGIN`, en `ServicioDeUsuarios`), el pedido siguiente responde 429 `demasiados-intentos` con `Retry-After`, aunque la clave sea correcta; vale también para emails que no tienen cuenta, así el límite no revela cuáles existen. Los intentos los guarda y consulta `IntentosDeLogin` (Session Bean stateless: los intentos viven en la base y no en memoria, así el límite es el mismo con varias instancias o después de un redespliegue). Un login correcto borra los fallidos de ese email. El 401 del login es `CredencialesIncorrectas`, que no deshace la transacción (`rollback = false`) para que el intento fallido quede guardado.

Método	Ruta	Rol	Qué hace
POST	`/api/usuarios/registro`	`@PermitAll`	Alta de usuario; queda pendiente de verificar el email
POST	`/api/usuarios/verificacion`	`@PermitAll`	Verifica el código y devuelve la sesión
POST	`/api/usuarios/verificacion/reenvio`	`@PermitAll`	Reenvía el código
POST	`/api/usuarios/login`	`@PermitAll`	Inicia sesión y devuelve el JWT; 401 con credenciales incorrectas y 429 al superar el límite de intentos
GET	`/api/usuarios/yo`	`user`, `organizer`, `staff`	Datos del dueño del token (Ventas la usa para saber si el comprador verificó su identidad)
POST	`/api/usuarios/servicios/token`	`@PermitAll` (lo protege el secreto compartido)	Token de rol `servicio`
GET	`/api/usuarios/jwks`	`@PermitAll`	Clave pública para validar los JWT

- **Datos (esquema `usuarios`):** `usuario` (nombre, apellido, DNI único, email único, teléfono, fecha de nacimiento, hash de la contraseña, rol, email verificado, identidad verificada), `codigo_verificacion` (hash del código, vencimiento, intentos) e `intento_login` (SHA-256 del email normalizado y momento del intento fallido, con un índice por email y momento). V2 y V3 cargan usuarios de prueba; V4 crea `intento_login`; V5 rehashea las cuentas de prueba a 600.000 iteraciones (contraseña sin cambios, y solo si la fila conserva el hash original), para que su verificación no tarde distinto que la del hash falso que se calcula para un email inexistente y el tiempo de respuesta no deje ver qué emails existen.

- **Integraciones:** la consumen el frontend (sesión) y Ventas (`ClienteDeUsuarios`, con el token del propio comprador). Todos los servicios dependen de sus tokens.
- **Estado:** Implementado y probado en el servidor el 16/10. La sesión del usuario dura 8 horas (`DURACION_SESION`) y el token de servicio, 5 minutos. El pedido sin token a una operación con `@RolesAllowed` responde 401 con Problem Details (`ManejadoresDeErrores`); el 401 por token inválido, que rechaza el propio servidor antes de llegar al recurso, también sale como Problem Details (`ErrorDeAutenticacionServlet`, sección 3.4), y el 500 lleva `idError`. Verificación del 16/10, con el servidor armado desde cero: Usuarios desplegado en la migración v5; seis logins fallidos de un mismo email dieron cinco 401 y, el sexto, 429 con `Retry-After: 898`; una cuenta recién registrada completó registro, verificación, login y compra hasta `EMITIDA`. Los 22 tests de Usuarios pasan. Diseñado, pendiente: la vigencia de la sesión (SEC-W05) está en discusión, sin decidir.

4.2 Eventos

- **Responsabilidad:** catálogo de eventos y sus sectores (precio y cupo). Alta de eventos por el organizador y control del cupo vendido.
- **Interfaz:** `IEventos`: `listarEventos`, `obtenerDetalle`, `crearEvento`, `actualizarCupo`.
- **Stateless.** `ServicioDeEventos`: "cada operación es autocontenida y no depende de la llamada anterior, así que el contenedor reutiliza un pool de instancias". Fue el primer componente completo, presentado en la Entrega Parcial N.º 1 con esa misma justificación.

Método	Ruta	Rol	Qué hace
GET	`/api/eventos?categoria=&q=`	`@PermitAll`	Lista eventos, con filtro por categoría y búsqueda
GET	`/api/eventos/{slug}`	`@PermitAll`	Detalle de un evento con sus sectores
POST	`/api/eventos`	`organizer`	Alta de evento
POST	`/api/eventos/{eventoId}/sectores/{sectorId}/cupo`	`servicio`	Reserva o devuelve lugares de un sector (la llama Ventas)

- **Datos (esquema `eventos`):** `evento` (slug único, título, categoría, lugar, fecha, descripción, destacado, imagen, organizador) y `sector` (nombre, precio en pesos enteros, cupo, vendidas). La base garantiza que `vendidas` esté entre 0 y el cupo (`sector_vendidas_validas`): ese chequeo es la última red contra la sobreventa. V2 a V4 cargan eventos iniciales, corren sus fechas y agregan fotos.
- **Integraciones:** frontend (catálogo y alta) y Ventas (cupo, con token de servicio).
- **Estado:** Implementado.

4.3 Ventas

- **Responsabilidad:** la compra de punta a punta: reserva de lugares, datos de los titulares, cobro, confirmación y emisión de las entradas nominativas. Es dueña de las entradas.
- **Interfaz:** `IVentas`: `iniciarCompra`, `confirmarCompra`, `cancelarCompra`. Las tres son pasos de una misma conversación y se llaman sobre la misma instancia, por eso `confirmarCompra` y `cancelarCompra` no reciben el id de la venta. La Entrega Parcial N.º 1 listaba además `agregarEntrada`; en la implementación los titulares llegan todos juntos en `iniciarCompra` y esa operación no existe.
- **Stateful.** `ServicioDeVentas`: "una instancia por compra en curso, que el contenedor mantiene viva entre el pedido que abre la compra y el que la paga o la cancela. Guarda lo que todavía no está en la base: cuál es su venta y a nombre de quién va cada entrada (los titulares). [...] Un stateless no serviría: el pago podría caer en otra instancia del pool, que no conoce a los titulares." Muere con `@Remove` al pagar o cancelar (sobrevive a un pago rechazado, `retainIfException = true`) o por `@StatefulTimeout` si se abandona.
- **Piezas que rodean al stateful:**
  - `ComprasEnCurso` (`@ApplicationScoped`): índice de las compras abiertas, fachada que ordena los pasos que quedan fuera de la transacción (consultar a Usuarios, devolver cupo, marcar el intento de pago) y freno de la segunda llamada simultánea sobre la misma compra (409 `operacion-en-curso`).
  - `VencimientoDeReservas` (`@Singleton` con `@Schedule`, sin estado): una vez por minuto pasa a `VENCIDA` las compras con la reserva vencida y devuelve el cupo a Eventos. La reserva tiene un solo reloj, la hora guardada en la base, porque el `@PreDestroy` del stateful puede no ejecutarse.
  - `EmisionDeEntradasMDB` (sin estado): consume `PAGO_CONFIRMADO` de la cola, pone el código QR a cada entrada, las deja `VALIDA` y pasa la venta de `CONFIRMADA` a `EMITIDA`. Es idempotente y lo garantiza la base.
  - `Cobros` con dos implementaciones, `ClienteDePagos` (Adapter sobre la API REST de Pagos) y `CobrosFake`, elegidas por `FabricaDeCobros` según `PALCO_VENTAS_COBROS` (por defecto, Pagos real).

Método	Ruta	Rol	Qué hace
POST	`/api/v1/ventas`	`user`	Inicia la compra (cabecera `Idempotency-Key`); reserva cupo
GET	`/api/v1/ventas`	`user`	Compras del comprador
GET	`/api/v1/ventas/{ventaId}`	`user`	Una compra (404 si no es del comprador)
POST	`/api/v1/ventas/{ventaId}/pagos`	`user`	Paga la compra; al aprobarse, confirma y publica `PAGO_CONFIRMADO`
DELETE	`/api/v1/ventas/{ventaId}`	`user`	Cancela la compra y devuelve el cupo
GET	`/api/v1/entradas/{entradaId}`	`user`	Una entrada del comprador

- **Datos (esquema `ventas`):** `venta` (comprador, evento, estado, estado de la factura, subtotal, cargo de servicio, total, vencimiento de la reserva, claves de idempotencia de la venta y del pago,

id del pago), `venta_item` (sector, precio unitario, cantidad) y `entrada` (titular con nombre y DNI, precio pagado, estado, código QR único). La base exige que el total sea subtotal más cargo, que no haya dos entradas con el mismo DNI en una compra y que la clave de idempotencia sea única por comprador.

- **Integraciones:** REST síncrono con Usuarios (`/yo`), Eventos (cupo) y Pagos (cobro), con token de servicio salvo Usuarios; publica `PAGO_CONFIRMADO` en la cola y en el tópico dentro de la transacción de `confirmarCompra`; consume la cola.
- **Estado:** Implementado. `factura_estado` vale `NO_APLICA` en toda venta hasta que exista Facturación.

4.4 Pagos

- **Responsabilidad:** cobrar una venta a través de una pasarela, guardar el pago y garantizar la idempotencia (un mismo pedido repetido no cobra dos veces).
- **Interfaz:** `IPagos`: `procesarPago`, `obtener`. `confirmarPago` (el aviso de la pasarela) y `reembolsar`, de la Entrega Parcial N.º 1, se declaran cuando se implementen.
- **Stateless.** `ServicioDePagos`: "no recuerda nada entre una llamada y la siguiente. Lo que tiene que durar (el pago y la clave con la que se pidió) queda en la base, así que cualquier instancia del pool puede atender cualquier pedido, también la repetición de uno anterior". Corre sin transacción propia (`NOT_SUPPORTED`) para no retener una conexión del pool mientras espera a la pasarela.
- **Pasarela:** `PasarelaDePagos`, elegida por `FabricaDePasarela` según `PALCO_PAGOS_PROVEEDOR` (por defecto `fake`; un valor desconocido impide el despliegue). Hoy la única implementación es `PagosFake`, que decide el resultado por el token de la tarjeta (`tok_rechazado`, `tok_sin_respuesta`, cualquier otro aprueba).

Método	Ruta	Rol	Qué hace
POST	`/api/v1/pagos`	`servicio`	Procesa un pago (cabecera `Idempotency-Key`)
GET	`/api/v1/pagos/{pagoId}`	`servicio`	Consulta un pago

- **Datos (esquema `pagos`):** `pago` (venta, monto, medio de pago, proveedor, estado, motivo, clave de idempotencia única, id externo único, fechas).
- **Integraciones:** la llama solo Ventas, por REST. Mercado Pago no está conectado.
- **Estado:** Implementado, con pasarela **Simulada**. Mercado Pago real con su webhook: Diseñado, pendiente (diagrama de secuencia en `docs/entrega-obligatoria-2/diagramas/diagrama_secuencia_mercadopago.png`).

4.5 Notificaciones

- **Responsabilidad:** avisar al comprador cuando se confirma una compra y dejar registro de auditoría de cada compra confirmada.

- **Interfaces:** `INotificaciones` (`enviarConfirmacion`; `enviarRecordatorio` y `enviarAlertaCancelacion`, de la Entrega Parcial N.º 1, se declaran cuando se implementen) e `IAuditoria` (`registrar`, `listar`). Son dos interfaces porque "avisar puede fallar o perderse, auditar no".
- **Stateless.** `ServicioDeNotificaciones`: "cada aviso se arma solo con el mensaje que llega y no depende del aviso anterior". `ServicioDeAuditoria`: "registrar un mensaje no depende del anterior, y lo que tiene que durar queda en la base".
- **Consumidores del tópico `topico.compra-confirmada`:**
  - `AvisoDeCompraMDB`: suscripción no durable. Si el servicio está caído el aviso se pierde, y se acepta: "un aviso viejo no le sirve a nadie".
  - `AuditoriaDeComprasMDB`: suscripción durable (el broker guarda los mensajes mientras está detenido), un solo consumidor (`maxSession = 1`) para registrar en orden, y transacción del contenedor: si falla, reintento y DLQ al quinto intento.
- **Herramienta de desarrollo:** `PruebasDeMensajeria` publica mensajes de prueba (con `ventaId` que empieza con `prueba-`) para ver la mensajería sin hacer una compra. Para producción, el comentario de la clase propone habilitarla con una variable de entorno y excluir esas filas del listado; hoy no está hecho.

Método	Ruta	Rol	Qué hace
POST	`/api/notificaciones/pruebas/pago-confirmado`	`servicio`	Publica un `PAGO_CONFIRMADO` de prueba
POST	`/api/notificaciones/pruebas/pago-confirmado-con-falla`	`servicio`	Publica y falla, para mostrar que el mensaje no sale (forma parte de la transacción)
POST	`/api/notificaciones/pruebas/mensaje-invalido`	`servicio`	Publica un mensaje que no respeta el contrato, para mostrar el camino a la DLQ
GET	`/api/notificaciones/auditoria`	`servicio`	Lista el registro de auditoría

- **Datos (esquema `notificaciones`):** `registro_auditoria` (id del mensaje como clave, tipo, venta, comprador, evento, monto total, cuerpo completo y hora de recepción). V2 agranda la columna del cuerpo.
- **Integraciones:** solo JMS (consume el tópico). Nadie la llama por REST salvo las pruebas y el listado de auditoría.
- **Estado:** Implementado. El aviso al comprador es **Simulado**: se escribe en el log, no se envía un email.

4.6 Validación

- **Responsabilidad (diseñada):** validar en la puerta la autenticidad de una entrada y que se use una sola vez.
- **Diseño:** la Entrega Parcial N.º 1 la define como `ServicioDeValidación` **stateless**, con interfaz `IValidacion` (`validarEntrada`, `marcarComoUsada`). Además se prevé: QR firmado, `POST /api/v1/validacion/escaneos` con rol `staff` y uso único atómico. La justificación de stateless

que da el diseño es que cada escaneo es independiente; el comentario de la clase todavía no existe.

- **Datos:** no tiene migraciones ni esquema todavía.
- **Integraciones previstas:** necesita las entradas, que son de Ventas (que ya genera el `codigo_qr` al emitir); se integraría por REST.
- **Estado:** Diseñado, pendiente. Hoy solo existe `ValidacionApplication` y `GET /api/validacion/health`. La pantalla de puerta del frontend funciona con datos locales.

4.7 Reventa

- **Responsabilidad (diseñada):** reventa oficial de entradas nominativas entre usuarios verificados, con tope de precio original + 10 %.
- **Diseño:** la Entrega Parcial N.º 1 define `ServicioDeReventa` con interfaz `IReventa` (`publicarReventa`, `comprarReventa`, `cancelarPublicacion`), sin decidir todavía si es stateless o stateful. Las reglas ya están en `ReglasDeNegocio`: tope +10 %, mínimo 50 % del precio pagado y cierre 3 horas antes del evento. Se prevé la ruta `/api/v1/reventas`, con el diseño detallado a aprobar el 30/10 y cierre el 17/11.
- **Datos:** no tiene migraciones ni esquema todavía.
- **Integraciones previstas:** Ventas (cambio de titular de la entrada), Pagos (cobro de la reventa) y Validación; el diseño detallado de Reventa tiene que decir qué le pide a cada uno.
- **Estado:** Diseñado, pendiente. Hoy solo existe `ReventaApplication` y `GET /api/reventa/health`. El marketplace del frontend funciona con datos locales.

4.8 Facturación

- **Responsabilidad (diseñada):** emitir la factura electrónica de cada compra contra ARCA (ex AFIP), el sistema legado del dominio, por SOAP. Es la integración SOAP con WSDL que pide la consigna.
- **Diseño:** la Entrega Parcial N.º 1 define `ServicioDeFacturación` con interfaz `IFacturacion` (`emitirFactura`, `consultarEstado`, `reintentar`). El diseño de la facturación por SOAP lo detalla así:
  - un WAR aparte que **simula** ARCA y expone por SOAP un subconjunto del WSFEv1, con WSDL publicado por WildFly (decisión recomendada, pendiente de aprobación);
  - `IFacturacion` y `ServicioDeFacturacion`, que piden el CAE con un cliente JAX-WS y guardan la factura en su esquema;
  - la factura se dispararía desde una suscripción durable al tópico `topico.compra-confirmada` (recomendado) en lugar de una llamada REST dentro de `confirmarCompra`, para no retener una conexión de la base dentro de la transacción de la compra; la llamada a ARCA sigue siendo síncrona;
  - con timeout de 5 segundos o SOAP Fault, la factura queda `PENDIENTE` y se reintenta por `cola.reintento-facturacion`, con la DLQ al final.
- **Datos:** no tiene migraciones ni esquema todavía.

- **Estado:** Diseñado, pendiente. Hoy solo existe `FacturacionApplication` y `GET /api/facturacion/health`; toda venta responde `facturaEstado: NO_APLICA`. La facturación por SOAP tiene hitos entre el 19/10 y el 28/10.

5. Decisiones de diseño

Las decisiones vienen de la Entrega Parcial N.º 1 (`docs/entrega1/`) y de lo que se eligió al implementar. Cada fila dice dónde se ve en el código. Cuando una decisión cambió, la tabla da la versión vigente y la sección 5.8 resume qué cambió y por qué.

5.1 Arquitectura

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Ocho componentes de negocio, cada uno un WAR propio con su esquema de base; se hablan solo por REST o JMS	Un único WAR con módulos; servicios que comparten clases	Es la descomposición de la Entrega Parcial N.º 1. Separar los despliegues obliga a que la integración sea explícita (contratos REST y mensajes), que es lo que evalúa la materia	`backend/servicios/*`, un `persistence.xml` y una migración por servicio
Capas `api/` → `negocio/` → `dominio/`, con `datos/` (repositorios) y `mensajeria/` (MDB) donde hacen falta	Lógica en el recurso REST	El recurso valida la forma del pedido y delega; la regla de negocio queda en el Session Bean, donde el contenedor da transacciones y seguridad declarativas	`VentasResource` → `ComprasEnCurso` → `ServicioDeVentas` → `VentaRepository`
Reglas de negocio (tope +10 %, tasa 10 %, 6 entradas por compra, 10 minutos de reserva, DNI de 7 u 8 dígitos) en un solo lugar	Una constante por servicio	Con copias distintas, un servicio puede aceptar lo que otro rechaza. La regla del DNI estaba escrita dos veces y Ventas iba a sumar una tercera	`common/ReglasDeNegocio.java` (`esDniValido`, `MINUTOS_RESERVA`, `MAX_ENTRADAS_POR_ORDEN`)
`ServicioDeVentas` es `@Stateful`: una instancia por compra en curso	(a) `@SessionScoped` con sesión HTTP; (c) sin stateful, todo en la base	La instancia guarda lo que todavía no está en la base: los titulares de cada entrada, el total y el vencimiento. La identidad sigue en un solo mecanismo, el token; la cookie de sesión solo vale dentro de un WAR. La reserva temporal es el ejemplo de stateful que da la consigna	`ServicioDeVentas` (`@Stateful(passivationCapable = false)`, `@StatefulTimeout(MINUTOS_RESERVA)`)
El stateful se ubica por el id de la venta, con un índice en un bean `@ApplicationScoped` que antes comprueba en la base que la venta sea del `sub` del token	Guardar la referencia en el recurso REST (se crea de nuevo en cada pedido)	Una venta ajena responde 404, igual que una que no existe	`ComprasEnCurso.enCurso`, `delComprador`
Una sola llamada a la vez por compra, frenada en la fachada con 409 `operacion-en-curso`	Dejarlo solo en `@AccessTimeout(0)` del stateful	En WildFly 41 la llamada que el contenedor rechaza hace además que descarte la instancia, con la primera llamada todavía adentro: un doble clic dejaba la compra sin poder pagarse. `@AccessTimeout(0)` queda como última barrera	`ComprasEnCurso.ocupar` / `desocupar`

Decisión	Alternativas consideradas	Por qué	Dónde se ve
La reserva la vence un barrido periódico sobre `vence_reserva`	(a) Que el `@PreDestroy` del stateful escriba en la base y llame a Eventos	La especificación avisa que `@PreDestroy` puede no ejecutarse (se vio: ante una excepción inesperada no corrió) y el timeout del stateful mide inactividad, no una hora fija. Queda un solo reloj, el de la base. Costo: la reserva puede durar hasta un minuto de más	`VencimientoDeReservas` (`@Singleton`, `@Schedule(minute = "*", persistent = false)`)
Mientras se espera a otro servicio no se retiene una conexión de la base	Agrandar el pool	Los 8 servicios comparten un pool de 10 conexiones (`configurar.cli`, `--max-pool-size=10`) y el servicio llamado necesita una para contestar. Con el orden anterior (guardar la venta antes de reservar), 80 compras simultáneas dieron 28 abiertas, 52 respuestas 504 y 20 lugares tomados sin venta; con el orden actual las 80 responden 201 (`docs/entrega3/evidencia/compras-simultaneas.txt`). Agrandar el pool solo corría el límite	`ServicioDeVentas.iniciarCompra` (reserva primero, guarda al final), `confirmarCompra` (cobra con el total que recuerda la instancia), `ServicioDePagos` (`NOT_SUPPORTED`)
Versionado de rutas: los servicios nuevos en `/api/v1`	Reescritura en el servidor; esperar; un WAR de puerta de entrada	Cambia una línea por servicio y no toca lo que ya funcionaba. Usuarios, Eventos y Notificaciones siguen en `/api/<servicio>`	Context root de Ventas (`/api/v1`) y Pagos (`/api/v1/pagos`)

5.2 Datos

Decisión	Alternativas consideradas	Por qué	Dónde se ve
H2 en modo PostgreSQL en desarrollo, Supabase después; migraciones Flyway con SQL que corra en las dos	Una base por motor; `hbm2ddl`	Se desarrolla sin instalar nada y el SQL se escribe una vez. Estado: las migraciones no se corrieron todavía contra PostgreSQL	`src/main/resources/db/migracion` de cada servicio, `configurar.cli` (PalcoDS)
Ninguna tabla lleva `check` de lista de valores	Mantenerlos y sumar una migración que los borre	En H2 2.4.240 un `check (rol in (...))` quedó atado a la conexión que corrió la migración; al cerrarla el pool, todo `INSERT` fallaba con 500. Los valores válidos los controla el enumerado de Java. Se editó la V1 porque esas restricciones no tienen nombre y todavía no había base permanente	V1 de Usuarios y Eventos; enumerados `EstadoVenta`, `EstadoPago`, `Rol`
Las fechas de los eventos de prueba se corren en una migración nueva	Editar la V2	Flyway rechaza un script ya aplicado que cambió	`eventos/.../V3__correr_fechas.sql`
Ids de texto con prefijo y 8 caracteres de UUID (`V-3f9a1c2b`)	Número de la base mostrado como `V-00001`	Es la convención que ya funcionaba. Con la base en memoria la numeración volvería a 1 en cada reinicio mientras el broker conserva mensajes viejos	`ServicioDeVentas.nuevoId`

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Dinero en pesos enteros (`long`)	`double`, `BigDecimal`	Sin errores de redondeo y simple de defender	Entidades `Venta`, `Pago`; DTOs
Cada cambio de estado de la venta es un `UPDATE` con la condición adentro, y solo sigue quien cambió la fila	Leer y después escribir	Con lectura y escritura separadas, pagar, cancelar y vencer se pueden pisar. Es la base la que decide, en una sola sentencia	`VentaRepositoryJpa.cambioUnaFila`, `confirmar`, `cancelar`, `vencer`, `marcarEmitida`; en Eventos, `update Sector ... where s.vendidas + :cantidad <= s.cupo`
Las entradas nacen con el pago, en `PENDIENTE_EMISION`, y el consumidor de la cola las emite	Que las cree el MDB	El mensaje ya lleva el `entradaId` y el `GET` de la entrada responde apenas vuelve el pago	`ServicioDeVentas.entradasPara`, `EmisionDeEntradasMDB`
Los titulares no se guardan hasta que se aprueba el pago	Guardarlos al reservar	Viven en la instancia stateful; recién con el pago aprobado se crean las entradas a su nombre. Consecuencia: una compra que pierde su instancia ya no se puede pagar (sección 7.7)	`ServicioDeVentas.titulares`
Restricciones únicas como defensa de idempotencia: `(comprador_id, clave_idempotencia)` en la venta, la clave en el pago	Verificar solo en Java	Dos pedidos simultáneos pasan los dos la búsqueda previa; la restricción deja guardar a uno solo. `BaseDeDatos.esClaveDuplicada` busca el SQLState 23505 para no convertir en 409 cualquier falla	`ventas/.../V1__crear_tablas.sql` (`venta_clave_idempotencia_unica`), `common/datos/BaseDeDatos.java`

5.3 API

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Errores en Problem Details (`application/problem+json`) para todos los servicios	Formato propio `{estado, mensaje, campos}`; excepciones propias en Ventas	Es un estándar, es lo que promete la OpenAPI, y `type` distingue dos errores con el mismo código (un 409 por cupo agotado de uno por reserva vencida) sin leer el texto	`common/rest/Problema.java`, `ManejadoresDeErrores.java`
La excepción lleva el tipo de problema; el código HTTP y el título los pone el manejador	Guardar el código HTTP en la excepción	El negocio no conoce HTTP. Todas las `ErroresDeNegocio` llevan `@ApplicationException(rollback = true)`	`common/ErroresDeNegocio.java`
Para errores que no son de la aplicación (405, 415), `type` vale `about:blank` y no lleva `detail`	Exponer el mensaje de la excepción	Ese texto es interno del servidor. Nunca se devuelve una traza	`ManejadoresDeErrores`
Un pago rechazado: Pagos responde 201 con `RECHAZADO`; Ventas le responde 402 al comprador	Que Pagos responda 402	Para Pagos el pago se procesó y quedó guardado; si respondiera 402 el intento no quedaría registrado	`PagosResource`, `ClienteDePagos`

Decisión	Alternativas consideradas	Por qué	Dónde se ve
`Idempotency-Key` obligatoria al iniciar y al pagar; dos columnas en la venta (`clave_idempotencia`, `clave_pago`)	Una tabla clave → respuesta	Repetir la clave devuelve la venta como está hoy, sin reservar ni cobrar de nuevo, y no hay que guardar respuestas. Repetirla con otro pedido responde 409 `clave-reutilizada`	`ComprasEnCurso.iniciar` (`mismoPedido`), `pagar`
La clave que Ventas manda a Pagos es `<ventaId>:<clave>`	Reenviar la clave del comprador tal cual	En Ventas la clave es única por comprador; en Pagos, en todo el servicio. Sin el prefijo, dos compradores con la misma clave chocarían en Pagos	`ClienteDePagos.claveParaPagos`
Un 409 de Eventos se distingue por `type`: `evento-finalizado` o `cupo-agotado`	Decidir solo por el código	Eventos responde 409 en los dos casos; mirar solo el código diría "no quedan entradas" de un evento que ya pasó	`ClienteDeEventos`
Las respuestas de Ventas llevan enlaces a las operaciones siguientes	Respuestas sin enlaces	El cliente no arma las rutas a mano	`ventas/api/Enlaces.java`

5.4 Mensajería

El contrato del mensaje está en `docs/api/mensajeria-pago-confirmado.md`; la mensajería se describe en detalle en su propia sección.

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Un mensaje `PAGO_CONFIRMADO` que va a una cola (emisión de entradas) y a un tópico (aviso y auditoría)	Solo tópico; llamadas REST síncronas	La cola tiene un único consumidor: las entradas se emiten una sola vez. El tópico le da una copia a cada suscriptor. El pago no espera a la emisión	`PublicadorPagoConfirmadoJms`, `configurar.cli`
El publicador es `@Stateless` con `MANDATORY` y publica por `java:/JmsXA`	`REQUIRED`; la fábrica `java:/ConnectionFactory`	Con `REQUIRED`, llamado sin transacción abriría una propia y el mensaje saldría aunque la venta fallara. `java:/ConnectionFactory` queda fuera de la transacción	`common/mensajeria/PublicadorPagoConfirmadoJms.java`, `DestinosJms.FABRICA_XA`
La base no es XA y entra a la transacción como último recurso; los consumidores son idempotentes	(b) Base XA; (c) tabla de salida con un proceso que publica	La consigna pide transacciones declarativas y explicar las fallas, no dos fases. La base XA no está verificada contra Supabase y la tabla de salida era más código	Sección 7.4
El mensaje es un `record` con `timestamp` `Instant`, y `desdeJson()` siempre valida	Clase con setters; fecha como texto	JSON-B no avisa si falta un campo: lo deja en `null` o en 0	`common/mensajeria/PagoConfirmadoMensaje.java`
Aviso y auditoría son dos MDB con dos interfaces en el mismo WAR	Una sola interfaz; un noveno WAR	Una interfaz mezclaba responsabilidades; otro WAR sumaba un despliegue	`AvisoDeCompraMDB`, `AuditoriaDeComprasMDB`, `INotificaciones`, `IAuditoria`

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Auditoría es suscripción durable, con una sola sesión; el aviso no es durable	Las dos durables, o ninguna	Auditoría no puede perder eventos; un aviso viejo no le sirve a nadie. Con una sesión procesa en el orden de entrega	`AuditoriaDeComprasMDB` (`palco-auditoria`, `compras-confirmadas`, `maxSession` 1)
La clave de `registro_auditoria` es el `JMSMessageID`	Clave única por `venta_id`	El id no cambia en la reentrega; una clave por venta escondería que Ventas publicó dos veces	`ServicioDeAuditoria`
5 entregas, con 2 segundos entre una y otra, y después la DLQ	Los valores de fábrica (10 seguidas, sin espera)	Una falla pasajera agotaría los intentos en un instante	`configurar.cli`, `address-setting=#`
Ante un mensaje inválido: la cola y auditoría deshacen (`setRollbackOnly`) y van a la DLQ; el aviso lo descarta con un `WARNING`	Lanzar una excepción	Una excepción dejaba una traza por intento, y reintentar en los dos suscriptores dejaba dos copias en la DLQ	`EmisionDeEntradasMDB.onMessage`, MDB de Notificaciones

5.5 Pagos

Decisión	Alternativas consideradas	Por qué	Dónde se ve
`ServicioDePagos` decide y guarda; la pasarela solo habla con el exterior	(b) Que cada pasarela implemente `IPagos`	Guardar el pago y detectar repetidos se escribe una sola vez y vale para cualquier pasarela. Coincide con la Entrega Parcial N.º 1	`IPagos`, `ServicioDePagos`, `pasarela/PasarelaDePagos`
La pasarela la elige una Factory por configuración; un valor desconocido impide el despliegue	Un `if` en el servicio; caer al simulador	Separa configuración y negocio; no se cobra de mentira sin avisar	`FabricaDePasarela` (`palco.pagos.proveedor`, por defecto `fake`)
Ventas cobra detrás de la interfaz `Cobros`, con dos implementaciones elegidas por una Factory; el defecto es el servicio de Pagos	Llamar al simulador desde el bean	El servicio de Pagos se conectó sin tocar el bean de la compra. El simulador solo se usa si alguien escribió `fake`; un valor desconocido cobra contra Pagos con un `WARNING`	`FabricaDeCobros` (`palco.ventas.cobros`), `ClienteDePagos`, `CobrosFake`
Solo el cobro con pasarela simulada antes del 23/10; Mercado Pago, webhook y reembolso después	Sumar también Mercado Pago, webhook y reembolso antes del 23/10	Esas piezas no responden a ningún punto del 23/10 y la firma del webhook no se pudo verificar contra la documentación oficial. Estado: **Simulado** (`PagosFake` decide por el token: `tok_rechazado`, `tok_sin_respuesta`, cualquier otro aprueba)	`PagosFake`; diagrama de secuencia de Mercado Pago (diseño) en `../entrega-obligatoria-2/diagramas/diagrama_secuencia_mercadopago.png`
El token de la tarjeta no se guarda	Guardarlo para reintentos	Es sensible y sirve para un solo cobro. Consecuencia: tras un `RECHAZADO`, la misma clave devuelve siempre ese rechazo; un intento nuevo necesita clave nueva	`Pago` (sin columna de token)

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Ninguna transacción abierta mientras se espera a la pasarela	Una transacción por cobro	Retendría una conexión mientras contesta un tercero y, después de una clave duplicada, quedaría marcada para deshacerse: ya no se podría releer el pago ganador	`ServicioDePagos` (`NOT_SUPPORTED`), `PagoRepositoryJpa` (`@Stateless`, transacción corta por método)

5.6 Seguridad

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Seguridad declarativa con `@RolesAllowed` / `@PermitAll` en cada operación; roles `user`, `organizer`, `staff`, `servicio`	Comprobaciones a mano en cada método	Lo pide la consigna y queda visible en la firma de cada operación	Recursos de cada servicio; `configurar.cli` (`resteasy-role-based-security`)
JWT firmado con RSA por Usuarios; roles en el claim `groups`	Sesión con cookie	Un token vale en los 8 WAR; la cookie solo dentro de uno	`usuarios/seguridad/Tokens.java`, `ClavesJwt`
Token de rol `servicio` por secreto compartido (credenciales de cliente)	(b) Un usuario "servicio" que hace login; (c) firmar a mano con una clave fija	Se explica en una frase; (b) obligaba a inventarle DNI y contraseña a algo que no es una persona	`POST /api/usuarios/servicios/token`, `common/seguridad/TokenDeServicio.java`
El token de servicio dura 5 minutos y se guarda en un bean `@ApplicationScoped`, que lo renueva cuando le quedan menos de 30 segundos; ante un 401 se invalida y se reintenta una vez	Pedir uno por llamada	Pedir uno por llamada duplicaba las llamadas a Usuarios	`TokenDeServicio.obtener` / `invalidar`
El secreto se compara con `MessageDigest.isEqual`	`equals`	Tarda lo mismo acierte o no	`ServicioDeUsuarios`
Contraseñas con PBKDF2 y 600.000 iteraciones (desde el 16/10; antes 310.000); el formato guarda las iteraciones	Otro algoritmo	El formato permite subir las iteraciones sin invalidar hashes viejos, como pide la guía actual de OWASP. La migración V5 rehashea las cuentas de prueba para que no tarden distinto que el hash falso	`usuarios/seguridad/Contrasenas.java`, `V5__cuentas_de_prueba_con_600000_iteraciones.sql`
Después de 5 logins fallidos en 15 minutos para un email, 429 con `Retry-After` aunque la clave sea correcta, también para emails inexistentes	Bloquear la cuenta; limitar por IP	No revela qué emails existen y no deja la cuenta bloqueada para siempre. Los intentos van en una tabla (V4), así funciona con varias instancias	`ServicioDeUsuarios`, `IntentosDeLogin`, `V4__intentos_de_login.sql`
El 401 por token inválido lo responde una página de error de `backend/common` como Problem Details, con `WWW-Authenticate` y CORS	Dejar la página HTML del servidor	El servidor lo rechazaba antes de la aplicación, en HTML y sin CORS, y el navegador lo veía como error de red	`common/rest/ErrorDeAutenticacionServlet.java`, `META-INF/web-fragment.xml`

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Una venta o entrada ajena responde 404, igual que una que no existe	403	No revela que el recurso existe	`ComprasEnCurso.delComprador`, `obtenerEntrada`
Validación de caracteres de control en nombres e ids que van al log o al mensaje	Limpiar solo al escribir	Un salto de línea en el nombre del titular haría aparecer una línea falsa en el log del consumidor	`ServicioDeVentas.validar`, `common/Textos.java`

Lo que la matriz de seguridad de la clase 11 deja para decidir en grupo: la vigencia de la sesión (hoy 8 horas, sin refresh token) y los permisos por destino en el broker. Con `PALCO_SECRETO_SERVICIOS` ausente el secreto vale `palco-desarrollo`, que está escrito en el repo; el servidor de desarrollo ya usa uno propio en `backend/.env`, y que fuera de desarrollo la variable sea obligatoria está diseñado, pendiente.

5.7 Frontend

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Dos contextos: sesión (`SesionContext`) y catálogo y datos locales (`AppContext`)	Uno solo	Son responsabilidades distintas	`frontend/src/store/SesionContext.tsx`, `frontend/src/store/AppContext.tsx`
`fetch` con `useEffect`, sin TanStack Query	Una librería de datos	Pocas llamadas; la librería sumaba conceptos que habría que saber explicar	`frontend/src/lib/api.ts`
Token en `localStorage`, confirmado después con `GET /yo`	Memoria; cookie HttpOnly	En memoria no sobrevive a recargar; la cookie exigía cambiar el backend	`SesionContext`
Sin datos de respaldo local para usuarios y eventos	Mantener los de mentira	Escondían las fallas del backend y creaban datos que el servidor no conoce	Catálogo, sesión
La reserva se crea al terminar el paso "Titulares"	Reservar al entrar al checkout	El backend pide los titulares en el mismo pedido que reserva y no deja cambiarlos	Checkout
Dos claves de idempotencia: la de la compra cambia si cambia el pedido; la del pago nace con la reserva, se guarda en el carrito y cambia solo después de un 402	Deshabilitar el botón	El botón no cubre un corte de red. Ante un 504 se reintenta con la misma clave, que es lo único que destraba esa venta	`frontend/src/servicios/ventas.ts`, carrito
Cuenta regresiva recalculada contra el vencimiento del servidor, corregida por el desfase del reloj del navegador	Restar uno por segundo	El contador se atrasa con la pestaña en segundo plano y se reinicia al recargar; con el reloj adelantado la página cancelaba reservas recién hechas	Checkout (`desfaseReloj`)
Espera de la emisión: una consulta por segundo, hasta 30	WebSockets	El backend no los ofrece; alcanza con un `setTimeout`	Confirmación

Decisión	Alternativas consideradas	Por qué	Dónde se ve
Un corte de red no cierra la sesión	Consultar el `health` y cerrar la sesión si responde	En un corte el `health` también responde un instante después: se cerraba la sesión y se perdía la clave de idempotencia	`api.ts`
Reventa, puerta y estadísticas del organizador siguen con datos locales	Conectarlas ya	Sus servicios (Reventa, Validación) solo tienen `health`. Estado: **Simulado** en el navegador	`localStorage` y datos fijos

5.8 Decisiones que cambiaron

Decisión original	Fuente	Versión vigente	Por qué cambió
Repositorios para todos los servicios (`UsuarioRepository`, `EventoRepository`, `EntradaRepository`, `VentaRepository`)	Entrega Parcial N.º 1, sección 4	Hay repositorios en Ventas (`VentaRepository`, `EntradaRepository`) y Pagos (`PagoRepository`). Eventos y Usuarios usan el `EntityManager` desde su Session Bean	Los repositorios se sumaron donde había UPDATE condicionales y transacciones cortas que separar del negocio. En Eventos y Usuarios el acceso es directo
`IVentas` con `iniciarCompra`, `agregarEntrada`, `confirmarCompra`, `cancelarCompra`	Entrega Parcial N.º 1, sección 2	Tres operaciones: `iniciarCompra` (con todos los renglones y titulares), `confirmarCompra`, `cancelarCompra`	La compra se arma completa en un pedido; agregar entradas de a una sumaba llamadas sin necesidad
"XA entre la base y el broker"	`TODO.md`, Fase 6 (ya corregido)	Broker XA y base como último recurso	No es un commit en dos fases: ver 7.4
Guardar la cabecera de la venta antes de reservar	Primera versión de Ventas	Reservar primero y guardar la venta al final, de una vez	La prueba de 80 compras simultáneas agotaba el pool (5.1)
Concurrencia por compra solo con `@AccessTimeout(0)`	Primera versión de Ventas	Freno en `ComprasEnCurso` y `@AccessTimeout(0)` como última barrera	Comportamiento de WildFly 41, visto en vivo (5.1)
Una compra sin instancia seguía `EN_CHECKOUT` hasta el barrido	Ventas, hasta el 14/10	Se cancela y devuelve el cupo cuando el comprador intenta pagarla	La compra nueva podía encontrar el sector agotado por la propia compra perdida
"Mis entradas" se recordaba en el navegador porque el backend no listaba ventas	Checkout del frontend	`GET /api/v1/ventas` lista las compras del usuario del token y el frontend las pide	Se sumó el listado en Ventas (`ComprasEnCurso.listar`, `MyTicketsPage.tsx`)
La línea del aviso decía "aviso enviado"	Evidencia del 13/10	"Aviso registrado en el log; no se envía: no hay proveedor de correo"	No hay correo: el texto no puede ser más optimista que lo que pasa
`registro_auditoria.cuerpo` `varchar(4000)`	V1 de Notificaciones	`varchar(100000)` desde la V2	Un mensaje válido de 6 entradas con nombres largos ocupaba 4733 caracteres y terminaba en la DLQ

6. Patrones aplicados

Solo se cuentan los patrones que están aplicados en el código. Para cada uno: dónde está, qué problema resuelve ahí y por qué ese y no otro.

6.1 Factory

Dónde	`pagos/pasarela/FabricaDePasarela` y `ventas/negocio/externos/FabricaDeCobros`. Las dos son beans `@ApplicationScoped` con un método `@Produces`
Problema	El código que usa la pasarela (o el cobro) no tiene que saber qué implementación concreta hay del otro lado ni leer la configuración. `ServicioDePagos` pide un `PasarelaDePagos` con `@Inject`; `ServicioDeVentas` pide un `Cobros`
Por qué este	Es el único lugar que conoce las clases concretas y la propiedad que elige (`palco.pagos.proveedor`, `palco.ventas.cobros`). Un `if` en el negocio mezclaría configuración con reglas. Con el `@Produces` de CDI no hace falta escribir una fábrica abstracta: el contenedor llama al método y entrega el resultado donde se inyecta
Detalle	Las dos fábricas tratan distinto el valor desconocido, y a propósito: en Pagos impide el despliegue; en Ventas cobra contra el servicio de Pagos y deja un `WARNING`, para que el simulador nunca quede puesto por olvido

6.2 Adapter

Dónde	`ventas/negocio/externos/ClienteDePagos` (implementa `Cobros`). Con la misma idea, `ClienteDeEventos` y `ClienteDeUsuarios`
Problema	Pagos habla con códigos HTTP y JSON; la compra solo entiende `Cobros.cobrar(...)` y su `ResultadoDeCobro`. `ClienteDePagos` traduce: 201 `APROBADO` o `RECHAZADO` a un resultado; 400 o 409 `clave-reutilizada` a un rechazo sin pago; 401 a renovar el token y reintentar una vez; 401 repetido, 403 o 404 a `CobroNoRealizado`; cualquier otra respuesta, o ninguna, a `SinRespuesta`
Por qué este	La compra no depende del protocolo: el mismo bean cobró primero con `CobrosFake` y después con el servicio de Pagos sin cambiar una línea
Estado	El adaptador de Mercado Pago, que traduciría la API del proveedor a `PasarelaDePagos`, está **diseñado, pendiente**. `PagosFake` no es un adaptador: es un simulador que implementa la interfaz

6.3 Strategy (por configuración)

Dónde	Las interfaces `Cobros` (implementaciones `ClienteDePagos` y `CobrosFake`) y `PasarelaDePagos` (hoy solo `PagosFake`)
Problema	Cobrar es un algoritmo intercambiable: de verdad o simulado. El contexto (`ServicioDeVentas`, `ServicioDePagos`) trabaja contra la interfaz
Por qué este	Permite trabajar en Ventas sin Pagos desplegado y conectar Pagos sin reescribir la compra
Límite	La estrategia se elige una vez, al primer uso, y vale hasta redesplegar (los dos productores son `@ApplicationScoped`); no cambia por pedido. En `PasarelaDePagos` hay una sola implementación: la variación todavía es potencial. Por eso en Pagos se cuenta como Adapter más Factory

6.4 Facade

Dónde	`ventas/negocio/ComprasEnCurso`
Problema	Una compra involucra el stateful de la compra, dos repositorios, Usuarios (identidad verificada) y Eventos (devolución de cupo), con pasos que quedan fuera de la transacción del stateful. Los recursos REST (`VentasResource`, `EntradasResource`) hablan solo con esta clase
Por qué este	Concentra en un lugar el orden de los pasos (consultar a Usuarios antes de abrir la compra, anotar el intento de pago antes de cobrar, devolver el cupo después de confirmar la cancelación) y la regla de una llamada a la vez. El recurso queda delgado
Detalle	No es un Session Bean: es `@ApplicationScoped` porque guarda el índice de compras abiertas, que tiene que ser el mismo para todos los pedidos, y no abre transacciones

6.5 Repository

Dónde	`ventas/datos/VentaRepository` (implementación `VentaRepositoryJpa`), `EntradaRepository` (`EntradaRepositoryJpa`) y `pagos/datos/PagoRepository` (`PagoRepositoryJpa`). Las implementaciones son `@Stateless`
Problema	El negocio necesita operaciones con significado (`guardarClaveDePago`, `confirmar`, `cancelar`, `vencer`, `buscarReservasVencidas`) y no consultas JPQL. Cada una es un `UPDATE` condicional que dice si cambió la fila (`cambioUnaFila`)
Por qué este	Aísla JPA del negocio, da a cada operación su propia transacción corta (el bean `@Stateless` usa `REQUIRED` por defecto, y se suma a la del llamador si existe) y permite probar el negocio con dobles: Mockito en Ventas y un repositorio en memoria en Pagos (`PagoRepositoryEnMemoria`), que necesita comportamiento (rechazar una clave repetida)
Alcance	Eventos y Usuarios no tienen repositorio: usan el `EntityManager` desde su Session Bean (5.8)

6.6 DTO

Dónde	`api/Dtos.java` de cada servicio (por ejemplo `NuevaVentaDto`, `VentaDto`, `EntradaDto` en Ventas), todos `record`. El mensaje `PagoConfirmadoMensaje` cumple el mismo papel entre servicios
Problema	Lo que entra y sale por la API no es la entidad JPA: la venta tiene columnas que no se exponen (`clave_pago`) y la respuesta lleva datos que la entidad no tiene (enlaces)
Por qué este	Separa el contrato público del modelo de datos; cambiar una tabla no rompe a los clientes. `ServicioDeVentas.aDto` hace la conversión en un solo lugar. Ventas no usa ninguna clase de otro servicio: tiene sus propios `record` para lo que recibe (`ClienteDeEventos.Sector`)

6.7 Singleton

Dónde	Singletons EJB (`@Singleton`): `VencimientoDeReservas`, `ClavesJwt` (`@Startup`) y `Migraciones<Servicio>` en cada servicio. Instancia única por CDI (`@ApplicationScoped`): `ComprasEnCurso`, `TokenDeServicio`, los clientes externos y las fábricas
Problema	Hay cosas que tienen que existir una sola vez por servicio: un solo barrido de reservas (el contenedor no deja correr dos a la vez), un solo par de claves RSA para firmar, un solo índice de compras abiertas, un solo token de servicio vigente

Por qué este	Lo da el contenedor, sin código propio de sincronización; donde varios pedidos comparten estado (el índice de compras) se usan colecciones concurrentes (`ConcurrentHashMap`). `@DependsOn("MigracionesVentas")` asegura que el barrido no arranque antes de que existan las tablas

6.8 Patrones de mensajería

Patrón	Dónde	Qué resuelve
Punto a punto (cola)	`cola.emision-entradas` (JNDI `java:/jms/queue/emisionEntradas`), consumida por `EmisionDeEntradasMDB`	Un único consumidor: las entradas se emiten una sola vez
Publicación y suscripción (tópico)	`topico.compra-confirmada` (JNDI `java:/jms/topic/compraConfirmada`), consumido por `AvisoDeCompraMDB` y `AuditoriaDeComprasMDB`	Cada suscriptor recibe su copia; sumar un suscriptor no cambia al productor
Suscriptor durable	`AuditoriaDeComprasMDB` (`clientId` `palco-auditoria`, `subscriptionName` `compras-confirmadas`)	El broker guarda los mensajes mientras el consumidor está detenido
Consumidor dirigido por mensajes	Los tres MDB	El contenedor entrega cada mensaje en una transacción, con pool de instancias
Selector de mensajes	`messageSelector` con `DestinosJms.SELECTOR_PAGO_CONFIRMADO`	El consumidor recibe solo el tipo de evento que entiende
Canal de mensajes fallidos (DLQ)	`configurar.cli`: 5 entregas cada 2 segundos y después `java:/jms/queue/DLQ`	Un mensaje que no se puede procesar no se pierde ni traba la cola
Receptor idempotente	Emisión: `UPDATE` condicionales sobre la entrada y la venta. Auditoría: clave primaria `JMSMessageID`	Tolera la entrega repetida que permite la garantía de último recurso (7.4)
Publicación transaccional	`PublicadorPagoConfirmadoJms` con `MANDATORY` y `java:/JmsXA`	El mensaje sale solo si la venta se confirma

No se cuentan como aplicados otros patrones que se evaluaron (por ejemplo una tabla de salida u *outbox*, que fue una de las alternativas para publicar el mensaje y se descartó: 5.4).

7. Transacciones

Las transacciones son declarativas: las abre y cierra el contenedor según el `@TransactionAttribute` de cada operación. Entre servicios no hay transacción distribuida: lo que se confirma en Eventos o en Pagos queda confirmado allá aunque después falle Ventas, y se compensa a mano o se protege con idempotencia. Todo lo de esta sección sale de `ComprasEnCurso`, `ServicioDeVentas`, `VencimientoDeReservas`, `VentaRepositoryJpa`, `EmisionDeEntradasMDB`, `PublicadorPagoConfirmadoJms` y los clientes de `ventas/negocio/externos/`.

7.1 Resumen de atributos

Operación	Clase	Atributo	Qué entra en la transacción
Iniciar compra	`ServicioDeVentas.iniciarCompra`	`NOT_SUPPORTED`	Nada propio. La única escritura (`ventas.guardar`) corre en la transacción corta del repositorio: la venta y sus renglones juntos

Operación	Clase	Atributo	Qué entra en la transacción
Anotar el intento de pago	`VentaRepositoryJpa.guardarClaveDePago` (desde `ComprasEnCurso.pagar`)	`REQUIRED` del repositorio, sin transacción del llamador	Un `UPDATE`, confirmado en el momento
Pagar y confirmar	`ServicioDeVentas.confirmarCompra`	`REQUIRED`	Pasar la venta a `CONFIRMADA`, crear las entradas, publicar `PAGO_CONFIRMADO`
Publicar	`PublicadorPagoConfirmadoJms.publicar`	`MANDATORY`	Se suma a la de `confirmarCompra`; sin transacción falla
Cancelar	`ServicioDeVentas.cancelarCompra`	`REQUIRED`	El `UPDATE` a `CANCELADA`
Vencer	`VencimientoDeReservas.vencerReservas`	`NOT_SUPPORTED`	Nada propio; cada `vencer` corre en la transacción corta del repositorio
Emitir	`EmisionDeEntradasMDB.onMessage`	`REQUIRED`	Recibir el mensaje, emitir cada entrada y pasar la venta a `EMITIDA`
Cobrar en Pagos	`ServicioDePagos`	`NOT_SUPPORTED`	Nada propio; cada método de `PagoRepositoryJpa` corre la suya
Devolver cupo	`ComprasEnCurso.devolverCupo` → `ClienteDeEventos.liberarCupo`	Sin transacción (bean CDI)	Una llamada REST a Eventos por renglón, después de confirmar el cambio de estado

Las excepciones de `ErroresDeNegocio` son excepciones de aplicación con `rollback = true`: deshacen la transacción y el stateful sobrevive. Cualquier otra excepción además hace que el contenedor descarte la instancia.

7.2 Iniciar: reserva de cupo y guardado

1. `ComprasEnCurso.iniciar` busca una venta con la misma clave de idempotencia y comprador; si existe y el pedido es el mismo, la devuelve sin reservar (si es otro pedido, 409 `clave-reutilizada`).
2. Consulta a Usuarios (`GET /yo` con el token del comprador): sin identidad verificada, 403 `identidad-no-verificada`, antes de pedir una instancia o reservar.
3. Pide una instancia nueva del stateful por JNDI y llama a `iniciarCompra` (`NOT_SUPPORTED`). Valida todo sin tocar la base, arma la venta en memoria y reserva sector por sector en Eventos (`ClienteDeEventos.reservarCupo`). Eventos aplica la reserva con un `UPDATE` condicional (`vendidas + cantidad <= cupo`), en su propia transacción.
4. Recién después guarda la venta completa (`ventas.guardar`), en la transacción del repositorio.
5. **Compensación:** si una reserva o el guardado fallan, `devolverCupo` libera los sectores ya reservados, que son los renglones agregados después de una respuesta correcta. El sector cuya llamada no respondió no se devuelve, porque no se sabe si la reserva se aplicó: puede quedar un lugar sin vender, nunca uno vendido dos veces.
6. **Carrera por la misma clave:** si dos pedidos con la misma clave llegan a la vez, los dos reservan; la restricción única `(comprador_id, clave_idempotencia)` deja guardar a uno. El otro recibe `clave-repetida`, devuelve su cupo y `ComprasEnCurso` responde con la venta ganadora. Si en el sector quedaba lugar para uno solo, el que pierde puede recibir 409 `cupo-agotado`; al repetir, recibe la venta.

Por qué `NOT_SUPPORTED`: con Eventos no hay nada que una transacción de esta base pueda deshacer, y así ninguna llamada a Eventos espera con una conexión tomada.

7.3 Pagar y confirmar

`ComprasEnCurso.pagar` ordena cuatro pasos:

1. **Mirar sin cambiar.** Lee la venta del comprador. Si ya está paga con la misma clave, la devuelve sin cobrar (idempotencia). Si está paga con otra clave, 409 `venta-ya-paga`. Si no se puede pagar (vencida, cancelada) 409 `reserva-vencida`. Si la venta está vigente pero su instancia ya no está, la cancela con el `UPDATE` condicional y devuelve el cupo antes de responder `reserva-vencida`.
2. **Anotar el intento.** `guardarClaveDePago` hace `update venta set clave_pago = :clave where estado = 'EN_CHECKOUT' and ((clave_pago is null and vence_reserva > :ahora) or clave_pago = :clave)` y lo confirma en el momento, fuera de la transacción del pago. Con la marca puesta, `cancelar` y `vencer` (que exigen `clave_pago is null`) no cambian la fila: el cupo no se libera con un cobro en curso. Y si el pago falla después de cobrar, queda el rastro.
3. **Cobrar y confirmar** en `confirmarCompra` (`REQUIRED`, `@Remove(retainIfException = true)`):
   - Valida con el estado de la instancia, sin leer la base.
   - Cobra (`cobros.cobrar`) con el total que la instancia calculó al abrir la compra. Es una llamada hacia afuera: va antes de escribir porque un rollback no devuelve la plata. La transacción ya está abierta, pero no tomó ninguna conexión: la toma con la primera consulta, en el paso siguiente.
   - Si el cobro se rechaza, `PagoRechazado` (402): no hay nada que deshacer y la instancia sigue viva para pagar con otro medio.
   - Si se aprueba, en la misma transacción: `ventas.confirmar` (UPDATE condicional a `CONFIRMADA` con el `pagoId`), `entradas.guardar` (una entrada por titular, en `PENDIENTE_EMISION`) y `publicador.publicar` (`PAGO_CONFIRMADO` a la cola y al tópico). Si algo falla se deshace todo junto: no queda una venta confirmada sin entradas ni un mensaje de una venta que no se confirmó. El mensaje sale del servidor recién con el commit.
   - El `@Remove` termina la conversación al volver bien.
4. **Decidir sobre la marca.** La regla es una: la marca se quita cuando se sabe que no se cobró. Se quita ante `PagoRechazado`, ante `CobroNoRealizado` (el pedido se frenó antes del cobro: 401 repetido, 403, 404, sin token de servicio), ante `NoSuchEJBException` (la instancia ya no existía y la llamada no entró) y ante una reserva que venció entre la marca y la revisión. Se deja ante `SinRespuesta` (504) y ante una falla inesperada (`EJBException`): la venta queda `EN_CHECKOUT` con su cupo. En un reintento nunca se quita, porque del intento anterior con esa clave no se sabe el resultado.

Si `ventas.confirmar` no cambia la fila después de un cobro aprobado (no debería pasar, porque la marca impide cancelar y vencer), queda un `SEVERE` en el log con el id del pago para devolverlo y la operación sale como falla inesperada.

7.4 La publicación dentro de la transacción: broker XA y base como último recurso

- El publicador usa la fábrica `java:/JmsXA` (`DestinosJms.FABRICA_XA`): el broker participa de la transacción JTA como recurso XA, con fase de preparación.
- `PalcoDS` es un datasource común, no XA (`configurar.cli`, `data-source add`). Entra a la misma transacción como último recurso: el administrador de transacciones prepara el broker, confirma la base y después confirma el broker.
- `MANDATORY` en el publicador impide publicar fuera de la transacción de quien llama. Con `REQUIRED` abriría una propia y el mensaje saldría aunque la venta se deshiciera.
- **No es un commit en dos fases completo.** Si el servidor se cayera justo entre el commit de la base y el del broker, quedaría una venta `CONFIRMADA` con sus entradas sin emitir y sin mensaje. Esa ventana no se observó en ejecución ni en una caída real: el orden es el que la documentación del servidor describe para un recurso no XA. Lo que sí se probó es que un mensaje publicado dentro de una transacción que se deshace no sale: el endpoint de prueba con falla de Notificaciones responde 409, los contadores de mensajes agregados de la cola, del tópico y de la DLQ no cambian y los tres consumidores no escriben ninguna línea en el log (`docs/entrega3/evidencia/reproduccion.txt`, paso 8). En Ventas, la excepción forzada después de publicar se probó una vez con un WAR temporal que no quedó en el repo.
- Por eso los consumidores son idempotentes (7.6).

7.5 Cancelar y vencer: cambio de estado primero, compensación después

Cancelar (`ComprasEnCurso.cancelar`): si la instancia está, `cancelarCompra` (`REQUIRED`, `@Remove`) hace `update venta set estado = 'CANCELADA' where estado = 'EN_CHECKOUT' and clave_pago is null`; si no está, el mismo `UPDATE` lo hace el repositorio. Solo quien cambió la fila devuelve el cupo, y recién cuando la cancelación ya quedó confirmada. Repetir la cancelación sobre una venta cancelada o vencida no hace nada. Con un pago en curso responde 409 `operacion-en-curso`; si ya está paga, 409 `venta-ya-paga`.

Vencer (`VencimientoDeReservas`, `@Singleton`, `@Schedule` cada minuto, `persistent = false`, `NOT_SUPPORTED`): busca las ventas `EN_CHECKOUT` con la reserva vencida y sin `clave_pago`, y por cada una hace `update ... set estado = 'VENCIDA' where estado = 'EN_CHECKOUT' and vence_reserva <= :ahora and clave_pago is null`. Solo si cambió la fila devuelve el cupo y quita la compra del índice. Si la base no responde, la corrida se corta con un `WARNING` y la toma la del minuto siguiente.

Con este orden, el cupo de una venta lo devuelve uno solo (quien la cerró) y solo por una venta que ya está cerrada en la base. Entre Ventas y Eventos no hay transacción: la devolución es una compensación (`ClienteDeEventos.liberarCupo`, `POST .../cupo` con cantidad negativa).

7.6 Emisión e idempotencia del consumidor

`EmisionDeEntradasMDB.onMessage` (`REQUIRED`): recibir el mensaje, poner el código QR a cada entrada, dejarla `VALIDA` y pasar la venta de `CONFIRMADA` a `EMITIDA` se confirman juntos. Si algo falla, no queda nada escrito y el broker reentrega (5 entregas, 2 segundos, DLQ). Cada cambio es un `UPDATE` condicional:

la entrada tiene que seguir `PENDIENTE_EMISION` y la venta `CONFIRMADA` (`marcarEmitida`). Una segunda entrega no cambia ninguna fila y el mensaje se confirma sin error; vale también si dos copias llegan a la vez a dos instancias. Un mensaje inválido se deshace con `setRollbackOnly()` y termina en la DLQ; un mensaje de una venta que no existe se descarta con un `WARNING`.

7.7 Idempotencia: resumen

Paso	Mecanismo	Qué pasa si se repite
Iniciar	`Idempotency-Key` + restricción única `(comprador_id, clave_idempotencia)`	Devuelve la venta como está hoy, sin reservar otra vez
Pagar	`clave_pago` en la venta; clave `<ventaId>:<clave>` hacia Pagos, que deduplica con restricción única	Devuelve la venta paga sin cobrar; tras un 504 el reintento con la misma clave hace que Pagos devuelva el pago ya guardado. Ante 409 `pago-duplicado` `ClienteDePagos` repite una vez con la misma clave
Cancelar / vencer	`UPDATE` condicional	No cambia nada y no vuelve a devolver cupo
Emitir	`UPDATE` condicional por entrada y por venta	No cambia nada
Auditar	Clave primaria `JMSMessageID`	No inserta otra fila

Límite del frontend: la clave de la compra vive en la memoria de la página; si la respuesta de la reserva se pierde y se recarga la página antes de reintentar, el reintento lleva otra clave y la primera reserva queda sin dueño hasta que la vence el barrido.

7.8 Límites conocidos

Límite	Qué pasa	Por qué se acepta	Qué lo resolvería
Pago sin respuesta y reintento tardío	Después de un 504 la venta queda `EN_CHECKOUT` con `clave_pago`. El reintento con la misma clave vale aunque la reserva haya vencido, pero solo mientras viva la instancia (titulares en memoria). Si la instancia murió (10 minutos sin llamadas, redespliegue, falla), pagar responde 409 `operacion-en-curso` ("puede que ya se haya cobrado") con un `WARNING`; cancelar no puede y el barrido la saltea. Hoy se resuelve a mano	Ventas no sabe si Pagos cobró, y Pagos no tiene consulta por clave ni por venta. Vencerla podría vender dos veces un lugar cobrado; retenerla deja, en el peor caso, un lugar sin vender. Con la pasarela simulada solo pasa con `tok_sin_respuesta` o con una caída	Conciliación con Pagos: **diseñado, pendiente**
Devolución de cupo sin reintento	`liberarCupo` hace un solo intento (repite solo ante 401). Si Eventos no responde, deja un `WARNING` con venta, evento, sector y cantidad y devuelve `false`, que nadie mira. Nada en la base dice que falta devolver	Es el lado seguro: un lugar sin vender, nunca uno vendido dos veces. El `WARNING` trae los datos para devolverlo a mano	Una marca de devolución pendiente que el barrido reintente, con una devolución segura de repetir, como parte de la conciliación con Pagos: **diseñado, pendiente**
Caída entre los dos commits	Venta `CONFIRMADA` sin mensaje ni emisión	Es la garantía elegida (5.4)	Base XA o tabla de salida; no planificado

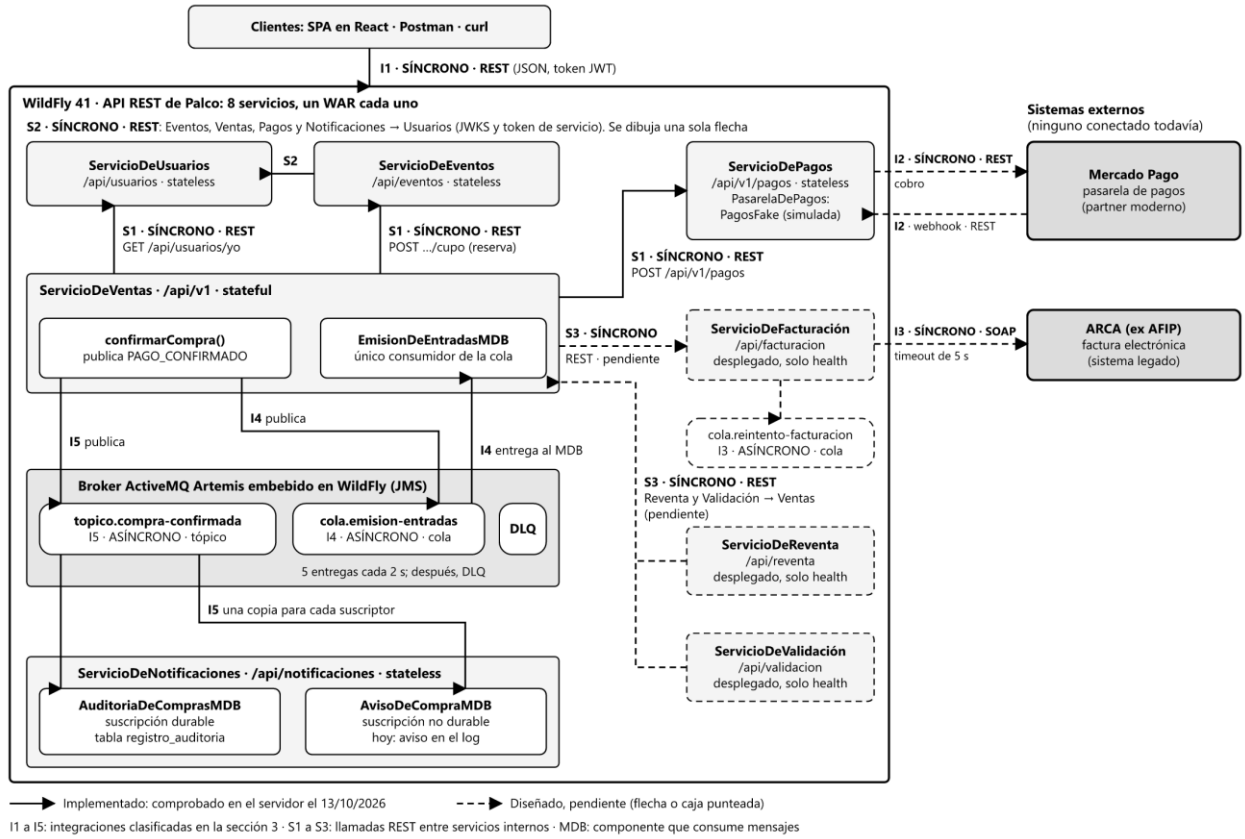
Límite	Qué pasa	Por qué se acepta	Qué lo resolvería
Reserva sin dueño ante una caída	Si el servidor se cae entre la reserva en Eventos y el guardado de la venta, el cupo queda tomado sin venta	Ventana corta; queda en el log	La compensación de la conciliación con Pagos
Compras en curso en memoria	Redesplegar Ventas vacía el índice y pierde los titulares: la compra se puede consultar y cancelar; pagarla la cancela y devuelve el cupo	El stateful es el diseño elegido (5.1)	Fuera de alcance
Timeouts de los clientes	Eventos y Usuarios: 2 s de conexión y 5 s de lectura; Pagos: 2 s y 10 s. Al vencer, `SinRespuesta` (504)	Valores fijos en el código, no medidos contra un servicio lento	Ajustarlos con mediciones

Sin probar en vivo: una caída entre el commit de la base y el del broker, más de 80 compras o 60 pagos simultáneos, Eventos o Usuarios caídos (solo con tests) y las migraciones contra PostgreSQL.

8. Estrategia de integración (SOA)

Cada integración lleva uno de tres estados: **Implementado** (hay código y se comprobó contra el servidor en marcha), **Simulado** (el código existe, pero el otro lado es un reemplazo local: la pasarela `PagosFake` o el aviso que se escribe en el log) y **Diseñado, pendiente** (decidido y documentado, sin código).

8.1 Diagrama de integración



Fuente editable: `docs/entrega3/Diagrama_Integracion_SOA.drawio`. El diagrama es el del 13/10/2026. Línea llena: implementado; punteada: diseñado, pendiente. Las flechas I1 a I5 son las integraciones con clientes, sistemas externos y mensajería; S1 a S3, las llamadas entre servicios internos.

8.2 Componentes internos

El backend son ocho servicios Jakarta EE, cada uno un WAR propio desplegado en el mismo WildFly 41, con su esquema de base. Se hablan solo por REST o por JMS, nunca por clases del otro servicio.

Componente	Ruta base	Integra con	Estado
`ServicioDeUsuarios` (stateless)	`/api/usuarios`	Lo consumen todos: firma los JWT, publica la clave pública en `/api/usuarios/jwks` y entrega el token de rol `servicio`	Implementado
`ServicioDeEventos` (stateless)	`/api/eventos`	Recibe de Ventas la reserva y la devolución de cupo	Implementado
`ServicioDeVentas` (stateful)	`/api/v1`	Llama a Usuarios, Eventos y Pagos por REST; publica `PAGO_CONFIRMADO`; consume la cola de emisión	Implementado
`ServicioDePagos` (stateless)	`/api/v1/pagos`	Lo llama Ventas; cobra contra la interfaz `PasarelaDePagos`	Implementado, con la pasarela **simulada** (`PagosFake`)
`ServicioDeNotificaciones` (stateless)	`/api/notificaciones`	Suscriptor del tópico (aviso y auditoría)	Implementado; el aviso es **simulado** (una línea en el log)
Facturación, Reventa y Validación	`/api/facturacion`, `/api/reventa`, `/api/validacion`	Facturación: ARCA por SOAP (sección 11)	Desplegados; solo responden `health`. Diseñados, pendientes

8.3 Sistemas externos y canal

Sistema	Naturaleza	Canal previsto	Estado
Mercado Pago	Partner moderno	REST (cobro) y webhook entrante	Diseñado, pendiente (decisión del 13/10, sección 12)
ARCA (ex AFIP), WSFEv1	Sistema legado, solo SOAP con WSDL	SOAP 1.1	Diseñado, pendiente; se implementará contra un ARCA simulado (sección 11)
Proveedor de correo	Sin elegir	Sin definir	No clasificado: trabajo futuro

El broker (ActiveMQ Artemis embebido en WildFly) **no es un sistema externo**: es infraestructura propia, en el mismo proceso. Hoy el código no hace ninguna llamada a un sistema externo: no hay cliente SOAP, ni WSDL, ni cliente de Mercado Pago, ni envío de correo (en `backend/servicios/facturacion` solo existe `FacturacionApplication.java`).

8.4 Modelo de integración y por qué no un ESB

La integración se resuelve con tres piezas, sin un bus de integración como producto:

1. **REST directo entre servicios** cuando quien llama necesita la respuesta para seguir. Cada llamada sale de una clase cliente propia de Ventas, con tiempos de espera explícitos: `ClienteDeEventos` y `ClienteDeUsuarios` (2 s para conectar, 5 para leer) y `ClienteDePagos` (2 y 10, porque Pagos espera a su pasarela).

2. **El broker JMS** para lo que puede resolverse después; quien publica no conoce a quienes consumen.
3. **Un adaptador por sistema externo detrás de una interfaz propia.** Existe `PasarelaDePagos` con `PagosFake`, elegida por `FabricaDePasarela`; el adaptador de Mercado Pago sería otra implementación. Facturación seguirá el mismo esquema (`AutorizadorDeComprobantes`, sección 11).

Por qué no un ESB: con 11 nodos (8 servicios, Mercado Pago, ARCA y el broker) hay 55 conexiones posibles; el diseño usa 18 y hoy funcionan 8. Un producto de ESB sumaría infraestructura sin resolver un problema que el sistema tenga hoy.

8.5 Criterio de clasificación

- **Síncrona** cuando quien llama necesita la respuesta para decidir cómo sigue y la operación es corta.
- **Asincrónica** cuando quien publica puede contestarle a su cliente sin esperar, y conviene que una demora o una caída del otro lado no lo afecte. Dentro de lo asincrónico, **cola** para lo que resuelve un único responsable una sola vez y **tópico** para lo que les interesa a varios que no se conocen.

8.6 Clasificación de las integraciones (tabla de la Entrega Parcial N.º 2, al día)

N.º	Integración	Tipo y canal	Por qué	Si el otro lado no responde	Estado
I1	Clientes → API REST de Palco	Síncrona · REST (JSON sobre HTTP)	La persona espera en pantalla si hay lugar y si el pago pasó	El cliente repite con la misma `Idempotency-Key`: no se reserva ni se cobra de nuevo	Implementado (`docs/entrega3/evidencia/reproduccion.txt`, pasos 2 a 4)
I2	`ServicioDePagos` → Mercado Pago: cobro	Síncrona · REST	Pagos necesita el resultado para contestarle a Ventas	Pagos responde 504 y no guarda nada; Ventas reintenta con la misma clave	Diseñado, pendiente. Hoy cobra `PagosFake` (**simulado**)
I2	Mercado Pago → `ServicioDePagos` : webhook	Canal síncrono · REST, para un resultado que la compra recibe de forma diferida	Solo para pagos que la pasarela no resuelve en el momento	La pasarela reenvía; Pagos descarta repetidos con un `UPDATE` condicional	Diseñado, pendiente. No existe la ruta
I3	Facturación → ARCA	Síncrona · SOAP con WSDL; **disparo asincrónico** por el tópico y reintento asincrónico por `cola.reintento-facturacion`	ARCA solo ofrece SOAP (pedido y respuesta). La compra no necesita la factura para confirmarse: el pago ya está cobrado	Tope de 5 s; la factura queda `PENDIENTE` y se reintenta por cola; agotados los intentos, DLQ	Diseñado, pendiente. Toda venta responde `facturaEstado` `NO_APLICA`

N.º	Integración	Tipo y canal	Por qué	Si el otro lado no responde	Estado
I4	Ventas → `cola.emision-entradas` → `EmisionDeEntradasMDB`	Asincrónica · cola JMS	Emitir no tiene que demorar ni hacer fallar el pago; un solo responsable, una sola vez	El mensaje espera en el broker; si el consumidor falla, 5 entregas y DLQ	Implementado (sección 10)
I5	Ventas → `topico.compra-confirmada` → `AvisoDeCompraMDB` y `AuditoriaDeComprasMDB`	Asincrónica · tópico JMS	Dos interesados en el mismo hecho que no se conocen; sumar otro no toca Ventas	Auditoría es durable (el broker le guarda lo pendiente); el aviso no, y se acepta perderlo	Implementado; el aviso es **simulado** (log)

Cambio respecto del documento entregado: la fila I3 de la Entrega Parcial N.º 2 decía que la factura se pedía *dentro* de la compra (Ventas → Facturación, S3). El diseño de la facturación por SOAP recomienda (sección 11.4, decisión 2) que Facturación se suscriba al tópico: la llamada a ARCA sigue siendo síncrona, pero el disparo es asincrónico, para no retener una conexión de la base dentro de `confirmarCompra`. Esa decisión figuraba como pendiente de aprobación del grupo al 16/10; la tabla ya la refleja y conviene confirmarla antes del 13/11.

8.7 Llamadas entre servicios internos

Todas son síncronas por REST: quien llama necesita el resultado.

N.º	Llamada	Para qué	Estado
S1	Ventas → Eventos: `POST /api/eventos/{eventoId}/sectores/{sectorId}/cupo`, rol `servicio`	Reservar cupo al abrir la compra; devolverlo al cancelar o vencer	Implementado (`reproduccion.txt`, paso 10)
S1	Ventas → Usuarios: `GET /api/usuarios/yo` con el token del comprador	Datos del comprador antes de abrir la compra	Implementado (`ClienteDeUsuarios`)
S1	Ventas → Pagos: `POST /api/v1/pagos`, rol `servicio`, con `Idempotency-Key`	Cobrar	Implementado (`ClienteDePagos`, `reproduccion.txt`, paso 6)
S2	Todos → Usuarios: JWKS y token de servicio	Verificar tokens y autenticar llamadas internas	Implementado
S3	Facturación → Ventas (`PUT /api/v1/ventas/{ventaId}/factura`, según el diseño de la sección 11); Reventa → Usuarios, Pagos y Ventas; Validación → Ventas	Informar la factura, revender, validar en puerta	Diseñado, pendiente

Entre servicios no hay transacción compartida: cada llamada confirma por su cuenta. La regla para el cupo es que, ante la duda, queda un lugar sin vender y nunca uno vendido dos veces.

9. API REST

9.1 Rutas y versionado

Servicio	Ruta base	Motivo
Usuarios	`/api/usuarios`	Existía antes de la convención de versionado
Eventos	`/api/eventos`	Ídem
Notificaciones	`/api/notificaciones`	Ídem
Ventas	`/api/v1` (`/ventas`, `/entradas`)	Servicio nuevo: versión en la ruta
Pagos	`/api/v1/pagos`	Servicio nuevo
Facturación, Reventa, Validación	`/api/<servicio>/health`	Solo control de estado

La regla es que los servicios nuevos van bajo `/api/v1`. Los tres primeros no se migraron para no romper al frontend ni a la colección de Postman; la diferencia queda declarada.

9.2 Operaciones implementadas

Operación	Rol (`@RolesAllowed` / `@PermitAll`)	Respuesta correcta
`POST /api/usuarios/registro`, `/verificacion`, `/verificacion/reenvio`, `/login`, `/servicios/token`	`@PermitAll` (el token de servicio lo protege un secreto compartido)	201 / 200 / 204 / 200 / 200
`GET /api/usuarios/yo`	`user`, `organizer`, `staff`	200
`GET /api/usuarios/jwks`	`@PermitAll`	200
`GET /api/eventos`, `GET /api/eventos/{slug}`	`@PermitAll`	200
`POST /api/eventos`	`organizer`	201 con `Location`
`POST /api/eventos/{eventoId}/sectores/{sectorId}/cupo`	`servicio`	200
`POST /api/v1/ventas` (abre la compra y reserva cupo)	`user`	201 con `Location`
`GET /api/v1/ventas` (las compras propias), `GET /api/v1/ventas/{ventaId}`	`user`	200
`POST /api/v1/ventas/{ventaId}/pagos`	`user`	201 con `Location` al pago
`DELETE /api/v1/ventas/{ventaId}`	`user`	204 (repetido, también 204)
`GET /api/v1/entradas/{entradaId}`	`user`	200
`POST /api/v1/pagos`, `GET /api/v1/pagos/{pagoId}`	`servicio`	201 con `Location` / 200
`GET /api/notificaciones/auditoria`	`servicio`	200
`POST /api/notificaciones/pruebas/...` (tres operaciones de prueba de mensajería)	`servicio`	Para la evidencia; no son parte del negocio

Fuentes: los `*Resource.java` de cada servicio en `backend/servicios/<servicio>/src/main/java/com/palco/<servicio>/api/`.

9.3 Convenciones

- **Errores como Problem Details (RFC 9457).** `ManejadoresDeErrores` (en `backend/common/.../rest/`) traduce cada excepción de `ErroresDeNegocio` a un cuerpo `Problema` con `Content-Type: application/problem+json` (`type`, `title`, `status`, `detail`, `instance` y, si corresponde, los errores por campo). El `type` es una ruta relativa `/problemas/<tipo>`. Nunca se devuelve una traza. El único 401 que no pasa por JAX-RS, el del token que el servidor no puede verificar, lo escribe `ErrorDeAutenticacionServlet` con el mismo formato (página de error registrada por el `web-fragment.xml` de `common`).
- **Códigos que usa la API:**

Código	Cuándo	Ejemplo
400 `datos-invalidos`	Validación de Bean Validation o datos mal formados	Falta la cabecera `Idempotency-Key`
401 `no-autenticado`	Sin token o credenciales incorrectas; lleva `WWW-Authenticate: Bearer`. El login responde el mismo 401 con el mismo mensaje exista o no el email	`POST /api/v1/ventas` sin token
401 `token-invalido`	Token con la firma alterada o mal formado: lo rechaza el mecanismo MP-JWT del servidor antes de llegar al recurso. Lleva `WWW-Authenticate: Bearer error="invalid_token"` y las cabeceras CORS	`GET /api/v1/ventas` con un token alterado
402	Pago rechazado por la pasarela	`tok_rechazado` en `PagosFake`
403	Rol insuficiente	Token de organizador en `GET /api/v1/ventas`
404	No existe o no es del que pregunta	La venta de otro comprador responde 404, no 403, para no revelar que existe
409	Conflicto de estado	Cancelar una venta paga; misma clave con otro pedido (`clave-reutilizada`)
422	Regla de negocio incumplida	DNI repetido en la misma compra
429 `demasiados-intentos`	Cinco logins fallidos en 15 minutos para un mismo email: el siguiente se rechaza aunque la clave sea correcta, también si el email no tiene cuenta. Lleva `Retry-After` con los segundos que faltan	Sexto login seguido con la clave mal
504	El servicio llamado no respondió	Pagos sin respuesta (`sin-respuesta`)
500 `error-inesperado`	Cualquier otra excepción: se responde un mensaje genérico con el miembro de extensión `idError`, un UUID que queda escrito en el log junto con la traza para encontrarla cuando alguien reporta el error. Los demás errores no llevan `idError`	—

- **`Idempotency-Key`.** Obligatoria en `POST /api/v1/ventas`, `POST /api/v1/ventas/{id}/pagos` y `POST /api/v1/pagos`. La clave se guarda en la venta (única por comprador) y en el pago; repetir con la misma clave devuelve la misma venta o el mismo pago, en su estado actual, sin reservar ni cobrar otra vez. La misma clave con un pedido distinto da 409. Ventas no le pasa a

Pagos la clave del comprador tal cual, sino esa misma con el id de la venta adelante (`ClienteDePagos.claveParaPagos`): en Ventas la clave es única por comprador y en Pagos en todo el servicio, así que dos compradores con la misma clave no chocan.

- **`Location`.** Las creaciones responden 201 con la cabecera `Location` del recurso creado (venta, pago, evento).
- **Enlaces en las respuestas (`_links`).** Las respuestas de Ventas llevan `_links` (clase `Enlaces`): `self` siempre, y `pagar` (`POST`) y `cancelar` (`DELETE`) solo mientras la venta está `EN_CHECKOUT`; cada entrada lleva su `self`. Los `href` son rutas sin host. Los demás servicios no llevan enlaces.
- **Seguridad declarativa.** Cada operación declara `@RolesAllowed` o `@PermitAll`. JWT firmado por Usuarios, verificado en cada servicio con la clave del JWKS. CORS en `common/rest/CorsFilter`, que admite las cabeceras `Authorization`, `Idempotency-Key` e `If-None-Match`.
- **Dinero** en pesos enteros (`long`); fechas en UTC con `Z`.

9.4 Cambios de seguridad del 16/10

Los tres cambios que figuraban en curso están en el código desde el 16/10 y se comprobaron el 16/10 con el servidor armado desde cero.

Cambio	Dónde	Estado
Límite de intentos de login (SEC-W04): con 5 fallidos en 15 minutos para un email, 429 Problem Details con `Retry-After`	`ServicioDeUsuarios`, `IntentosDeLogin` y la tabla `intento_login` (sección 4.1)	Implementado y probado: seis logins fallidos de un mismo email dieron cinco 401 y, el sexto, 429 con `Retry-After: 898`
401 por token inválido como Problem Details, con `WWW-Authenticate` y CORS. Antes salía en `text/html`, sin CORS ni `WWW-Authenticate`; el 401 sin token y el 403 por rol ya salían bien desde la aplicación	`ErrorDeAutenticacionServlet` y `web-fragment.xml` en `common`	Implementado y probado: `GET /api/v1/ventas` con un token inválido y `Origin: http://localhost:5173` respondió 401, `application/problem+json`, `type /problemas/token-invalido`, `WWW-Authenticate: Bearer error="invalid_token"` y `Access-Control-Allow-Origin: http://localhost:5173`
Id de error en el 500: generarlo, loguearlo con la traza y devolverlo como `idError`	`ManejadoresDeErrores.Inesperado`	Implementado, probado con tests de `common` (`ErrorInesperadoTest`); en vivo no se provocó un 500

Queda abierta la vigencia de la sesión (SEC-W05, hoy 8 horas): está para decidir en grupo.

9.5 Contrato OpenAPI

`docs/api/openapi.yaml` (OpenAPI 3.1.0, versión 1.1.0 del documento) describe las rutas de Usuarios (registro, verificación, reenvío, login, perfil, token de servicio, JWKS), Eventos (catálogo, detalle, alta, cupo), Ventas (`/v1/ventas` con `POST` y `GET`, `/v1/ventas/{ventaId}` con `GET` y `DELETE`, `/v1/ventas/{ventaId}/pagos`, `/v1/entradas/{entradaId}`) y Pagos (`/v1/pagos`, `/v1/pagos/{pagoId}`). En la Entrega Parcial N.º 2 el contrato estaba desactualizado (cuatro operaciones); se puso al día después (commit "OpenAPI y colección de Postman al día"). Notificaciones no figura en el contrato.

9.6 Colección de Postman

`docs/api/palco-api.postman_collection.json` la genera `scripts/postman/generar.js` (no se edita a mano) y se corre entera, en orden y sin intervención (`docs/api/GUIA_POSTMAN.md`).

Carpeta	Requests	Qué prueba
1 Usuarios	10	Registro, login correcto, incorrecto y sin verificar, perfil con y sin token, token de servicio con secreto incorrecto, JWKS
2 Eventos	13	Catálogo, filtro, detalle, 404, alta sin token, con `organizer` e inválida, reserva y liberación de cupo con rol `servicio` y 403 con otro rol
3 Diseño v1 (Ventas)	31	Compra con reserva de cupo, idempotencia (incluida la clave reutilizada, 409), pago rechazado (402) y aprobado, emisión asincrónica, compras propias, aislamiento entre compradores, control de rol y cancelación
4 Seguridad (clase 11)	15	Los criterios de la matriz de seguridad de la clase 11 que se prueban por HTTP, cada uno con su id: recurso ajeno pedido por otro comprador (404), rol equivocado (403), sin token (401), mismo 401 para email inexistente y clave mal, token alterado y sin firma (401 Problem Details), vigencia del token, 400 sin información interna, perfil sin contraseña y límite de intentos de login (429). Usa las cuentas de prueba `titular@palco.test` y `otro.titular@palco.test` y toma una compra emitida que deja la carpeta 3; el request del límite de intentos usa un email nuevo en cada corrida para no bloquear las cuentas de prueba
Fuera de la corrida	2 + 4	Pasos manuales de verificación de email; y cuatro requests de reventa y validación marcados `PENDIENTE` (hoy responden 404)

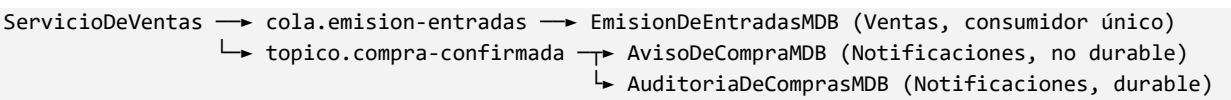
La colección tiene 75 requests: los 69 de la corrida y los 6 que quedan fuera. El secreto para el token de servicio sale de la variable de la colección `secretoServicios` (por defecto `palco-desarrollo`; si el servidor arrancó con el de `backend/.env`, se pasa con `--env-var secretoServicios=...`).

Última corrida: 16/10/2026, con la colección completa contra el servidor armado desde cero: Newman cuenta 74 requests (los 69 de la colección más los cinco intentos de login que SEC-W04 manda antes de enviarse) y 252 aserciones, con 1 falla: SEC-W05c, "el token dura 15 minutos o menos", porque la sesión dura 8 horas. Queda en rojo a propósito hasta que el grupo decida la vigencia. La corrida anterior, del 14/10, dio 54 requests y 206 aserciones sin fallas con las carpetas 1 a 3. Hay además una colección más vieja, `palco-eventos.postman_collection.json`.

10. Mensajería

10.1 El proceso

Cuando se aprueba el pago de una compra, `ServicioDeVentas.confirmarCompra` pasa la venta a `CONFIRMADA`, crea las entradas en `PENDIENTE_EMISION` y publica `PAGO_CONFIRMADO` **en la misma transacción**. El mismo texto va a dos destinos:



El comprador recibe la respuesta del pago (201, con las entradas todavía sin emitir) sin esperar a ningún consumidor. Estado: **Implementado**; el aviso es **simulado** (una línea en el log, porque no hay proveedor de correo).

10.2 Contrato de `PAGO_CONFIRMADO`

Documento normativo: `docs/api/mensajeria-pago-confirmado.md`. Clases en `backend/common`, paquete `com.palco.common.mensajeria`: `PagoConfirmadoMensaje` (record, con `aJson`, `desdeJson` y `validar`), `MensajeInvalido`, `DestinosJms` (nombres JNDI, propiedad y selector), `PublicadorPagoConfirmado` y su implementación `PublicadorPagoConfirmadoJms`.

- `TextMessage` con cuerpo JSON (JSON-B, campos en orden alfabético), entrega persistente.
- Propiedad `tipoEvento` = `PAGO_CONFIRMADO`.
- Cuerpo: `ventaId`, `compradorId`, `eventoId`, `entradas[]` (1 a 6, cada una con `entradaId`, `sector` por nombre y `titular` con `nombre` y `dni`), `montoTotal` en pesos enteros y `timestamp` en UTC truncado a segundos.
- Se rechaza (`MensajeInvalido`) un texto vacío o que no es JSON, ids vacíos o de más de 40 caracteres, 0 o más de 6 entradas, DNI que no tiene 7 u 8 dígitos (`ReglasDeNegocio.esDniValido`), monto no positivo o sin `timestamp`. El publicador valida antes de mandar.
- Compatibilidad: un campo desconocido se ignora, así que agregar campos no rompe a los consumidores.
- El test `PagoConfirmadoMensajeTest` lee el ejemplo del documento del contrato: si contrato y clase dejan de coincidir, el build falla.

Pendiente del contrato: decidir si el DNI viaja completo en el tópico (hoy viaja completo).

10.3 Broker, cola y tópico

Configuración versionada en `backend/servidor/src/main/wildfly/configurar.cli`, leída del servidor en marcha el 14/10 (`reproduccion.txt`, paso 0).

Qué	Valor
Broker	ActiveMQ Artemis 2.54.0 embebido en WildFly 41.0.1
Cola	`cola.emision-entradas`, JNDI `java:/jms/queue/emisionEntradas`
Tópico	`topico.compra-confirmada`, JNDI `java:/jms/topic/compraConfirmada`
Persistencia	Activada: los mensajes pendientes sobreviven a un reinicio (la base H2 no)
Reintentos	`address-setting=#`: `max-delivery-attempts` 5 y `redelivery-delay` 2000 ms
DLQ	La que trae el servidor, `java:/jms/queue/DLQ`
Fábrica para publicar	`java:/JmsXA`, transaccional (XA)

Por qué cola y por qué tópico. La emisión la hace un único responsable, exactamente una vez: cola punto a punto. Aviso y auditoría son interesados independientes en el mismo hecho: tópico; sumar un tercero (por ejemplo Facturación, sección 11) no obliga a tocar Ventas.

10.4 Los tres consumidores

Consumidor	Servicio	Destino y suscripción	Selector	Qué hace	Idempotencia
`EmisionDeEntradasMDB`	Ventas	Cola, consumidor único (el broker informa `consumer-count` 15: son sesiones del mismo MDB, y cada mensaje va a una sola; `broker-despues.txt`)	`tipoEvento = 'PAGO_CONFIRMADO'`	Pone el código QR a cada entrada (`PENDIENTE_EMISION` → `VALIDA`) y pasa la venta a `EMITIDA`, en una transacción `REQUIRED`	Cada cambio es un `UPDATE` que exige que la entrada siga `PENDIENTE_EMISION`; un repetido no cambia ninguna fila y se loguea "es un mensaje repetido y no cambió nada". Si la venta no existe, el mensaje se descarta con un `WARNING`
`AuditoriaDeComprasMDB`	Notificaciones	Tópico, **durable**: `clientId` `palco-auditoria`, `subscriptionName` `compras-confirmadas`, `maxSession` 1	El mismo	Guarda una fila en `registro_auditoria`	La clave de la fila es el `JMSMessageID`: una reentrega no inserta dos veces. Ante un mensaje inválido marca `setRollbackOnly()` (reintentos y DLQ)
`AvisoDeCompraMDB`	Notificaciones	Tópico, **no durable**	El mismo	Arma el aviso al comprador y lo escribe en el log (**simulado**)	No guarda estado: un repetido produce otra línea de log, sin efecto de negocio. Un mensaje inválido se descarta con un `WARNING`

Por qué durable una y no la otra. La auditoría no puede perder un hecho: si Notificaciones está detenido, el broker le guarda los mensajes. Perder un aviso que hoy es una línea de log se acepta; cuando haya correo real, esta decisión se revisa.

Por qué el selector. Hoy todos los mensajes son `PAGO_CONFIRMADO`, pero el selector deja preparado el destino para otros tipos (el diseño de la facturación por SOAP define `FACTURA_PENDIENTE` en su propia cola, sección 11.5) sin que estos consumidores tengan que cambiar.

10.5 Garantía transaccional

- `confirmarCompra` lleva `@TransactionAttribute(REQUIRED)` y el publicador `MANDATORY`: no puede publicar fuera de la transacción de quien lo llama.
- El broker participa como recurso XA; el datasource `PalcoDS` no es XA y entra como último recurso. **No es un commit en dos fases completo.**
- Si la transacción se deshace, el mensaje no sale (probado, escenario 13).
- Si el servidor se cayera entre el commit de la base y el del broker, quedaría una venta `CONFIRMADA` con entradas `PENDIENTE_EMISION` y sin mensaje. **No probado**; el reenvío queda para la conciliación y el manejo de transacciones.
- La transacción cubre solo la confirmación. La reserva de cupo es REST y se compensa devolviéndolo; el cobro se protege con la clave de idempotencia. `ServicioDePagos` lleva `NOT_SUPPORTED`: no retiene una transacción mientras espera a la pasarela.

10.6 Reintentos, DLQ y procedimiento

Si un consumidor falla, su transacción se deshace entera (no quedan entradas emitidas a medias) y el broker reentrega a los 2 segundos; después de 5 entregas, el mensaje pasa a la DLQ. En la cola de emisión eso deja la venta `CONFIRMADA` con entradas sin emitir y nada lo reintenta solo. Procedimiento:

1. **Encontrar el mensaje** en la DLQ filtrando por la propiedad `_AMQ_ORIG_QUEUE` = `jms.queue.cola.emision-entradas` (`list-messages` y `count-messages` con ese filtro).
2. **Ver la causa** en `server.log` (la línea `AMQ222149` con el id del mensaje y las 5 fallas anteriores) y corregirla; si persiste, devolverlo solo lo manda otra vez a la DLQ.
3. **Devolverlo** con `move-messages` y el mismo filtro hacia `cola.emision-entradas`. Es seguro porque la emisión es idempotente. Con `jboss-cli.bat` el filtro va en un archivo (`--file`), porque `--command` pierde las comillas.
4. **Comprobar** que la venta pasó a `EMITIDA` y sus entradas a `VALIDA`.

Estado del procedimiento: la lectura filtrada se probó el 14/10; `move-messages` **sin** filtro se usa en el escenario 12 y en la evidencia complementaria; `move-messages` **con** filtro no se ejecutó todavía.

10.7 Evidencia: escenarios 7 a 13

Salida en `docs/entrega3/evidencia/integracion-13-escenarios.txt` (14/10/2026, servidor armado desde el tag `entrega-parcial-2`, 177 comprobaciones, todas bien). Se repiten con `node scripts/pruebas/escenarios.mjs [n]`. Última corrida: 16/10/2026, con el servidor armado desde cero con los cambios de seguridad de la sección 9.4: 13 de 13 escenarios y 177 comprobaciones, todas bien (esa salida no se guardó en el repositorio).

N.º	Qué prueba	Resultado
7	Al confirmarse un pago sale un mensaje a la cola y uno al tópico, con el `tipoEvento` del contrato	Pasa
8	El consumidor de la cola emite una sola vez aunque el mensaje llegue dos veces (la copia de auditoría se mueve a la cola con `move-messages`)	Pasa: códigos, fechas de emisión y DLQ sin cambios
9	Los dos suscriptores del tópico reciben cada uno su copia: el aviso (log) y la auditoría (fila)	Pasa
10	Con el consumidor de emisión detenido (`stop-delivery`) el mensaje espera en la cola; al reanudarlo (`start-delivery`) se emite	Pasa
11	Suscripción durable: con auditoría detenida se publican mensajes y al reanudarla los recibe	Pasa (10/10)
12	Un mensaje inválido se reintenta 5 veces y termina en la DLQ sin trabar a los demás	Pasa (15/15)
13	Si la transacción se deshace después de publicar, el mensaje no sale	Pasa (11/11)

Evidencia complementaria en la misma carpeta: `suscriptor-durable.txt`, `mensajes-pendientes.txt` (64 mensajes esperando sin consumidor, procesados al desplegarlo), `venta-inexistente.txt` y `mensaje-repetido-y-reintentos.txt`. Este último se tomó con una versión modificada del servicio que no está en el repositorio, y así se declara.

11. Servicios web SOAP: Facturación con un ARCA simulado

Estado: Diseñado, pendiente. Hoy no hay WSDL, ni endpoint, ni cliente SOAP; la capa `webservices` de WildFly no está en el servidor y Facturación solo responde `health`. Toda venta responde `facturaEstado NO_APLICA`. Lo que sigue resume el diseño, previsto del 15/10 al 30/10.

11.1 Qué pide la consigna y por qué ARCA

La consigna pide para la Entrega Obligatoria N.º 2 (13/11) al menos un servicio SOAP con su WSDL, justificado como integración con un sistema externo legado. En Palco ese papel lo cumple ARCA (ex AFIP), cuyo servicio de factura electrónica WSFEv1 solo se ofrece por SOAP. **El canal no se elige: lo impone el sistema legado.** Lo propio del sistema (Ventas, Pagos, el frontend) es REST.

11.2 Qué es real y qué simulado

Pieza	Real o simulada
SOAP 1.1 sobre HTTP, WSDL publicado por WildFly, Apache CXF (subsistema `webservices`), cliente JAX-WS	Real
ARCA	**Simulado**: un WAR aparte (`backend/legado/arca`, en `/legado/arca`) con un subconjunto del WSFEv1 según el manual oficial (versión 4.7)
Autenticación WSAA (exige certificado digital)	Simulada: `Auth` con `Token`, `Sign` y `Cuit`; se acepta cualquier valor no vacío
CAE y su vencimiento	Inventados con el formato real (14 dígitos, fecha `yyyyMMdd`)
Importes	Pesos enteros (`long`) en el simulador; el WSFEv1 real usa decimales
Tópico, cola de reintento, DLQ y factura guardada	Reales

11.3 Contrato del ARCA simulado

Espacio de nombres propio (`http://palco.daii/legado/arca/wsfe`), document/literal. Operaciones: `FEDummy` (control de estado), `FECompUltimoAutorizado` (último número), `FECAESolicitar` (pedir el CAE; `Resultado` A o R) y `FECompConsultar` (saber si un pedido sin respuesta llegó a autorizarse). Valores fijos: punto de venta 1, Factura B (tipo 6), concepto servicios, consumidor final.

Errores en dos niveles, como en el WSFEv1: un **rechazo de negocio** es una respuesta normal con `Resultado` R (la factura queda `RECHAZADA`, no se reintenta); una **falla técnica** es un SOAP Fault o la falta de respuesta (queda `PENDIENTE` y se reintenta). Para la demo, la propiedad de sistema `palco.arca.modo` (`normal`, `rechaza`, `caido`, `lento`) cambia el comportamiento en caliente.

11.4 Decisiones de diseño

N.º	Decisión	Recomendada y por qué
1	Dónde vive el simulador	WAR aparte: cliente y sistema externo no comparten clases ni despliegue; se puede bajar solo el simulador para mostrar que la compra sigue

N.º	Decisión	Recomendada y por qué
2	Cómo se dispara la factura	Suscripción durable de Facturación al tópico (`FacturacionDeComprasMDB`), no una llamada dentro de `confirmarCompra`: esperar hasta 5 s ahí retendría una conexión de la base y la compra no necesita la factura. La llamada a ARCA sigue siendo síncrona. Costo: la factura ya no viaja en la respuesta de la compra
3	Quién actualiza `facturaEstado`	Facturación llama a `PUT /api/v1/ventas/{ventaId}/factura` con rol `servicio`, idempotente
4	Numeración	Como el ARCA real: pedir el último autorizado y mandar el siguiente, serializado por un `@Singleton` con `@Lock(WRITE)` (`NumeradorDeComprobantes`). Alternativa de corte: que numere el simulador
5	Cliente SOAP	Escrito a mano con una copia local del WSDL (`Service.create`), sin generador en el build; un `diff` controla que coincida con el publicado
6	Patrones	`ServicioDeFacturacion` `@Stateless`; salida detrás de `AutorizadorDeComprobantes` con un Adapter (`ClienteArcaSoap`) y un fake; Factory `FabricaDeAutorizador`

Al 16/10 estas decisiones figuraban como pendientes de aprobación del grupo.

11.5 Tiempos, reintento y transacción

- Cliente SOAP con 2 s para conectar y 5 para recibir.
- Ante timeout o Fault: la factura queda `PENDIENTE` y, en la misma transacción, sale un mensaje `{"ventaId":"...","intento":1}` con `tipoEvento` = `FACTURA_PENDIENTE` a `cola.reintento-facturacion`, que consume `ReintentoDeFacturacionMDB`. Esa cola tendría reintentos propios con espera creciente (10, 20, 40 y 80 s) y después la DLQ.
- Antes de reintentar se consulta `FECompConsultar`: si ARCA ya había autorizado el comprobante y se perdió la respuesta, se toma ese CAE y no se pide otro.
- La llamada SOAP va antes de tocar la base, para no retener una conexión mientras se espera a ARCA.

11.6 Consulta y datos

`GET /api/facturacion/facturas/{ventaId}` (roles `user`, `staff`, `servicio`; un comprador ve solo las suyas). Tabla `factura` en el esquema `facturacion` con estado `PENDIENTE`, `AUTORIZADA` o `RECHAZADA`, CAE, número, intentos y motivo de rechazo.

11.7 Riesgos y preguntas abiertas

- Sumar la capa `webservices` obliga a rearmar el servidor una vez (se vacían la base H2 y los mensajes).
- JAX-WS no está en la API de Jakarta EE 11 que usa el repositorio: hace falta la dependencia.
- Preguntas a la cátedra: si vale un sistema legado simulado y declarado como tal; si cuenta como "integración síncrona vía SOAP" una llamada síncrona disparada desde un consumidor del

tópico; si el reintento por cola cuenta como un segundo proceso por cola; si el WSDL va completo en el documento.

12. Sistemas externos

12.1 Mercado Pago

Estado: Diseñado, pendiente (decisión del 13/10). No se crea cuenta de Mercado Pago ni se integra la pasarela real; `ServicioDePagos` sigue con `PagosFake` y el requisito de servicio REST lo cubre la API propia (I1). Se retoma si la cátedra pide una pasarela real. Del diseño queda el diagrama de secuencia (sección 13.2).

Qué existe hoy en el código de Pagos: `IPagos` y `ServicioDePagos` (`@Stateless`, `NOT_SUPPORTED`), la interfaz `PasarelaDePagos`, `ResultadoPasarela`, `FabricaDePasarela` y `PagosFake`. El pago se guarda en el esquema de Pagos con su clave de idempotencia; repetir la clave devuelve el pago guardado.

Qué simula `PagosFake`: el token `tok_test` aprueba y `tok_rechazado` rechaza; no mueve plata. Los estados `PENDIENTE`, `VENCIDO` y `REEMBOLSADO` están en el enumerado `EstadoPago`, pero nadie los escribe. Tampoco existen el webhook ni el reembolso.

Diseño del adaptador: `MercadoPagoPagos` como otra implementación de `PasarelaDePagos` (Adapter), elegida por configuración en `FabricaDePasarela` (Factory). Cobro con `POST /v1/payments` y `X-Idempotency-Key`; traducción de `approved`, `rejected` e `in_process` a `APROBADO`, `RECHAZADO` y `PENDIENTE`; webhook con validación de firma antes de leer el cuerpo. La firma del webhook no se pudo verificar contra la documentación oficial y queda como punto abierto.

12.2 ARCA

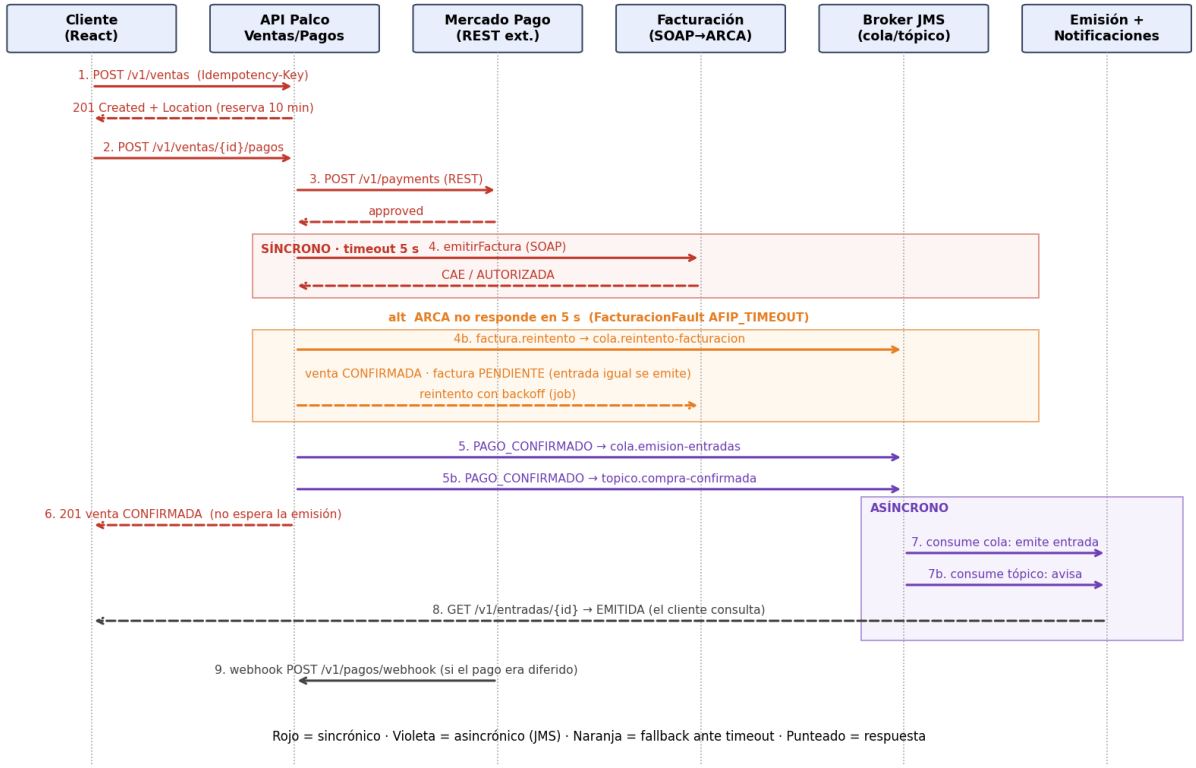
Estado: Diseñado, pendiente, contra un ARCA simulado (sección 11). Hoy no se factura.

12.3 Qué se simula hoy

Parte	Hoy
Cobro	Real hasta Pagos (REST, guardado, control por rol); la pasarela es **simulada** (`PagosFake`)
`CobrosFake` en Ventas	Simulador local que solo se activa con `palco.ventas.cobros=fake`; ningún archivo del repositorio lo define y no estaba activo en la evidencia
Aviso al comprador	**Simulado**: una línea en el log
Factura	No se emite
Correo	No existe; el mensaje trae solo `compradorId`, así que el email habría que pedírselo a Usuarios
Broker, cola, tópico, DLQ y MDB	Reales

13. Diagramas de secuencia

13.1 Compra de punta a punta (actividad de la clase 10)



Fuente: `actividad_integradora_clase10/diagrama_secuencia.png`. Rojo: síncrono; violeta: asincrónico (JMS); naranja: alternativa ante timeout; punteado: respuesta.

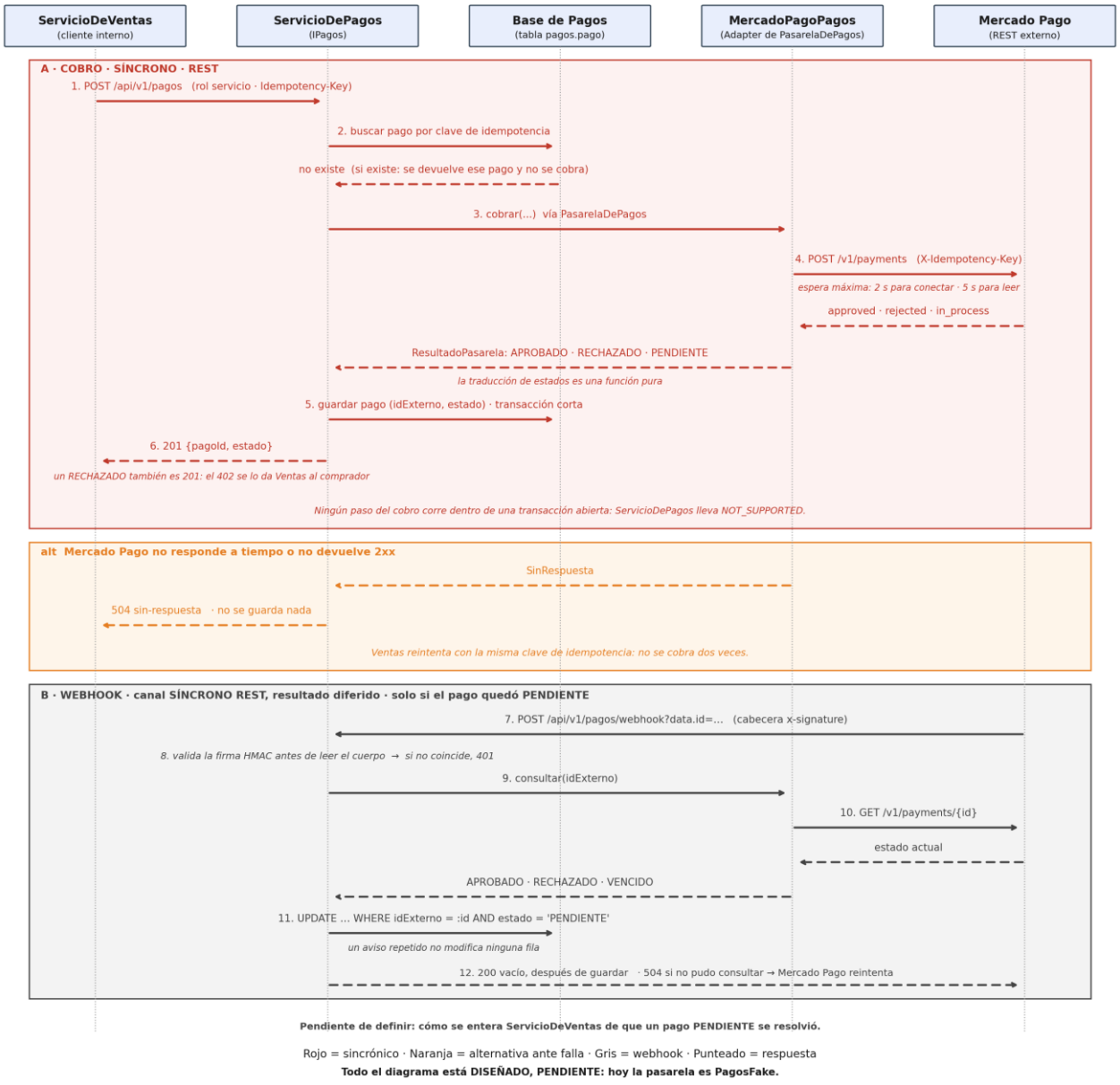
Paso a paso:

1. El cliente abre la compra con `POST /v1/ventas` e `Idempotency-Key`; la API responde 201 con `Location` y una reserva de 10 minutos.
2. El cliente paga con `POST /v1/ventas/{id}/pagos`.
3. La API cobra en Mercado Pago (`POST /v1/payments`) y recibe `approved`.
4. Con un tope de 5 s pide la factura a Facturación por SOAP y recibe el CAE. Alternativa: si ARCA no responde en 5 s, la factura pasa a `cola.reintento-facturacion`, la venta queda `CONFIRMADA` con factura `PENDIENTE` y un trabajo la reintenta.
5. Publica `PAGO_CONFIRMADO` en `cola.emision-entradas` y en `topico.compra-confirmada`.
6. Responde 201 con la venta `CONFIRMADA`, sin esperar la emisión.
7. Los consumidores emiten la entrada (cola) y avisan (tópico).
8. El cliente consulta la entrada.
9. Si el pago fuera diferido, Mercado Pago llama al webhook.

Coincidencia con el código:

Paso	¿Coincide?
1, 2, 5, 6, 7	Sí. Detalles: Ventas y Pagos son dos servicios (el diagrama los junta en "API Palco"); el pago responde 201 con `Location` al pago; el tópico tiene dos suscriptores (aviso y auditoría), no uno
3	No: el cobro va a `ServicioDePagos` por `POST /api/v1/pagos`, y Pagos cobra con `PagosFake`. Mercado Pago es diseño, pendiente
4	No: no hay facturación. Además, el diseño de la facturación por SOAP (sección 11.4) lo cambia: la factura se dispara desde el tópico, fuera de la compra
8	Parcial: `GET /api/v1/entradas/{id}` devuelve la entrada `VALIDA`; `EMITIDA` es el estado de la venta, no de la entrada
9	No existe la ruta del webhook

13.2 Cobro con Mercado Pago



Fuente: `docs/entrega-obligatoria-2/diagramas/diagrama_secuencia_mercadopago.png` (también en SVG, generado por `diagrama_secuencia_mercadopago.py`). Todo el diagrama es diseño, pendiente.

Paso a paso:

- **A. Cobro, síncrono por REST.** (1) Ventas llama a `POST /api/v1/pagos` con rol `servicio` e `Idempotency-Key`. (2) Pagos busca un pago con esa clave; si existe, lo devuelve sin cobrar. (3) Cobra a través de `PasarelaDePagos`. (4) El adaptador `MercadoPagoPagos` llama a `POST /v1/payments` con `X-Idempotency-Key` y espera como máximo 2 s para conectar y 5 para leer; recibe `approved`, `rejected` o `in_process`, que traduce a `APROBADO`, `RECHAZADO` o `PENDIENTE`. (5) Guarda el pago en una transacción corta. (6) Responde 201; un rechazo también es 201 y el 402 se lo da Ventas al comprador. Ningún paso corre dentro de una transacción abierta (`NOT_SUPPORTED`).
- **Alternativa.** Si Mercado Pago no responde a tiempo, Pagos responde 504 `sin-respuesta` sin guardar nada y Ventas reintenta con la misma clave.
- **B. Webhook, solo si el pago quedó `PENDIENTE`.** (7) Mercado Pago llama a `POST /api/v1/pagos/webhook` con la cabecera `x-signature`. (8) Pagos valida la firma HMAC antes de leer el cuerpo (401 si no coincide). (9 y 10) Consulta el estado real con `GET /v1/payments/{id}`. (11) Actualiza con un `UPDATE` condicional sobre `estado = 'PENDIENTE'`, así que un aviso repetido no cambia nada. (12) Responde 200, o 504 si no pudo consultar, para que Mercado Pago reintente.

Coincidencia con el código: los pasos 1, 2, 5, 6 y la alternativa (504 sin guardar, reintento con la misma clave) coinciden con lo que hace hoy `ServicioDePagos` contra `PagosFake`. Los pasos 4 y 7 a 12 no existen. El diagrama deja explícitamente sin definir cómo se entera Ventas de que un pago `PENDIENTE` se resolvió.

13.3 Otros diagramas existentes

Los talleres de las clases 6, 8 y 9 (`docs/clases/`) tienen diagramas propios en PNG, que no son de secuencia o son anteriores a la implementación. El de la integración SOA está en la sección 8.1. No hay otros diagramas de secuencia en el repositorio.

13.4 Diagramas que faltan para el 13/11

La consigna pide un diagrama de secuencia por integración para la Entrega Obligatoria N.º 2. Faltan:

Integración	Diagrama que falta	Debe mostrar
I1 + S1	Compra tal como está implementada	Cliente, Ventas, Usuarios (`/yo`), Eventos (cupo), Pagos (`PagosFake`), broker; idempotencia, 402, 504 y la compensación del cupo
I4	Emisión por cola	Publicación en la transacción, `EmisionDeEntradasMDB`, `UPDATE` condicional, reentrega, DLQ y el procedimiento de recuperación
I5	Aviso y auditoría por tópico	Suscripción durable y no durable, `JMSMessageID` como clave, suscriptor detenido
I3	Facturación SOAP con ARCA simulado	Disparo por el tópico, `FECAESolicitar` con tope de 5 s, Fault y rechazo, `cola.reintento-facturacion`, `FECompConsultar`, `PUT .../factura` a Ventas

Integración	Diagrama que falta	Debe mostrar
S2	Autenticación	Login, JWT, JWKS y token de rol `servicio` entre servicios
S1	Cancelación y vencimiento	`DELETE`, barrido de reservas vencidas y devolución de cupo a Eventos
I2	Cobro con Mercado Pago	Existe como diseño (13.2); hace falta actualizar el de la clase 10 para que no lo muestre como implementado
S3	Reventa y validación en puerta	Diseñados, pendientes: diagrama si se implementan antes del 27/11

14. Seguridad

La seguridad se resuelve en tres lugares: el servicio de Usuarios (cuentas, contraseñas y emisión de tokens), la librería compartida `backend/common` (verificación de tokens, CORS, formato de errores, protección del log) y la configuración de WildFly (`backend/servidor/src/main/wildfly/configurar.cli`). Esta sección describe el código tal como está hoy y marca lo débil. Los cambios de seguridad del 16/10 (límite de intentos de login, id de error en el 500, PBKDF2 a 600.000 iteraciones, 401 por token inválido en Problem Details y con CORS, y secreto de servicios propio en desarrollo) están en el repo desde el 16/10 (commits df9f9ed a 0cbb2e9) y se verificaron ese día.

14.1 Autenticación: registro, verificación del email y login

Paso	Operación	Qué hace	Estado
Registro	`POST /api/usuarios/registro` (`@PermitAll`)	Crea la cuenta con la contraseña hasheada y genera un código de verificación. Edad mínima: 16 años (`EDAD_MINIMA`).	Implementado
Envío del código	—	El código se escribe en el log del servidor (`ServicioDeUsuarios.enviarCodigoNuevo`). No se manda ningún email.	Simulado
Verificación	`POST /api/usuarios/verificacion` (`@PermitAll`)	Compara el hash SHA-256 del código en tiempo constante. Vence a los 15 minutos (`VIGENCIA_CODIGO`) y admite 5 intentos (`MAX_INTENTOS_CODIGO`); después hay que pedir otro. Al verificar, la cuenta queda con el email y la identidad verificados, así que ya puede comprar.	Implementado
Reenvío	`POST /api/usuarios/verificacion/reenvio` (`@PermitAll`)	Renueva el código, con 30 segundos de espera mínima entre envíos (`ESPERA_REENVIO`).	Implementado
Login	`POST /api/usuarios/login` (`@PermitAll`)	Devuelve un JWT. Si el email no existe, igual calcula un hash contra `HASH_FALSO` para que el tiempo de respuesta no delate qué emails están registrados, y el mensaje de error es siempre el mismo ("Revisá el email y la contraseña.").	Implementado
Perfil	`GET /api/usuarios/yo` (`user`, `organizer`, `staff`)	Datos de la cuenta del token. El frontend lo usa para confirmar la sesión.	Implementado

Paso	Operación	Qué hace	Estado
Límite de intentos de login	`POST /api/usuarios/login`	Con 5 fallidos en 15 minutos para un mismo email (`MAX_INTENTOS_LOGIN`, `VENTANA_INTENTOS_LOGIN`), el pedido siguiente responde 429 Problem Details (`/problemas/demasiados-intentos`) con `Retry-After` en segundos, aunque la contraseña sea la correcta y también para un email que no existe. Un login correcto borra los fallidos de ese email.	Implementado y probado (tests y en vivo)

Límite de intentos de login (SEC-W04). El control va antes de verificar la contraseña: si la correcta entrara durante el bloqueo, el sexto intento acertado le abriría la cuenta a quien está probando. Se cuenta por email, exista o no la cuenta, con el mismo texto en el 401 y en el 429, y el 429 no hace el cálculo PBKDF2, así que no delata qué emails están registrados. La ventana es deslizante: el bloqueo se levanta cuando el más viejo de los 5 fallidos sale de los 15 minutos, y eso es lo que informa `Retry-After` (redondeado hacia arriba, nunca menos de 1); los intentos bloqueados no se anotan, para que el bloqueo no se estire con cada reintento. Los fallidos se guardan en la base, en la tabla `intento_login` (migración `V4__intentos_de_login.sql` de Usuarios), a través del Session Bean stateless `IntentosDeLogin`, y no en memoria: así el límite vale igual con varias instancias y no se reinicia con un redespliegue. La tabla guarda el SHA-256 del email normalizado y no el email, porque también anota emails sin cuenta. Como los errores de negocio deshacen la transacción, el 401 del login es `CredencialesIncorrectas`, con `@ApplicationException(rollback = false)`, para que el intento fallido quede guardado. Las constantes están en `ServicioDeUsuarios` y no en `ReglasDeNegocio` porque son de seguridad de Usuarios, como `MAX_INTENTOS_CODIGO`, y no reglas del negocio de venta de entradas. Lo prueban `LimiteDeIntentosDeLoginTest` (con un reloj inyectable), `MigracionesEnH2Test`, `ErroresDeNegocioTest` y el request SEC-W04 de Postman; en vivo, el 16/10, seis logins fallidos de un mismo email dieron cinco 401 y un 429 con `Retry-After: 898`. Lo débil: el bloqueo es por email, así que alguien que conoce el email de otra persona puede bloquearle el login 15 minutos. Se acepta para esta etapa; contar también por IP lo atenuaría.

Contraseñas. `usuarios/seguridad/Contrasenas.java` usa PBKDF2-HMAC-SHA256, incluido en Java (sin dependencias), con sal aleatoria de 16 bytes, 600.000 iteraciones (lo que recomienda hoy OWASP para PBKDF2-SHA256; antes eran 310.000) y hash de 256 bits. Se guarda como `pbkdf2-sha256$<iteraciones>$<sal>$<hash>` y la verificación compara en tiempo constante (`MessageDigest.isEqual`). Se eligió PBKDF2 porque viene en la JVM; bcrypt o Argon2 pedían una librería más. Como el formato guarda las iteraciones de cada hash, los hashes viejos de 310.000 siguen verificando y subirlas no obligó a migrar datos. El hash falso del login (`HASH_FALSO`) se arma con `Contrasenas.hashear`, así que cuesta lo mismo que uno nuevo; la migración `V5__cuentas_de_prueba_con_600000_iteraciones.sql` rehashea las cuentas de prueba a 600.000, con la misma contraseña, para que no verifiquen en la mitad de tiempo que el hash falso y esa diferencia no deje ver qué emails existen. Lo débil: una cuenta registrada antes del cambio conserva su hash de 310.000 hasta que cambie la contraseña; rehashear al iniciar sesión cerraría esa diferencia y queda propuesto. La contraseña nunca vuelve en una respuesta: los DTO de `usuarios/api/Dtos.java` no la incluyen.

14.2 Tokens JWT

Aspecto	Cómo está	Archivo
Algoritmo	RS256 (`SHA256withRSA`), con `kid` en la cabecera	`usuarios/seguridad/Tokens.java`
Claves	Par RSA de 2048 bits. Si existe `PALCO_JWT_CLAVE_PRIVADA` (PKCS#8 en base64) se usa esa; si no, se genera una al desplegar y los tokens anteriores dejan de valer.	`usuarios/seguridad/ClavesJwt.java`
Publicación	Clave pública como JWKS en `GET /api/usuarios/jwks` (`@PermitAll`)	`usuarios/api/UsuariosResource.java`
Verificación	Cada servicio usa MicroProfile JWT: descarga el JWKS de Usuarios y exige el emisor `palco`. Los roles salen del claim `groups`.	`common/src/main/resources/META-INF/microprofile-config.properties`
Duración del token de usuario	8 horas (`DURACION_SESION`). Sin refresh token ni revocación al cerrar sesión.	`usuarios/negocio/ServicioDeUsuarios.java`
Identidad en el código	`Sesion.usuarioId` toma el `sub` del token; ningún recurso recibe el id del usuario por parámetro.	`common/seguridad/Sesion.java`

Por qué RS256 y no HS256. Con clave asimétrica solo Usuarios puede firmar; los otros siete servicios verifican con la clave pública y no guardan ningún secreto. Con HS256 cada servicio tendría la clave con la que se fabrican tokens.

Token de servicio entre servicios. Cuando un servicio llama a otro (por ejemplo, Ventas a Pagos) usa el esquema de credenciales de cliente: presenta un secreto compartido en `POST /api/usuarios/servicios/token` y Usuarios le firma un token con el rol `servicio` que dura 5 minutos (`DURACION_TOKEN_DE_SERVICIO`). `common/seguridad/TokenDeServicio.java` lo guarda, lo renueva cuando le quedan menos de 30 segundos y, si el servicio llamado responde 401 (pasa cuando se redespliega Usuarios y cambia la clave), lo invalida y reintenta una sola vez. Es un bean `@ApplicationScoped` y no un Session Bean: hay un único token por servicio, compartido por todos los pedidos (no es la conversación de un usuario, y un stateless no podría recordarlo), y sus métodos son `synchronized`. Las operaciones internas llevan `@RolesAllowed("servicio")` (siete en el código), así que un usuario común no puede usarlas aunque conozca la ruta.

Lo débil. Ocho horas de vida sin revocación: un token robado sirve hasta que vence. Está planteado elegir entre bajarlo a 15-30 minutos con un refresh token revocable o dejar escrita la justificación para el alcance del trabajo práctico; las opciones están planteadas (a cargo de Joaquin Molina) y todavía no está decidido. Mientras tanto, el request SEC-W05c de Postman ("el token dura 15 minutos o menos") queda en rojo a propósito.

Además, si falta `PALCO_SECRETO_SERVICIOS`, el secreto vale `palco-desarrollo`, que está escrito en el repo: quien leyó el código podría pedir un token de servicio a un servidor que arrancó así. El servicio lo avisa en el log, pero no se niega a arrancar. Para atenuarlo, el servidor de desarrollo arranca con un secreto propio definido en `backend/.env` (que `dev.sh` carga y que no se versiona); los guiones de demo, de escenarios y de evidencia lo toman de la variable o de `backend/.env`, y la colección de Postman, de la variable `secretoServicios`. En la corrida del 16/10 el log no mostró el aviso del secreto de desarrollo. Que un servidor fuera de desarrollo no arranque sin secreto está propuesto (una variable `PALCO_ENTORNO` y una comprobación al desplegar Usuarios): diseñado, pendiente, porque toca `backend/common`.

14.3 Autorización declarativa por rol

Cada operación de cada recurso lleva `@RolesAllowed` o `@PermitAll`, y `configurar.cli` activa `resteasy-role-based-security` para que esas anotaciones se apliquen. Roles: `user` (comprador), `organizer`, `staff` (puerta) y `servicio` (llamadas entre servicios).

Operaciones	Rol
Registro, verificación, reenvío, login, JWKS, pedido del token de servicio, catálogo de eventos, health	`@PermitAll`
Compra, pago, cancelación, `GET /api/v1/ventas`, `GET /api/v1/entradas/{id}`	`user`
Alta de eventos (`POST /api/eventos`)	`organizer`
Operaciones internas entre servicios	`servicio`
`GET /api/usuarios/yo`	`user`, `organizer`, `staff`

Además de quién puede usar cada operación, se controla de quién es el dato: `ComprasEnCurso` filtra ventas y entradas por el `compradorId` del token y, si son de otra persona, responde 404, igual que si no existieran. Así no se puede averiguar qué ids existen (BOLA). El rol `staff` todavía no tiene operaciones propias, porque Validación solo expone `health`.

Riesgo de este enfoque: un método nuevo sin anotación queda abierto, y no hay un test que recorra los recursos y falle si falta una.

14.4 CORS

`common/rest/CorsFilter.java` agrega las cabeceras CORS solo si el `Origin` del pedido está en `PALCO_CORS_ORIGENES` (separados por coma; por defecto `http://localhost:5173`, el servidor de Vite). Permite las cabeceras `Authorization`, `Idempotency-Key` e `If-None-Match`, y expone `Location` y `ETag`. No se usa el comodín `*`: con credenciales el navegador no lo acepta y, además, abriría la API a cualquier sitio.

El 401 por token inválido. Cuando el token no se puede verificar (firma alterada, mal formado), lo rechaza el mecanismo MP-JWT del servidor antes de llegar a la aplicación, así que ni `CorsFilter` ni `ManejadoresDeErrores` intervienen. Hasta el 16/10 ese 401 salía en `text/html`, sin CORS y sin `WWW-Authenticate`, y el navegador lo informaba como un error de red (se había observado el 14/10). Ahora `backend/common` trae un `META-INF/web-fragment.xml` que registra una página de error para el 401, `ErrorDeAutenticacionServlet`: responde `application/problem+json` con `type` `/problemas/token-invalido`, `WWW-Authenticate: Bearer error="invalid_token"` (RFC 6750) y las cabeceras CORS, que arma el mismo `CorsFilter.cabecerasPara`, así hay una sola lista de orígenes. Como viene en el jar compartido, vale para los 8 servicios sin tocar sus WAR. El 401 sin token y el 403 por rol ya salían bien, porque los arma la aplicación. Estado: implementado y probado. Lo prueban `ErrorDeAutenticacionServletTest` y `CorsFilterTest` y, en vivo el 16/10, `GET /api/v1/ventas` con un token inválido y `Origin: http://localhost:5173` respondió 401, `application/problem+json`, `type` `/problemas/token-invalido`, `WWW-Authenticate: Bearer error="invalid_token"` y `Access-Control-Allow-Origin: http://localhost:5173`. Qué cambia para el frontend, en 15.4, punto 7.

14.5 Formato de errores

Todos los errores de la aplicación salen como Problem Details (RFC 9457, `application/problem+json`) desde `common/rest/ManejadoresDeErrores.java`, que traduce cada excepción de `ErroresDeNegocio` a su código HTTP y a su `type`. El 401 agrega `WWW-Authenticate: Bearer`, y un 403 sin usuario autenticado se convierte en 401. El 429 del límite de intentos de login agrega `Retry-After` (14.1). El 401 por token inválido, que emite el servidor antes de llegar a la aplicación, sale con el mismo formato desde `ErrorDeAutenticacionServlet` (14.4). Cualquier otra excepción termina en un 500 genérico ("Ocurrió un error inesperado."): el detalle va solo al log, así que ninguna respuesta lleva un stack trace. La validación corre antes que la lógica: por ejemplo, `VentasResource` rechaza una clave de idempotencia inválida con 400 antes de reservar cupo.

Id de error en el 500. `ManejadoresDeErrores.Inesperado` genera un id nuevo por cada error inesperado (un UUID al azar, que no dice nada del error), lo escribe en el log en el mismo renglón que la traza (`Error no manejado (idError <id>)`) y lo devuelve como miembro de extensión `idError` del Problem. Quien reporta el error pasa el id y la traza se encuentra en el log sin mandarle al cliente ningún detalle interno. Los demás errores no cambian: `idError` solo se escribe cuando tiene valor. Estado: implementado y probado con `ErrorInesperadoTest` (el id es un UUID, coincide con el del log, el log lleva la traza, dos errores tienen ids distintos y un 409 no trae `idError`); no se provocó un 500 en vivo.

14.6 Protección del log

- **Renglones falsos.** `common/Textos.enUnaLinea` cambia los caracteres de control y los separadores de línea de Unicode por espacios. Se usa en cada línea de log que incluye un dato que vino de afuera (un nombre, un email, un id), para que nadie pueda hacer aparecer un renglón inventado en el log, que se usa como evidencia. Ventas, además, rechaza nombres e ids que traigan esos caracteres.
- **Datos personales.** Los logs no escriben el DNI (criterio SEC-D01, por la Ley 25.326 de protección de datos personales).
- **Excepción a propósito.** El código de verificación del email se escribe en el log porque es la forma simulada de mandarlo. Antes de producción hay que reemplazarlo por un envío real.

14.7 Secretos y variables de entorno

Ningún secreto de producción está en el código: todo sale de variables de entorno, y `.env` está en `.gitignore`.

Variable	Para qué	Si falta
`PALCO_JWT_CLAVE_PRIVADA`	Clave con que Usuarios firma los tokens	Se genera una al desplegar: tras cada reinicio hay que volver a iniciar sesión
`PALCO_SECRETO_SERVICIOS`	Secreto para pedir el token de servicio. En desarrollo se define en `backend/.env`, que `dev.sh` carga	Vale `palco-desarrollo`, que es público (con aviso en el log; el servidor arranca igual)
`PALCO_URL_INTERNA`	Dirección de los otros servicios y del JWKS	`http://localhost:8080`
`PALCO_CORS_ORIGENES`	Orígenes que pueden llamar a la API	`http://localhost:5173`

Variable	Para qué	Si falta
`PALCO_DB_DRIVER`, `PALCO_DB_URL`, `PALCO_DB_USUARIO`, `PALCO_DB_PASSWORD`	Base de datos (Supabase)	H2 en memoria con `sa`/`sa`

14.8 Broker de mensajería

`configurar.cli` crea la cola `cola.emision-entradas` y el tópico `topico.compra-confirmada`, con 5 entregas como máximo, 2 segundos entre reintentos y cola de mensajes fallidos (DLQ). No define usuarios ni permisos por destino: el broker queda con la configuración de fábrica de WildFly y cualquier componente desplegado puede publicar o consumir en cualquier destino. Es tolerable mientras todo corre en un mismo servidor, pero no cumple mínimo privilegio. Hay una propuesta de un usuario por servicio con permisos por destino (a cargo de Julian Zuchowicki), a decidir en grupo: diseñado, pendiente, sin nada aplicado en `configurar.cli`. La otra salida es dejar el riesgo registrado como aceptado.

14.9 Matriz de criterios de seguridad (taller de la clase 11)

En la clase 11 el grupo armó una matriz de diez criterios (`actividad_integradora_clase11/`), cada uno con Dado/Cuando/Entonces, referencia OWASP y veredicto por revisión de código y tests (`mvn test` del 16/10, antes de los cambios de esta sección: 313 tests, 0 fallas). Como no hay app móvil, los dos criterios mobile se reemplazaron por dos de datos sensibles. La columna "Estado actual" sale de la verificación conjunta del 16/10 (09:17 a 09:23), con el servidor armado desde cero con esos cambios:

- `mvn clean install`: BUILD SUCCESS, 333 tests y 0 fallas (common 52, usuarios 22, eventos 8, ventas 199, pagos 22, notificaciones 30); 8 de 8 WAR desplegados, Usuarios en la migración v5, 0 líneas `ERROR` en el log.
- Los 13 escenarios de `scripts/pruebas/escenarios.mjs`: 13 de 13, 177 comprobaciones.
- Newman con la colección completa: 74 requests ejecutados, 252 aserciones, 1 falla (SEC-W05c, ver abajo).

Los requests de cada criterio forman parte ahora de la colección generada en `docs/api/`: carpeta "4 Seguridad (clase 11)", con 15 requests, que lleva la colección a 75 (la corrida completa ejecuta 74, porque saltea la subcarpeta "Pendientes").

ID	Criterio	Referencia	Veredicto (clase 11)	Estado actual (16/10)
SEC-W01	Una entrada ajena no se puede consultar	OWASP API1:2023 (BOLA)	Cumple, test en verde	Cumple, prueba en verde: filtro por `compradorId` y 404; tests `ComprasEnCursoTest.unaVentaAjenaRespondeLoMismoQueUnaQueNoExiste` y `laEntradaSoloLaVeQuienLaCompro`, y requests SEC-W01 y SEC-W01b
SEC-W02	El rol decide quién usa cada operación	OWASP API5:2023 (BFLA)	Cumple en código	Cumple, prueba en verde: requests SEC-W02 (crear evento con rol `user`: 403) y SEC-W02b (sin token: 401 con `WWW-Authenticate`)

ID	Criterio	Referencia	Veredicto (clase 11)	Estado actual (16/10)
SEC-W03	El login no revela si el email existe	A07:2025	Cumple en código	Cumple en código; prueba parcial: SEC-W03a y SEC-W03b en verde (mismo `detail` con un email inexistente y con la clave mal). Las cuentas de prueba pasaron a 600.000 iteraciones como el hash falso (V5). Los tiempos no se midieron, y una cuenta registrada antes del cambio conserva 310.000
SEC-W04	Bloqueo tras 5 logins fallidos en 15 minutos	A07:2025	No cumple	Cumple, prueba en verde: `LimiteDeIntentosDeLoginTest`, request SEC-W04 y en vivo (cinco 401 y el 6.º, 429 con `Retry-After: 898`). Límite conocido: se puede bloquear el login de otra persona conociendo su email
SEC-W05	Tokens alterados, sin firma o de larga vida se rechazan	A07:2025	Parcial	Parcial. Firma alterada y `alg: none` se rechazan con 401 Problem Details, `WWW-Authenticate: Bearer error="invalid_token"` y CORS (`TokensTest`, `ErrorDeAutenticacionServletTest`, SEC-W05a y SEC-W05b en verde, y en vivo). Falta la vida corta: el token dura 8 h sin revocación y SEC-W05c queda en rojo a propósito hasta que el grupo decida la vigencia de la sesión
SEC-W06	Entrada inválida: 400 limpio y sin persistir	A05:2025	Cumple, test en verde	Cumple, prueba en verde (SEC-W06). Además el 500 trae `idError` (`ErrorInesperadoTest`)
SEC-I01	SOAP seguro y con estado controlado ante timeout	A02:2025	No verificable	Sin cambios: Facturación sigue sin contrato SOAP (solo `health`); al implementarla hay que deshabilitar DTD y entidades externas
SEC-I02	Permisos por destino en el broker	A01:2025	No cumple	No cumple: sin usuarios ni permisos por destino. Hay una propuesta, a decidir en grupo
SEC-D01	Los logs no contienen DNI	Ley 25.326	Cumple en código	Sin cambios: cumple en código, sin prueba propia
SEC-D02	Contraseñas con hash y nunca en respuestas	A04:2025	Cumple en código	Cumple, prueba en verde: PBKDF2 con 600.000 iteraciones (`ContrasenasTest`, `MigracionesEnH2Test`) y request SEC-D02 (`/yo` sin contraseña ni hash)

Resumen al 16/10: 5 criterios cumplen con prueba en verde (W01, W02, W04, W06, D02), 2 cumplen en el código con prueba parcial o sin prueba propia (W03, D01), 1 es parcial (W05, por la vida del token), 1 no cumple (I02, con propuesta) y 1 todavía no se puede verificar (I01). La única aserción en rojo de la colección es SEC-W05c, y es deliberada.

15. Frontend

15.1 Tecnología

React 19 con TypeScript 6, Vite 8 como servidor de desarrollo y empaquetador, React Router 7 para las rutas, Tailwind CSS 3 para los estilos y Framer Motion para las transiciones. El lint es `oxlint`. No se usa una librería de datos remotos (TanStack Query u otra): son pocas llamadas y se resuelven con `fetch` y

`useEffect`, que el grupo puede explicar entero. La URL del backend sale de `VITE_API_URL` (por defecto `http://localhost:8080`; ver `frontend/.env.example`). El diseño es responsive; no hay app móvil.

Carpeta de `frontend/src/`	Contenido
`lib/api.ts`	El único archivo que llama a `fetch`: agrega el token y la cabecera `Idempotency-Key`, y convierte los Problem Details en errores con su `type`
`servicios/`	`usuarios.ts`, `eventos.ts` y `ventas.ts`: una función por endpoint. `ventas.ts` es el único lugar que traduce una venta del backend a los tipos del frontend
`store/`	`SesionContext` (sesión) y `AppContext` (catálogo en memoria y datos locales), separados porque son responsabilidades distintas
`pages/`, `components/`	Pantallas y componentes
`data/`	`listings.ts` y `organizer.ts`: datos fijos de reventa y del organizador, para las pantallas que siguen locales. `events.ts` ya no tiene eventos: solo las fotos por categoría y por evento de prueba

15.2 Pantallas y recorridos

Ruta	Pantalla	Datos
`/`	Catálogo con búsqueda y filtro por categoría	Backend (Eventos)
`/evento/:slug`	Detalle del evento y selector de entradas por sector	Backend (Eventos)
`/ingresar`, `/registro`	Ingreso; registro en dos pasos (datos y código)	Backend (Usuarios)
`/checkout` (protegida)	Titulares, reserva con cuenta regresiva y pago	Backend (Ventas)
`/confirmacion/:ordenId` (protegida)	Espera de la emisión y entradas con QR	Backend (Ventas)
`/mis-entradas` (protegida)	Entradas emitidas y compras en curso	Backend (Ventas)
`/organizador` (protegida)	Alta de eventos (backend) y estadísticas (locales)	Mixto
`/reventa` (protegida)	Publicar, retirar y comprar entradas revendidas	Local
`/puerta` (protegida)	Validación de entradas en el acceso	Local

Recorrido del comprador: busca un evento, elige sectores y cantidades, ingresa, carga los titulares, la reserva aparta el cupo por 10 minutos, paga, espera la emisión y ve sus entradas con QR en la confirmación y en "Mis entradas". Recorrido del organizador: ingresa y crea un evento, que aparece en el catálogo.

15.3 Qué usa el backend y qué sigue con datos locales

Función	Estado	Detalle
Sesión (ingreso, registro con código y reenvío, perfil)	Implementado contra Usuarios	El token se confirma con `GET /api/usuarios/yo` al abrir la aplicación
Catálogo, búsqueda y detalle	Implementado contra Eventos	Sin filtros usa la lista ya cargada; con filtros busca el servidor, con 300 ms de demora y descartando respuestas viejas

Función	Estado	Detalle
Alta de eventos	Implementado contra Eventos	Usa la foto de la categoría, porque el backend no tiene subida de imágenes
Compra (reserva, pago, cancelación, emisión)	Implementado contra Ventas	El cobro lo hace la pasarela simulada de Pagos (PagosFake)
Mis entradas	Implementado contra Ventas	Lee `GET /api/v1/ventas`; se ve igual desde otro navegador
Reventa	Local	Estado en `localStorage`; Reventa solo tiene `health`
Puerta	Local	Validación solo tiene `health`
Estadísticas del organizador	Local (datos fijos)	No hay servicio que las calcule

Las entradas emitidas en el backend se copian al estado local para que la reventa y la puerta puedan usarlas; esa copia no pisa un estado de reventa (publicada, vendida) ni de puerta (usada) que el backend no conoce. Se quitaron los usuarios y eventos de mentira que había antes: mantenerlos escondía las fallas del backend.

15.4 Decisiones de experiencia de uso

1. **La reserva se crea al terminar "Titulares".** El backend pide los titulares en el mismo pedido que reserva y no deja cambiarlos después. Por eso la cuenta regresiva aparece en "Pago" y, para corregir un titular, se cancela y se empieza de nuevo.
2. **Cuenta regresiva con la hora del servidor.** Cada segundo se calcula "vencimiento que mandó el backend menos la hora actual", en lugar de restarle uno a un contador, que se atrasa con la pestaña en segundo plano y se reinicia al recargar. Al recibir la reserva se mide el desfase entre el reloj del navegador y el del servidor (el vencimiento es "hora del servidor más 10 minutos") y se corrige; un desfase menor a 5 segundos se toma como cero. Sin esa corrección, con el reloj del navegador 11 minutos adelantado, la página cancelaba la reserva recién hecha. En cero la página cancela la compra para devolver el cupo, pero decide el servidor: si el pago responde `reserva-vencida`, se muestra lo mismo. Límite: el frontend supone que la reserva dura 10 minutos (`MINUTOS_RESERVA`).
3. **Doble clic y reintentos: claves de idempotencia.** Iniciar la compra y pagar viajan con `Idempotency-Key`, y el backend hace la operación una sola vez por clave. Deshabilitar el botón no alcanzaba, porque no cubre un corte de red en el que el pedido llegó pero la respuesta se perdió. La clave de la compra se mantiene mientras el pedido no cambie; la del pago se guarda en el carrito (sobrevive a recargar) y cambia solo después de un rechazo (402), cuando se sabe que no se cobró. Límite conocido: la clave de la compra vive en memoria; si se pierde la respuesta de la reserva y además se recarga antes de reintentar, esa primera reserva queda sin dueño hasta que vence a los 10 minutos.
4. **Pago sin resultado.** Tras un 504 o un corte de red no se sabe si se cobró. La página consulta `GET /api/v1/ventas/{id}`: si sigue `EN_CHECKOUT`, dice que no se sabe si el pago se acreditó y que los lugares siguen reservados, sin cuenta regresiva y sin cancelar; si quedó paga, va a la confirmación; si se canceló o venció, lo dice. Ofrece "Reintentar pago" (mismos datos y misma clave, solo si la venta trae el enlace `pagar`), "Ver estado", "Ir a Mis entradas" y "Volver al catálogo". El aviso se guarda en el carrito y sigue después de recargar.

5. **Espera de la emisión.** Las entradas se emiten en un proceso asincrónico después del pago. La confirmación muestra "Emitiendo tus entradas…" y consulta la venta una vez por segundo, hasta 30 veces; si se agotan, avisa que el pago está aprobado y ofrece volver a consultar. "Mis entradas" repite el listado cada 2 segundos, hasta 15 veces, mientras haya una compra emitiéndose. El backend no ofrece WebSockets.
6. **Errores.** Las páginas deciden por el `type` del Problem Details (`cupo-agotado`, `pago-rechazado`, etc.) y muestran el `detail` que manda el backend, así el texto del error se escribe en un solo lugar. Los sectores del alta de eventos se validan también en el navegador, porque el servidor no dice en qué fila está el error. Límite: `api.ts` solo muestra el `detail` para los estados que tiene en su lista (401, 402, 403, 404, 409, 422, 504 y los 400 con campos). El 429 del límite de intentos de login (14.1) no está, así que la pantalla de ingreso muestra "Ocurrió un error inesperado." en lugar de pedir que se espere, y no usa `Retry-After`. Tampoco se muestra el `idError` de un 500 (14.5). Los dos quedan pendientes en el frontend.
7. **Un corte de red no cierra la sesión.** Hasta el 16/10 el 401 por token inválido llegaba sin CORS, y el navegador no distinguía "token rechazado" de "sin red". Por eso, si falla un pedido con token sin respuesta legible, `api.ts` prueba ese token con `GET /api/usuarios/yo`: si responde, el problema fue la red y la sesión sigue, con la misma clave para el reintento; solo si `/yo` tampoco se puede leer y el `health` de Usuarios responde, se cierra la sesión con un aviso. Cuesta un pedido más, y solo cuando algo falló. Desde el cambio del 16/10 (14.4) el 401 llega con CORS y en Problem Details, así que el navegador lo deja leer y `api.ts` cierra la sesión por la rama común (`respuesta.status === 401` con token), sin pasar por esa prueba; la prueba queda para los cortes de red de verdad. El frontend no se modificó para este cambio: el comentario de `tokenRechazado` en `api.ts` todavía describe el 401 sin CORS, y este recorrido no se volvió a probar en el navegador.
8. **Sesión en `localStorage`.** El token, el usuario y el vencimiento se guardan en `localStorage` para sobrevivir a recargar; una cookie HttpOnly exigía cambiar el backend. Es la opción más expuesta ante un XSS y queda como riesgo aceptado del alcance actual.
9. **Redirección segura después de ingresar.** El parámetro `next` solo acepta rutas internas simples, para que un enlace armado por otro no mande a un sitio externo.
10. **Teléfono.** El selector de entradas se ve en todos los anchos; en pantalla angosta el total y "Continuar" van en una barra fija abajo.

15.5 Cómo se prueba

- **Automático:** `cd frontend && npm run lint && npm run build` (lint con oxlint y compilación de TypeScript). El frontend no tiene tests unitarios propios.
- **A mano contra el backend real** (`backend/dev.sh` y `npm run dev`, en `http://localhost:5173`), con las cuentas sembradas `titular@palco.test` y `organizador@palco.test` (contraseña `palco1234`): búsqueda, registro (el código queda en `backend/servidor/target/server/standalone/log/server.log`), alta de evento, compra de dos entradas, recarga durante el pago, tarjeta terminada en `0002` (rechazo) y en `0504` (pago sin resultado; `servicios/ventas.ts` las traduce a los tokens `tok_rechazado` y `tok_sin_respuesta` de PagosFake), confirmación con QR y "Mis entradas".

- **En navegador automatizado (Playwright):** los casos que no se pueden provocar a mano en una sola computadora (el reloj del navegador adelantado 3 y 11 minutos, y la respuesta de la reserva cortada después de que el backend la procesó) se probaron así: la reserva siguió en pie, la sesión se conservó y hubo una sola venta y un solo pago. Esos guiones no están en el repo.

16. Pruebas y evidencia

El sistema se prueba en cuatro niveles. Cada uno contesta una pregunta distinta y necesita cosas distintas para correr:

Nivel	Qué contesta	Necesita	Dónde está
Tests unitarios (JUnit)	¿Cada componente cumple sus reglas, aislado del resto?	Solo Maven y un JDK 25; no levantan WildFly	`backend/*/src/test`
Escenarios de integración	¿La compra y la mensajería funcionan de punta a punta, entre servicios y con el broker real?	WildFly levantado con los 8 WAR, Node 18+ y `jboss-cli`	`scripts/pruebas/`
Colección de Postman (Newman)	¿La API responde lo que documenta el contrato, request por request?	WildFly levantado, Postman o Newman	`docs/api/`
Demo y guiones de evidencia	¿Se puede mostrar el proceso asincrónico paso a paso, y dejar constancia?	WildFly levantado, Git Bash o Bash, `curl`	`scripts/demo-entrega3.sh`, `docs/entrega3/evidencia/`

El frontend no tiene tests automáticos en el repo: se controla con `npm run lint` (oxlint) y `npm run build`, que incluye el chequeo de tipos (`tsc -b`).

16.1 Tests unitarios por módulo

Se corren con `cd backend && mvn -q test` (o `mvn -f backend\pom.xml test` desde PowerShell). **Última corrida conocida: 16/10/2026, 09:17, `mvn clean install` dentro de la verificación conjunta (sección 16.6): BUILD SUCCESS, 333 tests, 0 fallas.** El conteo por módulo de la tabla sale de contar las anotaciones `@Test` y `@ParameterizedTest` en cada `src/test`, y coincide con el de esa corrida. Incluye los cambios de seguridad del 16/10, commiteados ese día.

Módulo	Tests	Clases de test y qué cubren
`common`	52	`PagoConfirmadoMensajeTest` (16: armado y lectura del mensaje del contrato), `TokenDeServicioTest` (7), `TextosTest` (5), `ErroresDeNegocioTest` (5), `ReglasDeNegocioTest` (4), `ProblemaTest` (4: cuerpo Problem Details), `ErrorInesperadoTest` (3: el 500 con `idError`), `CorsFilterTest` (3), `BaseDeDatosTest` (3), `ErrorDeAutenticacionServletTest` (2: el 401 por token inválido)
Usuarios	22	`LimiteDeIntentosDeLoginTest` (8: el 429 después de 5 fallidos), `ContrasenasTest` (5), `TokenDeServicioEmitidoTest` (3), `TokensTest` (2), `MigracionesEnH2Test` (2), `RegistroCompletoTest` (1), `CuentasDePruebaTest` (1)
Eventos	8	`ServicioDeEventosTest` (8)

Módulo	Tests	Clases de test y qué cubren
Ventas	199	`ComprasEnCursoTest` (57), `ServicioDeVentasTest` (46), `ClienteDePagosTest` (20), `ClienteDeEventosTest` (15), `EmisionDeEntradasMDBTest` (11), `VencimientoDeReservasTest` (10), `EnlacesTest` (8), `VentasResourceTest` (8), `ClienteDeUsuariosTest` (6), `VentaTest` (6), `CobrosFakeTest` (4), `FabricaDeCobrosTest` (4), `EntradasResourceTest` (3), `EmisionEnH2Test` (1)
Pagos	22	`ServicioDePagosTest` (14), `PagosFakeTest` (5), `FabricaDePasarelaTest` (3); el repositorio se reemplaza por `PagoRepositoryEnMemoria`
Notificaciones	30	`ServicioDeAuditoriaTest` (6), `AuditoriaDeComprasMDBTest` (5), `PruebasDeMensajeriaTest` (5), `AvisoDeCompraMDBTest` (4), `LogEnUnSoloRenglonTest` (4), `ServicioDeNotificacionesTest` (4), `MigracionesEnH2Test` (2)
Validación, Reventa, Facturación	0	Son esqueletos: solo responden su `health`
Total	**333**	

Ventas concentra casi dos tercios de los tests porque es el componente con más reglas y estados: la compra en dos llamadas, la reserva de cupo con vencimiento, la idempotencia, el cobro a Pagos y la emisión asincrónica. Las clases `EmisionEnH2Test` y `MigracionesEnH2Test` corren las migraciones Flyway contra H2 en modo PostgreSQL.

Fuera de JUnit hay una prueba en vivo, `backend/servicios/ventas/src/test/en-vivo/pagos-simultaneos.mjs`, que sí necesita el servidor levantado: paga varias compras en el mismo instante.

Los cambios de seguridad del 16/10 tienen sus tests: el límite de intentos de login (`LimiteDeIntentosDeLoginTest`), el 401 por token inválido como Problem Details (`ErrorDeAutenticacionServletTest`), el `idError` del 500 (`ErrorInesperadoTest`) y las 600.000 iteraciones de PBKDF2 (`ContrasenasTest`). `MigracionesEnH2Test` de Usuarios corre sus migraciones, V4 y V5 incluidas, contra H2.

16.2 Los 13 escenarios de integración

`scripts/pruebas/escenarios.mjs` (con sus ayudas en `lib.mjs`) prueba la compra y la mensajería de punta a punta, por la API REST y por `jboss-cli`, contra el WildFly real. Usa el `fetch` de Node 18 o más nuevo y ningún paquete de npm. Cada comprobación imprime qué esperaba y qué vio; al final hay un resumen por escenario. Sale con código 0 si todo pasa y con 1 si algo falla.

```
node scripts/pruebas/escenarios.mjs                      # los 13 escenarios
node scripts/pruebas/escenarios.mjs 1 2 3 4 5 6 7 9 13   # solo los que no detienen consumidores
```

N.º	Escenario	Qué comprueba
1	Compra completa	Iniciar (201, reserva de cupo, vence a los 10 minutos), pagar contra Pagos, emisión asincrónica hasta `EMITIDA` con entradas `VALIDA`, el pago guardado en Pagos y el listado de compras propias
2	Idempotencia	La misma `Idempotency-Key` al iniciar y al pagar devuelve la misma venta y el mismo pago: un cupo, un cobro, un mensaje
3	Pago rechazado	402 en Problem Details, la venta sigue `EN_CHECKOUT` y no sale ningún mensaje; un segundo intento con clave nueva la completa

N.º	Escenario	Qué comprueba
4	Reglas de negocio	Máximo de entradas, DNI repetido, titulares que no coinciden, sector inexistente, cupo agotado, datos inválidos (400) y una línea falsa inyectada en el log
5	Seguridad	401 sin token o con firma alterada, 404 con el token de otro usuario, 403 con un rol que no corresponde, operaciones internas solo con rol `servicio`
6	Cancelación	Cancelar libera el cupo una sola vez y la venta ya no se puede pagar
7	Un mensaje por pago	Un mensaje en `cola.emision-entradas` y una copia por suscripción en `topico.compra-confirmada`, con el selector `tipoEvento = 'PAGO_CONFIRMADO'` y el cuerpo del contrato
8	Mensaje repetido	Se pasa una copia ya procesada a la cola de emisión con `move-messages`: el consumidor la reconoce y no cambia nada
9	Los dos suscriptores	El aviso (línea en el log) y la auditoría (fila en la base) reciben cada uno su copia
10	Consumidor detenido	Con `stop-delivery` el mensaje espera en la cola y la venta queda `CONFIRMADA`; con `start-delivery` se emite
11	Suscripción durable	Con la auditoría detenida se publican dos mensajes; al reanudarla los recibe
12	Mensaje inválido	Se reintenta 5 veces, cada 2 segundos, y termina en la DLQ sin trabar las compras
13	Transacción deshecha	Si la transacción se deshace después de publicar, el mensaje no sale; sin la falla, sí

Última corrida: 16/10/2026, en la verificación conjunta de 09:17 a 09:23, con el servidor armado desde cero con los cambios de seguridad del 16/10: 13 de 13 escenarios, 177 comprobaciones, todas bien. La salida guardada en `docs/entrega3/evidencia/integracion-13-escenarios.txt` es la de la corrida anterior (14/10, de 17:58 a 18:00, con el servidor armado desde el tag `entrega-parcial-2`, mismo resultado), que es la que acompaña a la Entrega Parcial N.º 2. El secreto de servicios lo toman de `PALCO_SECRETO_SERVICIOS` o de `backend/.env` (sección 17.4).

Decisiones de diseño de la prueba, porque condicionan cómo leer el resultado:

- **Crea su propio evento** (como organizador), con un sector igual a Platea B y otro de 3 lugares. Así el cupo se compara exacto aunque otra persona compre en otros eventos.
- **Los contadores del broker son de todo el servidor.** Si alguien compra mientras corre, la prueba busca esas ventas en el `server.log` y las descuenta.
- **Los escenarios 8, 10, 11 y 12 detienen un consumidor o mueven mensajes entre colas**; si hay otra persona probando contra el mismo servidor se corren solo los otros nueve. Cada uno vuelve a arrancar el consumidor que detuvo, aunque falle.
- **Deja datos**: ventas emitidas y canceladas de `titular@palco.test`, una cuenta nueva por corrida, el evento de prueba y el mensaje inválido del escenario 12 en la DLQ. Con H2 en memoria eso se borra al reiniciar.

16.3 Colección de Postman y Newman

`docs/api/palco-api.postman_collection.json` tiene 75 requests. Las carpetas ejecutables son cuatro: "1 Usuarios (implementado)", "2 Eventos (implementado)", "3 Diseño v1 (Ventas implementado)" y "4 Seguridad (clase 11)", con 15 requests que prueban criterios de la matriz de seguridad. Además hay subcarpetas de "Pasos manuales" y "Pendientes" que no se ejecutan. No se edita

a mano: la genera `scripts/postman/generar.js` (solo Node). No usa ids fijos: los `V-…`, `T-…` y `P-…` se toman de las respuestas, así que se corre entera, en orden y sin environment. El único valor que puede hacer falta pasar es el secreto de servicios, en la variable de colección `secretoServicios` (sección 17.4). La guía es `docs/api/GUIA_POSTMAN.md`.

```
npx newman run docs/api/palco-api.postman_collection.json
npx newman run docs/api/palco-api.postman_collection.json --folder "3 Diseño v1 (Ventas
implementado)"
npx newman run docs/api/palco-api.postman_collection.json --env-var secretoServicios=<el de
backend/.env>
```

Corrida	Resultado
14/10, Newman 6.2.3, con un servidor desplegado antes de que existiera la cuenta `otro.titular@palco.test` (registrada en `GUIA_POSTMAN.md`)	53 requests, 201 tests, 9 tests fallados y un script cortado, todos en los tres requests que usan esa cuenta (su login respondía 401). El resto, 0 fallados
14/10, alrededor de las 16:55, con el servidor redesplegado	54 requests, 206 aserciones, 0 fallas. Es el resultado que `GUIA_POSTMAN.md` daba como esperado; la corrida quedó anotada en el estado del proyecto, pero su salida no está guardada en el repo
16/10, verificación conjunta (09:17 a 09:23), servidor armado desde cero con los cambios de seguridad, colección completa	74 requests ejecutados, 252 aserciones, 1 falla: SEC-W05c ("el token dura 15 minutos o menos"), porque la sesión dura 8 horas. Queda en rojo a propósito hasta que el grupo decida la vigencia del token. Su salida no está guardada en el repo

Pendiente: repetir la corrida en la ventana de Postman (Runner) para sacar la captura que va en el documento.

16.4 La demo

`scripts/demo-entrega3.sh` hace una compra de punta a punta con `curl` y muestra, paso por paso, el proceso de negocio con su parte asincrónica. Cada paso comprueba lo que muestra; si algo no da lo esperado corta, dice qué esperaba y qué llegó y sale con código 1. Cada corrida usa claves de idempotencia nuevas, así que se puede repetir.

Modo	Pasos	Qué se ve
`bash scripts/demo-entrega3.sh`	12	Login, contadores del broker, cupo, inicio de la compra (`EN_CHECKOUT`, cupo reservado), pago, `CONFIRMADA` con entradas `PENDIENTE_EMISION`, espera, `EMITIDA` con entradas `VALIDA`, una entrada con su código, cupo después, la fila de auditoría y los contadores otra vez (cola +1, tópico +2)
`bash scripts/demo-entrega3.sh rechazado`	9	El pago responde 402 `pago-rechazado`, la venta sigue `EN_CHECKOUT` sin pago ni entradas, no hay fila de auditoría ni suben los contadores; al final cancela y comprueba que el cupo volvió

Última corrida: 16/10/2026, en la verificación conjunta, código de salida 0 en los dos modos. Se configura con variables de entorno (`PALCO_URL`, `EVENTO_SLUG`, `SECTOR_ID`, `COMPRADOR_EMAIL`, `COMPRADOR_CLAVE`, `SECRETO_SERVICIO`, `ESPERA_MAXIMA`, `DETALLE=1`, `JBOSS_CLI`, `NO_COLOR=1`), documentadas en `scripts/LEEME.md`. Si `SECRETO_SERVICIO` no está definida, la demo usa `PALCO_SECRETO_SERVICIOS` o la línea de `backend/.env`.

Qué no muestra: el cobro es el simulado (`PagosFake`: `tok_test` aprueba y `tok_rechazado` rechaza). La factura SOAP, Mercado Pago, la reventa y la validación en puerta están diseñadas y pendientes, y la demo no las toca. Para el 13/11 falta sumarle el tramo de la factura.

16.5 Evidencia del proceso asincrónico

`docs/entrega3/evidencia/` es la evidencia que acompaña a la Entrega Parcial N.º 2. Las tomas principales son del **14/10/2026, de 17:56 a 18:00**, con el servidor armado desde cero desde el commit `580dd05` (el del tag `entrega-parcial-2`). Cada archivo dice arriba qué muestra y, por paso, la fecha, el comando exacto y su salida sin retocar. Los tokens no se guardan: se guarda el comando con que se pidieron. `LEEME.txt` explica cada archivo.

La compra de la toma (venta `V-25d98b55`, pago `P-dbe21ef4`), escrita por `tomar-evidencia.sh`, que se puede repetir:

Archivo	Qué muestra
`broker-antes.txt`	Cola, tópico y DLQ antes de la compra (todo en 0: el servidor recién arrancaba)
`compra.txt`	Abrir la compra (201, `EN_CHECKOUT`) y pagarla (201, `CONFIRMADA`, entradas `PENDIENTE_EMISION`)
`venta.txt`	La venta segundos después: `EMITIDA`, con sus entradas `VALIDA` y sus códigos
`entrada.txt`	Una entrada: `VALIDA`, con `codigoQr` y fecha de emisión
`broker-despues.txt`	Cola +1 sin pendientes, tópico +2 (una copia por suscriptor), DLQ sin cambios
`server-log-extracto.txt`	Las líneas del log con el id de la venta: compra, pago guardado por `ServicioDePagos` y los tres consumidores
`auditoria.txt`	La fila que guardó la auditoría por ese mensaje

Guiones que repiten el proceso:

Archivo	Qué muestra
`reproducir.sh` / `reproduccion.txt`	Configuración del broker (5 entregas, 2000 ms entre una y otra, las de `configurar.cli`), una compra completa con la línea de `FabricaDeCobros` que dice quién cobra, la transacción deshecha (409 y contadores sin cambios), control por rol (401, 403, 200) y la reserva de cupo (+1 con la compra abierta, vuelve al cancelar)
`suscriptor-durable.sh` / `suscriptor-durable.txt`	Con la auditoría detenida (`stop-delivery`) los 2 mensajes esperan en la suscripción durable mientras el aviso y la emisión siguen; al reanudarla aparecen las 2 filas
`integracion-13-escenarios.txt`	La salida de los 13 escenarios (sección 16.2)
`compras-simultaneas.mjs` / `compras-simultaneas.txt`	Del 13/10, con código anterior al tag. 80 compras a la vez, 20 pedidos con la misma clave de idempotencia y 30 pagos a la vez: todos 201. Incluye el antes (28 abiertas y 52 sin respuesta con una versión anterior de Ventas) y muestra que Ventas no retiene una conexión de la base mientras espera a Eventos o a Pagos

Tomas de la mañana del 13/10 (servidor anterior, con código anterior al tag; su log ya no existe):

Archivo	Qué muestra	Límite
`venta-inexistente.txt`	El mensaje de una venta que no existe se descarta con un aviso, sin volver a la cola ni ir a la DLQ	
`mensajes-pendientes.txt`	64 mensajes esperando antes de que existiera el consumidor; al desplegarlo quedaron 0	Un paso está resumido
`mensaje-repetido-y-reintentos.txt`	Un mensaje repetido no emite dos veces; un mensaje que no se entiende va a reintentos y DLQ	Salida de un guion que no está en el repo; no se puede repetir solo con el repo. Ventas cobraba con su simulador local
`vencimiento-de-reserva.txt`	Una compra sin pagar pasa a `VENCIDA` a los 10 minutos, el cupo vuelve de 521 a 520 y pagarla después da 409	Es un resumen sin comandos; no se repitió porque exige esperar 10 minutos

Límites de la evidencia: no se mueve plata (pasarela simulada, `PagosFake`, sin llamadas a Mercado Pago); la base es H2 en memoria y las ventas se pierden al reiniciar (el broker sí guarda sus mensajes); los contadores `messages-added` son acumulados desde el arranque, así que vale la diferencia entre antes y después; y **no se probó una caída del servidor entre el commit de la base y el del broker**.

16.6 Resumen de la última corrida conocida

La última verificación conjunta fue el **16/10/2026, de 09:17 a 09:23**, con el servidor armado desde cero con los cambios de seguridad del 16/10.

Qué	Comando	Última corrida	Resultado
Tests del backend	`mvn -f backend/pom.xml clean install`	16/10, 09:17	BUILD SUCCESS, 333 tests, 0 fallas
Lint del frontend	`npm run lint --prefix frontend`	16/10	código 0, 8 avisos
Build del frontend	`npm run build --prefix frontend`	16/10	código 0
Armado desde cero	`mvn -f backend/pom.xml clean install` y `backend/dev.sh`	16/10, verificación conjunta	8 de 8 WAR desplegados, Usuarios en la migración v5, 0 líneas `ERROR` en el log y sin el aviso de secreto de desarrollo (arrancó con el de `backend/.env`)
13 escenarios	`node scripts/pruebas/escenarios.mjs`	16/10, verificación conjunta	13 de 13, 177 comprobaciones
Postman	`npx newman run docs/api/palco-api.postman_collection.json`	16/10, verificación conjunta	74 requests, 252 aserciones, 1 falla esperada (SEC-W05c, vigencia del token, a decidir)
Demo	`bash scripts/demo-entrega3.sh [rechazado]`	16/10, verificación conjunta	código 0 en los dos modos
Cuenta nueva de punta a punta	Registro, verificación, login y compra	16/10, verificación conjunta	La compra llegó a `EMITIDA`

Qué	Comando	Última corrida	Resultado
401 por token inválido, en vivo	`GET /api/v1/ventas` con un token inválido y `Origin: http://localhost:5173`	16/10, verificación conjunta	401, `application/problem+json`, `type` / `problemas/token-invalido`, `WWW-Authenticate: Bearer error="invalid_token"`, `Access-Control-Allow-Origin: http://localhost:5173`
Límite de intentos, en vivo	Seis logins fallidos con un mismo email	16/10, verificación conjunta	Los 5 primeros, 401; el 6.º, 429 con `Retry-After: 898`

Estado: **Implementado** para todo lo de la tabla, salvo la vigencia del token, que es la falla esperada de Newman (diseñado, pendiente de decisión). De los tests del backend, de Newman y de la verificación conjunta no hay salida guardada en el repo: el resultado está anotado, no archivado. La evidencia archivada en `docs/entrega3/evidencia/` sigue siendo la del 14/10. Pendiente: tests de Eventos más allá del negocio, tests de Validación, Reventa y Facturación cuando existan, una prueba de la falla a mitad del flujo transaccional que se pueda repetir desde el repo y tests automáticos del frontend.

17. Cómo levantar y probar el sistema

17.1 Requisitos

Para	Qué hace falta	Nota
Backend	JDK 25 y Maven 3.9+	WildFly 41 no se instala: lo descarga Maven la primera vez. En Windows, cualquier JDK 25 (por ejemplo Temurin) con `JAVA_HOME` apuntando a él
Frontend	Node.js `^20.19.0` o `>=22.12.0` (lo exige Vite 8) y npm	
Escenarios y Postman	Node 18+ (los escenarios no usan paquetes); Newman se instala aparte o se usa con `npx`	
Guiones	`bash`, `curl`, `grep`, `sed` y `od`	En Windows, Git Bash. La demo no usa `jq`

17.2 Backend

```
cd backend
cp .env.example .env   # solo la primera vez
./dev.sh
```

`dev.sh` carga el `.env`, corre `mvn -q -B install` (compila los 8 WAR y arma WildFly en `backend/servidor/target/server`, aplicando `configurar.cli`) y levanta el servidor en `http://localhost:8080`. La primera vez tarda unos minutos por la descarga. Para recompilar y redesplegar un solo servicio sin reiniciar: `./redesplegar.sh <servicio>`.

Comprobación:

```
curl localhost:8080/api/usuarios/health
curl localhost:8080/api/eventos/health
curl localhost:8080/api/notificaciones/health
curl localhost:8080/api/v1/health          # Ventas
curl localhost:8080/api/v1/pagos/health    # Pagos
curl localhost:8080/api/validacion/health
curl localhost:8080/api/reventa/health
curl localhost:8080/api/facturacion/health
```

En Windows, desde Git Bash:

```
export JAVA_HOME="/c/Users/<usuario>/.jdks/temurin-25"   # o donde esté el JDK 25
cd backend && ./dev.sh
```

Los tests también corren desde PowerShell: `mvn -f backend\pom.xml test`.

En Linux y macOS, los mismos comandos en una terminal Bash (en macOS: `brew install openjdk@25 maven`).

17.3 Frontend

```
cd frontend
npm install
npm run dev      # http://localhost:5173
npm run lint
npm run build
```

Hay que abrir la app en `http://localhost:5173` tal cual: el backend solo acepta los orígenes de `PALCO_CORS_ORIGENES`, así que con `127.0.0.1:5173` o con `npm run preview` (puerto 4173) las llamadas fallan. Sin backend, el catálogo muestra un error con el botón "Reintentar".

17.4 Variables de entorno

Ninguna es obligatoria en desarrollo: sin `.env` el sistema arranca con H2 en memoria y valores de desarrollo.

Variable	Dónde	Para qué	Por defecto
`PALCO_CORS_ORIGENES`	`backend/.env`	Orígenes que pueden llamar a la API	`http://localhost:5173`
`PALCO_DB_DRIVER`, `PALCO_DB_URL`, `PALCO_DB_USUARIO`, `PALCO_DB_PASSWORD`	`backend/.env`	Base PostgreSQL (Supabase)	Sin definir: H2 en memoria, que se borra al reiniciar
`PALCO_JWT_CLAVE_PRIVADA`	`backend/.env`	Clave RSA con que Usuarios firma los tokens	Se genera una nueva en cada arranque (hay que volver a iniciar sesión)
`PALCO_URL_INTERNA`	`backend/.env`	Dónde encuentran los servicios a Usuarios para validar tokens	`http://localhost:8080`
`PALCO_SECRETO_SERVICIOS`	`backend/.env`	Secreto compartido para pedir el token de rol `servicio`. Conviene definir uno propio: el servidor de desarrollo del grupo ya arranca con el suyo	`palco-desarrollo`, público; el log lo avisa
`PALCO_PAGOS_PROVEEDOR`	`backend/.env`	Pasarela de Pagos	`fake` (único valor posible hoy)

Variable	Dónde	Para qué	Por defecto
`PALCO_VENTAS_COBROS`	`backend/.env`	Quién cobra en Ventas: `pagos` (REST a Pagos) o `fake` (simulador local, sin pago guardado)	`pagos`
`VITE_API_URL`	`frontend/.env`	URL del backend	`http://localhost:8080`

El `.env` no se sube al repo. Supabase está **diseñado, pendiente**: las migraciones están escritas para correr en las dos bases, pero nunca corrieron contra un PostgreSQL real.

El secreto de servicios fuera del servidor. Si `backend/.env` define `PALCO_SECRETO_SERVICIOS`, quien pida un token de rol `servicio` desde afuera tiene que usar ese mismo valor:

- La demo, los 13 escenarios y los guiones de evidencia lo leen solos: primero de la variable de entorno `PALCO_SECRETO_SERVICIOS` y, si no está, de la línea de `backend/.env`; si no hay ninguna de las dos, usan `palco-desarrollo`.
- La colección de Postman lo toma de la variable de colección `secretoServicios` (por defecto `palco-desarrollo`). Con Newman se pasa con `--env-var secretoServicios=<valor>`; en Postman se cambia en las variables de la colección.

Que el servidor no arranque sin secreto fuera de desarrollo está propuesto y **no está implementado**.

17.5 Cuentas de prueba

Las cargan las migraciones de Usuarios. Todas usan la contraseña `palco1234`.

Email	Rol	Acceso
`titular@palco.test`	`user`	Compra, mis entradas, reventa
`organizador@palco.test`	`organizer`	Lo anterior y el panel `/organizador`
`puerta@palco.test`	`staff`	Lo anterior y `/puerta`

La colección de Postman usa además `otro.titular@palco.test`, una segunda cuenta compradora para probar que nadie ve compras ajenas. Las cuentas nuevas se crean desde `/registro` con rol `user`; el código de verificación de 6 dígitos se escribe en el log del servidor, porque el envío de emails no existe (**Simulado**).

Para el cobro simulado: `tok_test` (o cualquier otro token) aprueba, `tok_rechazado` rechaza y `tok_sin_respuesta` no contesta.

Desde el 16/10, 5 logins fallidos en 15 minutos con un mismo email bloquean ese email: el pedido siguiente responde 429 con `Retry-After`, aunque la contraseña sea la correcta. Si una cuenta de prueba queda bloqueada, se espera lo que dice `Retry-After` o se reinicia el servidor (con H2 en memoria, la tabla `intento_login` se vacía).

17.6 Recorrido de prueba completo

```
cd backend && mvn -q test                       # 1. tests unitarios
cd backend && ./dev.sh                          # 2. servidor (otra terminal)
curl localhost:8080/api/v1/health               # 3. health de cada servicio
node scripts/pruebas/escenarios.mjs             # 4. 13 escenarios
npx newman run docs/api/palco-api.postman_collection.json --env-var secretoServicios=<el de
backend/.env>   # 5. contrato de la API
bash scripts/demo-entrega3.sh                   # 6. demo, pago aprobado
bash scripts/demo-entrega3.sh rechazado         # 7. demo, pago rechazado
bash docs/entrega3/evidencia/reproducir.sh      # 8. evidencia repetible
cd frontend && npm run lint && npm run build    # 9. frontend
```

En Windows, los pasos 2 y 6 a 8 van en Git Bash, con `JAVA_HOME` en un JDK 25. Si la terminal es PowerShell, para la demo primero `chcp 65001` (la salida es UTF-8). Los escenarios 8, 10, 11 y 12 detienen consumidores: no conviene correrlos si otra persona está probando contra el mismo servidor.

18. Gestión del proyecto

18.1 Integrantes y responsabilidades

El grupo es de cinco integrantes (la consigna fija de 3 a 4; la confirmación de la cátedra está entre las preguntas pendientes). El criterio del reparto es que cada integrante tenga al menos un componente del backend propio que defender.

Integrante	Hecho (09/10 al 13/10)	14/10 al 05/11	13/11 al 26/11	Parte que defiende
Sebastian Deya	Ventas core; errores y CORS	Transacciones; patrones y tests	Lo que Reventa y Mercado Pago necesiten de Ventas	Ventas: el componente stateful, la transacción y la emisión
Julian Zuchowicki	Mensajería; integración Ventas–Pagos; token de servicio	Cola de reintento y revisión de la facturación por SOAP; guion de demo; plan y arranque de Reventa	Reventa	Mensajería y Notificaciones; el cobro de Ventas a Pagos; Reventa
Valentina Frisoli	Pagos; frontend (sesión, catálogo y compra)	Pantalla de puerta; diagrama de secuencia de Mercado Pago (diseño)	Reventa y panel del organizador en el frontend	Pagos; el frontend
Joaquin Molina	Documento de la Entrega Parcial N.º 2 (con Santiago)	Validación QR; documento técnico	Documentación final	Validación; el documento técnico
Santiago Loto	Puesta a punto del 09/10; cambios compartidos de `backend/common` y del servidor; coordinación y revisión	Facturación SOAP; OpenAPI; revisión	Revisión; armado desde cero y tag de cada entrega	Facturación y el servicio SOAP

- La revisión de cada tramo de trabajo la hace alguien distinto de quien escribió el código. La facturación por SOAP la revisa Julian, porque es de Santiago.
- Usuarios y Eventos se construyeron antes del reparto y todavía no tienen dueño asignado para la defensa.
- Del 06/11 al 12/11 nadie tiene tarea nueva: se cierra, se revisa y se ensaya.

18.2 Calendario de la comisión del viernes

El grupo cursa los viernes de 08:45 a 11:45. La consigna está escrita para la comisión del lunes, así que sus fechas no son las del grupo. **Cada entrega se sube el jueves anterior a la clase**, porque la hora límite (13:30, según la consigna) es de la otra comisión y no está confirmada.

Viernes	Hito	Cómo lo llama la consigna	Se congela	Se sube
04/09 y 18/09	1.er checkpoint y 1.ª entrega parcial (ya pasaron)	Entrega Parcial N.º 1 y Entrega Obligatoria N.º 1		
23/10	Checkpoint sin defensa. **Fecha sin confirmar**	Entrega Parcial N.º 2	Tag `entrega-parcial-2` (14/10)	Jue 22/10
06/11	Entrega del diseño del TP Obligatorio. Contenido a confirmar	No figura	Mié 04/11	Jue 05/11
13/11	2.ª entrega parcial, con defensa oral	Entrega Obligatoria N.º 2	Lun 09/11	Jue 12/11
20/11	3.er checkpoint. Contenido a confirmar	No figura	Mié 18/11	Jue 19/11
27/11	Prueba del TP Obligatorio: límite para el sistema completo	No figura con ese nombre	Mar 24/11	Jue 26/11
04/12 y 11/12	Final adelantado o final regular	Entrega Final (la consigna da fechas contradictorias)		

El 16/10, el 23/10 y el 30/10 hay clase. Las preguntas para la cátedra (fecha del checkpoint, contenido del 06/11 y del 20/11, fecha de la Entrega Final, validez del SOAP simulado y de la pasarela simulada, entre otras) todavía no tienen respuesta registrada.

Líneas de corte decididas de antemano, para no discutirlas sobre la fecha: si el 23/10 el WSDL de Facturación no responde, la facturación por SOAP se reduce al mínimo de la consigna; si los patrones y tests de Ventas no cierran el 05/11, pasan al tramo del 13/11 al 26/11; Reventa y Mercado Pago entran el 13/11 solo si están cerradas el 08/11; si Mercado Pago no funciona el 19/11 queda como "diseñado, pendiente"; el panel del organizador es lo último y, si no llega el 24/11, sigue con datos locales.

18.3 Organización por etapas y tareas

El trabajo se ordena en etapas con fecha. Una etapa agrupa tareas que pueden avanzar en paralelo porque tocan módulos distintos; una tarea no arranca si su dependencia no está mergeada. Cada tarea tiene un nombre corto que coincide con su rama.

Fechas	Tareas	Estado
09/10	Puesta a punto: convenciones, entorno, decisión del proceso asincrónico (`PAGO_CONFIRMADO`) y de la base (H2)	Hecha, salvo la identidad de git de cada integrante y el `.mailmap`
09/10	Cambios compartidos: errores Problem Details, CORS, token de servicio, entre otros	Hechos
10/10 al 13/10	Ventas core, mensajería, Pagos, documento de la Entrega Parcial N.º 2	Hechas

Fechas	Tareas	Estado
13/10	Integración Ventas–Pagos, frontend	Hechas. La validación QR y los patrones y tests de Ventas pasaron a la etapa siguiente
14/10 al 05/11	Facturación SOAP, documento técnico, transacciones, validación QR; después patrones y tests de Ventas, OpenAPI, pantalla de puerta; sin bloquear, Reventa y Mercado Pago	En curso. La facturación por SOAP todavía no tiene código
06/11 al 12/11	Congelar, revisar contra la consigna, armado desde cero, demo, documento, ensayo con preguntas cruzadas	Pendiente
13/11 al 26/11	Reventa, Mercado Pago y webhook, resto del frontend, documentación final	Pendiente

El orden de lo que falta sigue un criterio explícito: primero lo que la consigna exige para el 13/11 y hoy no existe (SOAP con WSDL y documento técnico con diagramas de secuencia), después lo que suma componentes implementados (Facturación y Validación) y por último el resto.

Cada tarea sigue el mismo ciclo:

1. **Plan antes de código.** Se escribe el plan (qué archivos, qué decisiones) y se aprueba antes de implementar.
2. **Archivos disjuntos y un dueño por módulo.** Si una tarea necesita cambiar el módulo de otra persona, se acuerda antes. `backend/common` y `backend/servidor` se tocan en un paso aparte, con un solo dueño, y terminan con los 8 servicios redesplegados y sus 8 `health` comprobados. El contrato del mensaje `PAGO_CONFIRMADO` (`docs/api/mensajeria-pago-confirmado.md`) no se cambia sin avisar al grupo.
3. **Verificar antes de dar por hecho.** Toda tarea termina con `mvn -q test` y el `curl` de su servicio, con el resultado real anotado.
4. **Aviso al grupo.** Al terminar una tarea se avisa qué cambió, qué archivos se tocaron, cómo se probó (comando y resultado real) y qué quedó pendiente o dudoso.

18.4 Flujo de trabajo con git

Regla	Cómo se aplica
Una tarea, una rama, un dueño	Rama `feat/<servicio>-<tema>` (por ejemplo `feat/ventas-core`), de preferencia en su propio `git worktree` para avanzar en paralelo
Commits chicos, en español	Formato `tipo: qué cambió` (`feat:`, `fix:`, `docs:`, `test:`), uno por paso verificable
Identidad propia	Cada integrante commitea con su `user.name` y `user.email`; nadie commitea con la identidad de otra persona
Sin reescribir historia	No se hace `push --force`; los commits viejos con identidades mal configuradas se atribuyen con un `.mailmap`
Sin secretos	No se commitean `.env`, certificados ni tokens
Cada entrega sale de un tag	Se congela en la fecha del calendario; desde ahí solo entran arreglos

Tag `entrega-parcial-2`: apunta al commit `580dd05` (14/10/2026, 17:50). La evidencia de la Entrega Parcial N.º 2 se tomó con el servidor armado desde cero desde ese commit, y se agregó en el commit siguiente (`ea5422b`). Para el 13/11 está previsto el tag `entrega-obligatoria-2`.

Estado del historial, sin maquillar. La consigna pide un historial incremental y participación visible de todos. Hoy el repositorio tiene una sola rama (`master`) y 30 commits: 14 con la identidad `santttii`, 6 con `vfrisoli`, 5 con `Improvement` (una identidad mal configurada que hay que atribuir a su dueño real), 3 de Sebastian Deya y 2 de Julián Zuchowicki. No hay commits a nombre de Joaquin Molina. Lo construido entre el 09/10 y el 14/10 entró en pocos commits grandes de una sola identidad, y el `.mailmap` todavía no existe. El grupo lo registra como el primer riesgo no técnico y fija que, desde el 14/10, cada dueño o dueña commitea su tarea en su máquina, con su identidad y en pasos chicos.

18.5 Tablero de tareas en Notion

Las tareas se siguen en un tablero de Notion ("Tablero de Tareas D Apps 2"): una tarjeta por tarea, que se corresponde con una rama. Los diez criterios de seguridad de la matriz de la clase 11 están cargados como tarjetas con la etiqueta "seguridad". El tablero no está en el repositorio, y el 09/10 estaba atrasado respecto del código (Eventos y Usuarios figuraban "Sin empezar"); su puesta al día quedó pendiente desde la puesta a punto. La fuente de verdad del estado es `TODO.md` (por fases).

Las dos decisiones de seguridad que siguen abiertas son la vigencia de la sesión (SEC-W05, Joaquin Molina) y los permisos por destino en el broker (SEC-I02, Julian Zuchowicki).

19. Evolución del diseño

Entre lo que se presentó (la Entrega Parcial N.º 1, los talleres de las clases 6, 8 y 9 y la actividad de la clase 10) y lo que hoy está construido hay diferencias. En todas vale el código, y cada una tiene su motivo. Esta sección parte de la sección 5 del documento de la Entrega Parcial N.º 2 (`docs/entrega3/Entrega_Parcial_2_Integracion.md`) y la amplía con lo que cambió después y con diferencias que ese documento no listaba.

19.1 Componentes e interfaces

Tema	Qué se presentó	Qué vale hoy	Motivo
`IUsuarios`	Entrega Parcial N.º 1: `registrar`, `login`, `obtenerPerfil`, `asignarRol`	`registrar`, `verificarEmail`, `reenviarCodigo`, `iniciarSesion`, `obtener` y `emitirTokenDeServicio`. `asignarRol` no existe	Hizo falta verificar el email antes de habilitar la cuenta y emitir un token para las llamadas entre servicios (rol `servicio`). Ningún flujo implementado cambia roles: las cuentas de organizador y de staff están sembradas
`IVentas`	Cuatro operaciones: `iniciarCompra`, `agregarEntrada`, `confirmarCompra`, `cancelarCompra`	Tres: `iniciarCompra`, `confirmarCompra`, `cancelarCompra`. Las entradas y sus titulares van en el pedido inicial	El cupo se reserva de una sola vez en Eventos, al iniciar. Agregar entradas de a una obligaba a reservar en varios pasos y a deshacer reservas parciales

Tema	Qué se presentó	Qué vale hoy	Motivo
Cómo se ata el componente stateful al usuario	Entrega Parcial N.º 1: "una instancia dedicada por cliente mientras dura el checkout", sin decir el mecanismo	Un índice por `ventaId` (`ComprasEnCurso`), validado con el `sub` del token. `ServicioDeVentas` es `@Stateful(passivationCapable = false)` con `@StatefulTimeout` de 10 minutos (`ReglasDeNegocio.MINUTOS_RESERVA`)	La identidad queda en un solo mecanismo, el token. Una sesión HTTP solo vale dentro de un WAR y los servicios son WAR separados
Quién vence la reserva	No estaba definido	Un barrido periódico (`VencimientoDeReservas`, `@Singleton` con `@Schedule`) sobre `vence_reserva`	La especificación avisa que `@PreDestroy` puede no ejecutarse; un solo reloj evita dos caminos para lo mismo
`IPagos`	Entrega Parcial N.º 1: `procesarPago`, `confirmarPago`, `reembolsar`, como integración directa con Mercado Pago	`procesarPago` y `obtener`. La pasarela va detrás de `PasarelaDePagos`, con `FabricaDePasarela` eligiendo la implementación; hoy solo existe `PagosFake`	`IPagos` es el componente (guarda y deduplica una sola vez) y la pasarela es lo intercambiable. `confirmarPago` y `reembolsar` dependen del webhook de Mercado Pago, que quedó como diseño (decisión del grupo del 13/10 sobre Mercado Pago real)
`INotificaciones`	`enviarConfirmacion`, `enviarRecordatorio`, `enviarAlertaCancelacion`, "multicanal"	Solo `enviarConfirmacion`, que escribe una línea en el log. La auditoría va aparte, en `IAuditoria`	No hay proveedor de email elegido ni canal definido. Los recordatorios y las alertas no tienen hoy un evento que los dispare
`IFacturacion`	`emitirFactura`, `consultarEstado`, `reintentar` contra ARCA (ex AFIP)	No existe. Facturación es un WAR con solo `GET /health`	Pendiente para el 13/11 (la facturación por SOAP). Ver 19.3
Validación y Reventa	Componentes con interfaz (`IValidacion`, `IReventa`)	Esqueletos con solo `GET /health`	Se priorizó el proceso asincrónico que pedía la Entrega Parcial N.º 2. Quedan como trabajo futuro (sección 22)
Capa de datos	Entrega Parcial N.º 1: `EventoRepository` vía JPA, y "tres capas separadas" en Eventos	Ventas y Pagos tienen `api/`, `negocio/` y `datos/` con Repository. En Eventos, Usuarios y Notificaciones el Session Bean usa `EntityManager` directo: no hay paquete `datos/` (`backend/servicios/eventos/src/main/java/com/palco/eventos/` tiene solo `api/`, `negocio/` y `dominio/`)	Ventas y Pagos se escribieron después y nacieron con la capa separada. Igualar los otros tres está pendiente (sección 22.1)
Verificación de identidad	Entrega Parcial N.º 1: DNI obligatorio para comprar y publicar	Al verificar el email se marca también `identidadVerificada` (desde el 14/10), así que una cuenta registrada puede comprar. No hay un paso aparte de verificación de identidad	El backend no tenía ninguna operación que pusiera ese dato en verdadero, y solo podía comprar la cuenta sembrada. El DNI de cada titular sigue siendo obligatorio en la compra, y en la puerta lo contrasta el staff (diseñado, pendiente con la validación en puerta)

19.2 Integración y mensajería

Tema	Qué se presentó	Qué vale hoy	Motivo
Broker	Entrega Parcial N.º 1 y clase 6: entre los sistemas externos	ActiveMQ Artemis embebido en WildFly, configurado en `backend/servidor/src/main/wildfly/configurar.cli`. Es infraestructura propia; los externos son Mercado Pago y ARCA	Viene con el servidor de aplicaciones y no hay que operar otro proceso
Modelo de integración	Clase 6: bus de integración (ESB) conceptual	REST directo entre servicios, el broker para lo asincrónico y un adaptador por sistema externo	Con ocho servicios y dos externos, un bus suma una pieza más sin resolver un problema que hoy exista
Quién publica	Diagrama de la Entrega Parcial N.º 1: la flecha asincrónica sale de Notificaciones	Publica Ventas (`PublicadorPagoConfirmadoJms`, en `common`), en la misma transacción que confirma la venta. Notificaciones consume	El hecho que dispara todo es la confirmación del pago, que ocurre en Ventas
Emisión de la entrada	Clase 8: "hoy es una llamada síncrona interna"	Va por la cola `cola.emision-entradas`. Las entradas nacen `PENDIENTE_EMISION` en la transacción del pago y `EmisionDeEntradasMDB` las pasa a `VALIDA`	La respuesta del pago no espera la emisión, y la cola garantiza que se procese una sola vez aunque haya varias instancias del consumidor. Que las entradas se creen en la transacción del pago (y no en el MDB) permite que `GET /entradas/{id}` responda apenas vuelve el pago
A quién se avisa	Clase 9: al comprador y al organizador. Clase 8: por email o SMS	Solo al comprador, con una línea en el log (`AvisoDeCompraMDB`)	El mensaje `PAGO_CONFIRMADO` no trae al organizador, y no hay proveedor de email
Garantía entre la venta y el mensaje	No estaba definida	La base no es XA y participa como último recurso; los consumidores son idempotentes (UPDATE condicionales)	La consigna pide transacciones declarativas y explicar las fallas, no un commit en dos fases. Una base XA no está verificada contra Supabase
Facturación con ARCA	Clases 9 y 10: SOAP síncrono con espera de 5 segundos y cola de reintento. La clase 9 dice que es "el que vamos a implementar" para la Entrega Parcial N.º 2	No está implementado: `facturaEstado` vale `NO_APLICA` (`Venta.java:111`). El diseño sigue, con decisiones a aprobar	Se priorizó el proceso con mensajería real que pedía el checkpoint. La consigna exige SOAP con WSDL recién para el 13/11
Por qué la llamada a ARCA es síncrona	Clase 9: "la venta no puede darse por válida sin un CAE autorizado"	Desde la clase 10 la compra se confirma aunque ARCA no responda, porque el pago ya está cobrado. Es síncrona porque SOAP es pedido y respuesta, y se hace dentro de la compra para devolver la factura en la misma respuesta cuando ARCA contesta a tiempo	Bloquear una compra cobrada por una falla de un tercero es peor que facturar después

Tema	Qué se presentó	Qué vale hoy	Motivo
Aviso de Mercado Pago (webhook)	Clase 10: tratado como asincrónico	Se clasifica por su canal, REST; para la compra, el resultado llega de forma asincrónica. Es "Diseñado, pendiente" (`docs/entrega-obligatoria-2/diagramas/diagrama_secuencia_mercadopago.svg`)	La clasificación de la consigna es por mecanismo de integración

19.3 API REST

Tema	Qué se presentó	Qué vale hoy	Motivo
Rutas	Clase 10: todo bajo `/api/v1`	`/api/v1` en Ventas y Pagos (`/api/v1/pagos`). Usuarios, Eventos y Notificaciones siguen en `/api/<servicio>`; Validación, Reventa y Facturación tienen solo `/api/<servicio>/health`	Pasar los servicios que ya funcionaban cambiaba rutas que usan el frontend, Postman, los guiones y la evidencia, y la consigna no lo pide. Queda como posible tarea al poner al día el contrato OpenAPI
Operaciones de Ventas	Catálogo de la clase 10: cuatro operaciones	Además existen `GET /ventas` (las compras de quien pregunta), `GET` y `DELETE /ventas/{id}` y `GET /entradas/{id}`. La venta trae `items`, `entradas`, `subtotal`, `cargoServicio`, `pagoId` y `eventoId`	El frontend necesitaba listar las compras ("Mis entradas") y cancelar desde el checkout
Pago pendiente	Clase 10: el pago responde 202 si queda pendiente	No hay 202: el pago responde aprobado, rechazado (402) o sin respuesta (504)	Con `PagosFake` no hay pagos diferidos. El 202 vuelve con el webhook de Mercado Pago, que está diseñado
Quién ve la entrada	Clase 10: "solo el titular"	La ve el comprador. Una venta o entrada ajena responde 404, igual que una inexistente	El titular puede no tener cuenta en Palco: es una persona con nombre y DNI cargada por el comprador
Idempotencia	Clase 10: el servidor guarda clave → respuesta	La clave (`Idempotency-Key`) se guarda en la venta y se devuelve la venta en su estado actual	Devolver el estado actual es más útil que repetir una respuesta vieja, y no exige otra tabla
Identificadores	Clases 8 y 10: correlativos, como `V-00231`	Un prefijo y 8 caracteres al azar, como `V-e022b63c`	Con H2 en memoria la numeración vuelve a 1 en cada reinicio mientras el broker conserva mensajes viejos, y se podían repetir ids
Formato de los errores	Clase 10, para Eventos: `{estado, mensaje, campos}`, con Problem Details como pendiente	Problem Details (RFC 9457) en todos los servicios: `common/rest/Problema.java` y `ManejadoresDeErrores`	Un solo formato para los ocho servicios, adoptado antes de que Ventas y Pagos sumaran el suyo. El documento de la clase 10 (`actividad_integradora_clase10`) se corrigió el 16/10: dice que Eventos ya responde en Problem Details y que Ventas y Pagos están en `/api/v1`. Desde el 16/10 también salen en Problem Details el 401 por token inválido, que antes emitía el servidor en `text/html`, y el 429 del límite de intentos; el 500 suma el miembro `idError`

Tema	Qué se presentó	Qué vale hoy	Motivo
Contrato OpenAPI	Clase 10: contract-first, `openapi-palco-v1.yaml`	`docs/api/openapi.yaml` se actualizó el 14/10 con lo implementado. Validación, Reventa y Facturación figuran como diseño	La implementación se adelantó al contrato en la primera tanda de trabajo; la diferencia se cerró antes del checkpoint

19.4 Nombres

Los documentos viejos usan un nombre interno para la Entrega Parcial N.º 2 y escriben "AFIP/ARCA". Hoy se usa el nombre de la consigna (Entrega Parcial N.º 2, Entrega Obligatoria N.º 2) y "ARCA (ex AFIP)". Las carpetas que ya existían conservan su nombre (`docs/entrega3/`, `scripts/demo-entrega3.sh`).

20. Qué es real y qué es simulado hoy

Estado leído en el código el 16/10. "Implementado" quiere decir que funciona contra el servidor y tiene evidencia o tests; "Simulado", que funciona con una pieza que reemplaza a un sistema externo; "Diseñado, pendiente", que solo existe en documentos.

Parte	Estado	Detalle
Usuarios: registro, verificación de email, inicio de sesión, JWT RS256, token de servicio	Implementado	El código de verificación no se envía: queda en el log del servidor, porque no hay proveedor de email (simulado)
Eventos: catálogo, alta por el organizador, cupo por sector	Implementado	Sin capa `datos/` separada (pendiente, sección 22.1); tests solo en `ServicioDeEventosTest`
Ventas: reserva stateful, pago, cancelación, vencimiento, idempotencia	Implementado	`facturaEstado` vale siempre `NO_APLICA`
Pagos: API, guardado, deduplicación por clave, control por rol	Implementado	La pasarela es `PagosFake`: `tok_rechazado` rechaza, `tok_sin_respuesta` no contesta y cualquier otro aprueba. No mueve plata
`CobrosFake` en Ventas	Simulado, apagado en el servidor	Existe para trabajar en Ventas sin Pagos (`palco.ventas.cobros=fake`). No está activo en la evidencia
Broker, cola, tópico, reintentos (5 entregas cada 2000 ms) y `DLQ`	Implementado	ActiveMQ Artemis embebido, configurado en `configurar.cli`
Emisión de entradas por cola (`EmisionDeEntradasMDB`)	Implementado	Idempotente: un `PAGO_CONFIRMADO` repetido no emite de nuevo (`EmisionEnH2Test`)
Auditoría por suscripción durable (`AuditoriaDeComprasMDB`)	Implementado	Recibe lo pendiente al volver
Aviso de compra por suscripción no durable (`AvisoDeCompraMDB`)	Simulado	Escribe una línea en el log; no envía email
Endpoints `/api/notificaciones/pruebas/*`	Implementado, solo para desarrollo	Publican mensajes de prueba (`ventaId` que empieza con `prueba-`); van en el WAR que se despliega
Transacción declarativa de `confirmarCompra`	Implementado	`REQUIRED` en la compra, `MANDATORY` en el publicador. La falla a mitad del flujo se probó una vez con un WAR temporal que no quedó en el repo

Parte	Estado	Detalle
Facturación SOAP con ARCA	Diseñado, pendiente	No hay ninguna clase con `@WebService` en `backend/` ni capa `webservices` en `backend/servidor/pom.xml`
Mercado Pago real y su webhook	Diseñado, pendiente	Diagrama de secuencia en `docs/entrega-obligatoria-2/diagramas/`. Los estados `PENDIENTE`, `VENCIDO` y `REEMBOLSADO` existen en `EstadoPago` y nadie los escribe
Validación en puerta	Diseñado, pendiente	Solo `GET /api/validacion/health`
Reventa oficial	Diseñado, pendiente	Solo `GET /api/reventa/health`. Las reglas (tope +10 %, tasa) ya están en `common/ReglasDeNegocio`
Base de datos	Implementado en desarrollo	H2 en memoria en modo PostgreSQL: se vacía al reiniciar WildFly. Las migraciones no se corrieron contra PostgreSQL (Supabase)
Frontend: sesión, catálogo, alta de eventos, compra y "Mis entradas"	Implementado	Llaman al backend (`frontend/src/servicios/`)
Frontend: reventa, puerta y estadísticas del organizador	Simulado	Datos locales (`localStorage` en `frontend/src/store/AppContext.tsx` y datos fijos en `frontend/src/data/`)
Seguridad: roles declarativos, recurso ajeno como 404, contraseñas PBKDF2 (600.000 iteraciones para los hashes nuevos; la migración V5 de Usuarios rehashea las cuentas de prueba)	Implementado	Matriz de la clase 11, tal como se armó: 2 criterios cumplen con test, 4 cumplen en código, 1 parcial, 2 no cumplen y 1 no se puede verificar todavía. Uno de los dos que no cumplían, SEC-W04, ya está implementado (fila siguiente); los veredictos de la matriz se actualizan solo con el test ejecutado
Límite de intentos de login (SEC-W04)	Implementado (16/10)	5 fallidos en 15 minutos para un mismo email → 429 `/problemas/demasiados-intentos` con `Retry-After`, aunque la clave sea correcta y también para emails sin cuenta. Tabla `intento_login` (migración V4, guarda el SHA-256 del email), Session Bean `IntentosDeLogin`; un login correcto borra los fallidos. Probado con `LimiteDeIntentosDeLoginTest` y en vivo el 16/10 (el 6.º intento dio 429 con `Retry-After: 898`)
401 por token inválido en Problem Details	Implementado (16/10)	`web-fragment.xml` y `ErrorDeAutenticacionServlet` en `backend/common`: `application/problem+json` (`/problemas/token-invalido`), `WWW-Authenticate: Bearer error="invalid_token"` y cabeceras CORS. Probado en vivo el 16/10 contra `GET /api/v1/ventas`
Id de error en el 500	Implementado (16/10)	`ManejadoresDeErrores.Inesperado` genera un UUID, lo escribe en el log con la traza y lo devuelve como `idError`. Probado con `ErrorInesperadoTest`; no se provocó un 500 en vivo
Vigencia del token (SEC-W05) y permisos por destino en el broker (SEC-I02)	Diseñado, pendiente	Pendientes de decisión del grupo. El request SEC-W05c de Postman falla a propósito mientras la sesión dure 8 horas

21. Limitaciones conocidas

Cada limitación sale del código o del documento de integración (`docs/entrega3/Entrega_Parcial_2_Integracion.md`). Se ordenan por impacto.

N.º	Limitación	Impacto	Cómo se resolvería	Tarea
L1	**Un cobro sin resultado cuya compra ya no tiene instancia queda retenido.** Después de un 504 de Pagos o una falla inesperada al confirmar, la venta queda `EN_CHECKOUT` con `clave_pago`. Si la instancia stateful muere (10 minutos sin llamadas o un redespliegue), pagar y cancelar responden 409 `operacion-en-curso` y el barrido la saltea	La venta y su cupo quedan tomados hasta resolverlo a mano. Es a propósito: Ventas no sabe si Pagos cobró, y liberarla podría vender dos veces un lugar ya cobrado	Una conciliación que le pregunta a Pagos por esa clave y confirma la venta o libera el cupo. Hoy se resuelve a mano; Pagos no tiene consulta por clave ni por venta	Fallas a mitad del flujo (Sebastian)
L2	**La devolución de cupo no se reintenta.** Al cancelar o vencer, Ventas le devuelve el cupo a Eventos una sola vez; si falla, queda un `WARNING` y nada en la base lo recuerda	Esos lugares quedan sin vender. Nunca se vende uno dos veces	Una marca de devolución pendiente en la venta que el barrido revisa hasta que Eventos acepte, con una devolución segura de repetir	Fallas a mitad del flujo (Sebastian)
L3	**La caída entre el commit de la base y el del broker no está probada**	Con la base como último recurso, una caída en ese instante podría dejar la venta confirmada sin su mensaje, o al revés	Se documenta como no probada. Una tabla de salida (outbox) con un proceso que publica lo cerraría	Fallas a mitad del flujo (Sebastian) la documenta
L4	**Redesplegar Ventas corta las compras en curso.** El índice y los titulares viven en memoria	Las compras en curso se pueden consultar y cancelar, pero pagar responde 409 `reserva-vencida`	Aceptado: la reserva dura 10 minutos. Persistir los titulares en la reserva permitiría retomarla	—
L5	**La base de desarrollo es H2 en memoria.** Las migraciones no se corrieron contra PostgreSQL	Todo se borra al reiniciar WildFly; un error de SQL en Supabase aparecería recién al conectarla	Conectar Supabase y correr las migraciones	Fase 13 de `TODO.md`, sin dueño
L6	**El secreto de servicios todavía tiene un valor de desarrollo escrito en el repo** (`palco-desarrollo`, `TokenDeServicio.java:57`), y el servidor arranca con él si no hay otro. El servidor de desarrollo del grupo ya arranca con un `PALCO_SECRETO_SERVICIOS` propio en `backend/.env` (que no se versiona), y los guiones y Postman lo toman de ahí	Fuera de desarrollo, un servidor sin la variable aceptaría el secreto público y quien lea el repo podría pedir un token de servicio	Que fuera de desarrollo el arranque falle sin la variable: propuesto, no implementado	Seguridad de la clase 11
L7	**La sesión dura 8 horas sin refresh token ni revocación** (SEC-W05, "parcial")	Un token robado vale hasta 8 horas; cerrar sesión no lo invalida. El request SEC-W05c de Postman falla por esto, a propósito	Bajar la vigencia con refresh revocable, o justificar las 8 horas. Pendiente de decisión del grupo	Vigencia de la sesión (Joaquin), a decidir en grupo

N.º	Limitación	Impacto	Cómo se resolvería	Tarea
L8	**El broker no tiene permisos por destino** (SEC-I02, "no cumple")	Cualquier conexión puede publicar o consumir en cualquier cola o tópico	Un usuario por servicio con `security-setting` por destino en `configurar.cli`. Hay una propuesta, sin nada aplicado	Permisos del broker (Julian), a decidir en grupo
L9	**El DNI del titular viaja completo en el mensaje** y queda en la columna `cuerpo` de auditoría (la consulta no lo devuelve)	Un dato sensible queda guardado fuera de Ventas	Mandar solo ids y que quien necesite el DNI lo pida	Fase 12 de `TODO.md`
L10	**Los endpoints `/pruebas/*` van en el WAR que se despliega**	Cada prueba deja un aviso en el log, una fila de auditoría con `ventaId prueba-…`, una copia en la cola y, si es inválido, un mensaje en la `DLQ`	Habilitarlos con una variable de entorno (404 si falta) y excluir de la auditoría las filas de prueba	Sin tarea
L11	**Pagos y la clave de idempotencia.** Tras un rechazo, la misma clave devuelve siempre ese rechazo; dos pedidos simultáneos con la misma clave llaman dos veces a la pasarela; si la pasarela cobra y falla el guardado, el pago queda sin registro hasta el reintento	Con `PagosFake` no hay impacto. Con Mercado Pago dependería de que deduplique por la clave	Guardar el pedido antes de llamar a la pasarela y deduplicar con Mercado Pago por su clave de idempotencia	Mercado Pago real (Valentina)
L12	**El monto que recibe Pagos se convierte antes de validarse**: el conversor de JSON trunca decimales y recorta números que no entran en un `long`	Pagos confía en Ventas, su único cliente. Un cliente nuevo (Reventa) podría cobrar otro monto	Recibir el monto como `BigDecimal` y rechazar lo que no sea entero	A decidir por el grupo, antes de Reventa
L13	**Dos pedidos simultáneos con la misma clave reservan los dos** en Ventas	Si queda lugar para uno, el otro recibe 409 `cupo-agotado`; al repetirlo recibe la venta	Aceptado	—
L14	**El listado de auditoría no ordena dentro del mismo segundo** (`recibido_en` truncada a segundos)	Tras una caída de la auditoría, varias filas quedan con la misma hora	Una columna de orden en una migración nueva	Sin tarea
L15	**Sin probar bajo carga:** más de 80 compras o 60 pagos a la vez, el arranque en frío de WildFly, Eventos o Usuarios caídos (solo con tests)	Desconocido	Pruebas de carga y de caída en el servidor	Fase 11 de `TODO.md`
L16	**Capas sin separar en Eventos, Usuarios y Notificaciones**, y Eventos con pocos tests (un archivo, `ServicioDeEventosTest`, con 8 casos)	La consigna pide capas explícitas en cada componente	Repository en los tres, ampliando los tests de Eventos antes del cambio	Capas explícitas (Sebastian)
L17	**El límite de intentos de login es por email**	Quien conozca el email de otra persona puede bloquearle el login 15 minutos con 5 intentos fallidos	Contar también por IP. Aceptado para esta etapa	Sin tarea

22. Trabajo futuro

Ordenado por importancia: primero lo que la consigna exige para la Entrega Obligatoria N.º 2 (13/11) y hoy no existe; después, lo que hace falta para el sistema completo del 27/11 (Prueba del TP Obligatorio). Los responsables y las fechas salen del reparto del grupo.

22.1 Para la Entrega Obligatoria N.º 2 (13/11)

Orden	Qué falta	Por qué es prioritario	Tarea	Responsable	Fecha de cierre prevista
1	Servicio SOAP con su WSDL: un sistema legado simulado que hace de ARCA, `ServicioDeFacturación` como cliente y una factura por compra confirmada. Después, espera de 5 segundos y `cola.reintento-facturacion`	La consigna lo exige y no existe nada. Si la cola de reintento se implementa, suma un segundo proceso asincrónico por cola	Facturación por SOAP	Santiago (revisa Julian)	28/10; punto de control el 23/10
2	Documento técnico de 8 a 12 páginas con un diagrama de secuencia por integración	La consigna lo exige y no existe. Cada diagrama lo dibuja quien defiende esa integración	Documento técnico	Joaquin integra; cada dueño su diagrama	Borrador el 31/10; PDF el 11/11
3	Prueba repetible de la falla a mitad del flujo, conciliación de cobros sin resultado (L1) y reintento de la devolución de cupo (L2)	La defensa pregunta qué pasa con la transacción si algo falla a mitad del flujo	Fallas a mitad del flujo	Sebastian	28/10
4	Validación en puerta: `ServicioDeValidación` stateless, escaneo con uso único atómico para el rol `staff`	Es el séptimo componente y cubre el mínimo de seis si la facturación por SOAP se atrasa	Validación en puerta	Joaquin	25/10
5	Capas explícitas (Repository) en Eventos, Usuarios y Notificaciones, y más tests de Eventos	La consigna pide capas explícitas en cada componente	Capas explícitas	Sebastian	05/11 (línea de corte)
6	`openapi.yaml` al día con Facturación y Validación	Entra en el paquete de diseño del 06/11	Contrato OpenAPI	Santiago	04/11
7	Seguridad de la clase 11, lo que queda: decidir en grupo la vigencia del token (SEC-W05, L7) y los permisos por destino en el broker (SEC-I02, L8), e implementar lo que se decida; que el arranque falle sin secreto de servicios fuera de desarrollo (L6). El límite de intentos de login, el 401 en Problem Details y el `idError` del 500 ya están implementados y probados (16/10), falta commitearlos	SEC-I02 "no cumple" y SEC-W05 es "parcial"; el request SEC-W05c de Postman sigue en rojo hasta decidir	Seguridad de la clase 11: vigencia de la sesión, permisos del broker y secreto de servicios	Joaquin (vigencia de la sesión), Julian (permisos del broker)	Sin fecha: depende de la decisión del grupo

Orden	Qué falta	Por qué es prioritario	Tarea	Responsable	Fecha de cierre prevista
8	Pantalla de puerta conectada a Validación	Muestra la validación en puerta desde el frontend	Pantalla de puerta	Valentina	03/11

22.2 Para el sistema completo (27/11)

Orden	Qué falta	Por qué	Tarea	Responsable	Fecha de cierre prevista
1	Reventa oficial: publicar, retirar, listar y comprar, con tope de +10 %, mínimo de 50 % y cierre 3 horas antes; cobro por Pagos, cambio de titular en Ventas e invalidación del QR por Validación	Es el octavo componente y lo que distingue al producto	Reventa oficial	Julian	17/11 (para el checkpoint del 20/11)
2	Pantallas de reventa y panel del organizador contra el backend	Hoy usan datos locales ("Mis entradas" ya lista las compras desde Ventas, `listarCompras` en `frontend/src/servicios/ventas.ts`)	Pantallas de reventa y del organizador	Valentina	Reventa el 19/11; panel el 24/11
3	Documentación final de 10 a 15 páginas	La exige la Entrega Final	Documentación final	Joaquin integra, con los cinco	25/11
4	Backend del panel del organizador (estadísticas)	El panel no tiene backend ni dueño (Fase 9 de `TODO.md`)	Sin tarea asignada	A definir	—
5	Mercado Pago real con su webhook	Decidido el 13/10 como "Diseñado, pendiente": el requisito REST lo cubre la API propia. Se retoma solo si la cátedra pide una pasarela real	Mercado Pago real	Valentina	Sin fecha
6	Base en PostgreSQL (Supabase), CI y hosting	La base de desarrollo se borra al reiniciar (L5)	Fase 13 de `TODO.md`	A definir	—
7	Aviso de compra por email	Hoy es una línea en el log. Falta elegir proveedor y pedirle a Usuarios el email del comprador	Aviso por email	Julian, solo si sobra tiempo	Sin fecha
8	Pruebas de frontend y de punta a punta (Vitest, Playwright)	Hoy no hay ninguna (Fase 11 de `TODO.md`)	Sin tarea asignada	A definir	—

Aprendizajes del equipo

Sección a completar por el grupo.

Declaración de uso de herramientas de IA generativa

Sección a completar por el grupo.

Anexos

A. Endpoints de la API

Generado desde los recursos JAX-RS y el `context-root` de cada WAR del commit `725b0ba`. El contrato completo, con cuerpos y errores, está en `docs/api/openapi.yaml`.

Servicio	Método	Ruta	Rol
eventos	GET	`/api/eventos/health`	público (sin anotación de rol)
eventos	GET	`/api/eventos`	público (@PermitAll)
eventos	GET	`/api/eventos/{slug}`	público (@PermitAll)
eventos	POST	`/api/eventos`	organizer
eventos	POST	`/api/eventos/{eventoId}/sectores/{sectorId}/cupo`	servicio
facturacion	GET	`/api/facturacion/health`	público (sin anotación de rol)
notificaciones	GET	`/api/notificaciones/health`	público (sin anotación de rol)
notificaciones	POST	`/api/notificaciones/pruebas/pago-confirmado`	servicio
notificaciones	POST	`/api/notificaciones/pruebas/pago-confirmado-con-falla`	servicio
notificaciones	POST	`/api/notificaciones/pruebas/mensaje-invalido`	servicio
notificaciones	GET	`/api/notificaciones/auditoria`	servicio
pagos	GET	`/api/v1/pagos/health`	público (sin anotación de rol)
pagos	POST	`/api/v1/pagos`	servicio
pagos	GET	`/api/v1/pagos/{pagoId}`	servicio
reventa	GET	`/api/reventa/health`	público (sin anotación de rol)
usuarios	GET	`/api/usuarios/health`	público (sin anotación de rol)
usuarios	POST	`/api/usuarios/registro`	público (@PermitAll)
usuarios	POST	`/api/usuarios/verificacion`	público (@PermitAll)
usuarios	POST	`/api/usuarios/verificacion/reenvio`	público (@PermitAll)
usuarios	POST	`/api/usuarios/login`	público (@PermitAll)
usuarios	GET	`/api/usuarios/yo`	user, organizer, staff
usuarios	POST	`/api/usuarios/servicios/token`	público (@PermitAll)
usuarios	GET	`/api/usuarios/jwks`	público (@PermitAll)
validacion	GET	`/api/validacion/health`	público (sin anotación de rol)
ventas	GET	`/api/v1/health`	público (sin anotación de rol)
ventas	GET	`/api/v1/entradas/{entradaId}`	user
ventas	POST	`/api/v1/ventas`	user

Servicio	Método	Ruta	Rol
ventas	GET	`/api/v1/ventas`	user
ventas	GET	`/api/v1/ventas/{ventaId}`	user
ventas	POST	`/api/v1/ventas/{ventaId}/pagos`	user
ventas	DELETE	`/api/v1/ventas/{ventaId}`	user

B. Contrato del mensaje PAGO_CONFIRMADO

El contrato completo está en `docs/api/mensajeria-pago-confirmado.md`. Lo publica `PublicadorPagoConfirmadoJms` (`backend/common`) dentro de la transacción que confirma la venta, con la propiedad `tipoEvento = PAGO_CONFIRMADO`, a la cola `cola.emision-entradas` y al tópico `topico.compra-confirmada`. La sección de mensajería explica sus consumidores.

C. Glosario

Término	Significado en este documento
Venta	Una compra de un comprador para un evento. Tiene estados: `EN_CHECKOUT` (reserva tomada), `CONFIRMADA` (pago aprobado), `EMITIDA` (entradas emitidas), `CANCELADA` y `VENCIDA`. Su id tiene la forma `V-e022b63c`
Entrada nominativa	Entrada ligada a una persona con nombre y DNI. La identidad se contrasta en la puerta
Titular	La persona cuyo nombre y DNI figuran en la entrada. Puede no ser el comprador ni tener cuenta en Palco
Comprador	Usuario con rol `user` que inicia y paga la venta. Es quien puede consultar la venta y sus entradas
Organizador	Usuario con rol `organizer`: da de alta eventos
Staff	Usuario con rol `staff`: valida entradas en la puerta
Rol `servicio`	Rol del token que un servicio pide a Usuarios para llamar a otro servicio (por ejemplo, Ventas a Eventos y a Pagos). Dura 5 minutos
Sector	Parte de un evento con su precio y su capacidad
Cupo	Lugares disponibles de un sector. Eventos lo descuenta al reservar y lo devuelve al cancelar o vencer
Reserva	El cupo tomado al iniciar una compra. Dura 10 minutos (`ReglasDeNegocio.MINUTOS_RESERVA`); si no se paga, el barrido la vence
Checkout	El proceso de compra en pasos (titulares, pago, confirmación). En el backend lo mantiene el componente stateful `ServicioDeVentas`
Stateful / stateless	Un componente stateful guarda estado de un cliente entre llamadas (la compra en curso); uno stateless no recuerda nada entre llamadas y el servidor atiende con un pool de instancias
Barrido	Tarea periódica (`VencimientoDeReservas`) que vence las reservas no pagadas y devuelve su cupo
Reventa oficial	Venta de una entrada entre usuarios dentro de Palco, con tope de precio. Cambia el titular e invalida el QR anterior. Diseñada, pendiente
Tope	Precio máximo de reventa: el precio pagado más 10 % (`ReglasDeNegocio.TOPE_REVENTA_PORCENTAJE`)

Término	Significado en este documento
Cargo por servicio	Lo que cobra Palco sobre el subtotal de la venta: 10 % (`TASA_SERVICIO_PORCENTAJE`)
Emisión	Paso de las entradas de `PENDIENTE_EMISION` a `VALIDA`, con su código QR. La hace `EmisionDeEntradasMDB` desde la cola
`PAGO_CONFIRMADO`	El mensaje que publica Ventas cuando se aprueba el pago. Su contrato está en `docs/api/mensajeria-pago-confirmado.md`
Broker	Servidor de mensajería. Acá, ActiveMQ Artemis embebido en WildFly
Cola	Destino de mensajería donde cada mensaje lo procesa un solo consumidor (`cola.emision-entradas`)
Tópico	Destino de mensajería donde cada suscripción recibe su copia del mensaje (`topico.compra-confirmada`)
MDB	Message-Driven Bean: componente del servidor que consume mensajes de una cola o un tópico
Suscripción durable	Suscripción a un tópico que el broker conserva mientras el consumidor está caído: al volver recibe lo pendiente (auditoría). Una no durable pierde lo que llegó mientras no estaba (aviso)
Selector	Filtro del consumidor sobre las propiedades del mensaje (`tipoEvento = 'PAGO_CONFIRMADO'`)
DLQ	Dead Letter Queue: cola donde el broker deja un mensaje que falló en todas sus entregas (5, cada 2000 ms)
Idempotencia	Propiedad de una operación que, repetida, deja el mismo resultado. En la API se pide con la cabecera `Idempotency-Key`; en los consumidores se logra con UPDATE condicionales
UPDATE condicional	Sentencia que cambia una fila solo si sigue en el estado esperado (por ejemplo, `where estado = 'EN_CHECKOUT'`). Evita aplicar dos veces el mismo cambio
Transacción declarativa	Transacción que define el contenedor según una anotación (`REQUIRED`, `MANDATORY`), sin código de commit ni rollback
Último recurso	Forma en que un recurso no XA (la base) participa en una transacción junto con uno XA (el broker): se confirma al final. Si el servidor cae entre los dos commits, el resultado puede quedar a medias
XA / commit en dos fases	Protocolo que confirma varios recursos a la vez de forma atómica. No se usa
Conciliación	Proceso que compara el estado de una venta con el de su pago en Pagos para cerrarla. Diseñado, pendiente
Problem Details	Formato estándar de errores HTTP (RFC 9457), con `type`, `title`, `status`, `detail` e `instance`, en `application/problem+json`
JWT	Token firmado (RS256) que lleva la identidad (`sub`) y los roles (`groups`) del usuario. Los servicios lo validan con la clave pública de `/api/usuarios/jwks`
Pasarela de pago	Sistema externo que cobra. Hoy es `PagosFake` (simulada); Mercado Pago está diseñado
Webhook	Aviso que la pasarela manda por REST cuando un pago cambia de estado. Diseñado, pendiente
SOAP / WSDL	Protocolo de servicios web basado en XML y el documento que describe su contrato. Lo usa ARCA. Pendiente
ARCA (ex AFIP)	Organismo que autoriza las facturas electrónicas y devuelve un CAE
CAE	Código de Autorización Electrónico con que ARCA autoriza una factura
Sistema legado	Sistema existente con tecnología previa (SOAP) que hay que integrar sin cambiarlo. En Palco lo representa ARCA

Término	Significado en este documento
Adapter, Factory, Repository, Facade	Patrones aplicados: Adapter traduce una interfaz ajena a una propia (`ClienteDePagos`; `MercadoPagoPagos`, diseñado); Factory elige una implementación por configuración (`FabricaDePasarela`, `FabricaDeCobros`); Repository aísla el acceso a la base (Ventas y Pagos); Facade ofrece una entrada simple a un subsistema (`ComprasEnCurso`)
Health	Endpoint `GET .../health` que responde 200 si el servicio está desplegado
WAR	Archivo con el que se despliega cada servicio en WildFly. Hay ocho, uno por componente

D. Documentos del repo

Documento	Qué contiene
`docs/consigna/`	La consigna de la cátedra
`docs/entrega1/`	Entrega Parcial N.º 1: arquitectura y diagrama de capas
`docs/entrega3/`	Entrega Parcial N.º 2: documento de integración, diagrama SOA y evidencia
`docs/entrega-obligatoria-2/`	Material para el 13/11: diagrama de secuencia de Mercado Pago
`docs/clases/`	Talleres de las clases 6, 8 y 9
`actividad_integradora_clase10/`, `actividad_integradora_clase11/`	Actividades de las clases 10 (integración) y 11 (seguridad)
`docs/api/`	OpenAPI, contrato del mensaje PAGO_CONFIRMADO, colección de Postman y su guía
`TODO.md`	Estado del proyecto por fases y calendario de entregas
`README.md`, `scripts/LEEME.md`	Cómo levantar y probar el sistema, y los guiones de prueba y demo